

EXEMPLAR DE PREZENTARE

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII



CEZARA BENEGUI · ANA-CRISTINA IOVA
RODICA SMÎNTÎNĂ · ANDREI PĂUN
CIPRIAN MIHAI CEAUȘESCU
PAUL ENACHE

Informatică

Curriculum de
specialitate (CS),
filiera teoretică,
profilul real, specializarea
matematică-informatică –
clasa a IX-a

9

CORINT
LOGISTIC

Informatică

Curriculum de
specialitate (CS),
filiera teoretică,
profilul real, specializarea
matematică-informatică –
clasa a IX-a

9

CEZARA BENEGUI · ANA-CRISTINA IOVA
RODICA SMÎNTÎNĂ · ANDREI PĂUN
CIPRIAN MIHAI CEAUȘESCU
PAUL ENACHE

Date despre autori:

CEZARA BENEGUI – lector universitar doctor la Facultatea de Matematică și Informatică a Universității din București, doctor în Informatică, cu activitate de cercetare în inteligența artificială și publicații științifice naționale și internaționale. A primit titlul de Profesor Bologna, distincție acordată de Alianța Națională a Organizațiilor Studentești din România (ANOSR) cadrelor didactice apreciate de studenți. Parcursul profesional reunește experiența academică și cea din industria IT, inclusiv în dezvoltare software. Este expertă în baze de date și dezvoltare software, discipline pe care le predă de peste un deceniu la Facultatea de Matematică și Informatică a Universității din București. A primit Premiul Senatului Universității din București pentru cel mai inovator curs și cea mai inovatoare metodă de predare, în cadrul disciplinei Dezvoltarea Aplicațiilor Web. În paralel, activează și în domeniul consilierii pentru dezvoltare personală și orientare în carieră, sprijinind tinerii să își clarifice direcția și să își construiască un parcurs autentic, în acord cu propriile valori. Crede într-o educație care depășește transmiterea de informații și creează un spațiu în care fiecare tânăr se simte văzut, înțeles și încurajat să-și descopere potențialul. Promovează o educație care formează nu doar competențe tehnice, ci și curaj, responsabilitate și autonomie.

ANA-CRISTINA IOVA – lector universitar doctor la Universitatea din București și cercetător în domeniul informaticii. Activitatea sa îmbină pasiunea pentru educație cu cercetarea în programare funcțională, algoritmi evolutivi și testare software. Activitatea sa de cercetare s-a concretizat în publicarea unor lucrări științifice în reviste și volume de specialitate naționale și internaționale. Teza sa de doctorat a fost distinsă cu Premiul Senatului Universității din București pentru cea mai bună teză în domeniul științelor exacte. Prin acest manual contribuie la dezvoltarea competențelor digitale, rezolvare de probleme și utilizare responsabilă a tehnologiei.

RODICA SMÎNTÎNĂ – profesor gradul I, titular la Colegiul Național „Gheorghe Șincai” din București și are o experiență de peste 35 de ani în învățământul preuniversitar. Parcursul său profesional este completat de 15 ani în calitate de metodist al ISMB, 17 ani de coordonare a practicii pedagogice a studenților Facultății de Matematică și Informatică, Universitatea din București și formator național pe programele oferite de Oracle Academy. Din anul 2019 este membru în cadrul Comisiei de asigurare a omologării mijloacelor de învățământ în vederea utilizării lor în sistemul de învățământ preuniversitar. Coautoare la următoarele culegeri: „Teste admitere Universitatea Politehnică București” (2019), „Informatică – breviar teoretic, exerciții și teste de evaluare pentru bacalaureat, modele pentru teză” (2015), „Informatică Atestat – subiecte rezolvate” (2012, 2015, 2020), „Competențe digitale – bacalaureat – 50 de variante pentru proba de competențe digitale” (2015).

ANDREI PĂUN – profesor al Universității din București, a fost directorul școlii doctorale de Informatică a UB (2019-2024), iar din 2024 a acceptat poziția de director al Institutului de Cercetare pentru Inteligență Artificială „Mihai Drăgănescu” al Academiei Române. A predat cursuri de programare POO în calitate de lector/conferențiar universitar la Louisiana Tech University, SUA. De asemenea, predă cursul de Programare Orientată pe Obiecte la Facultatea de Matematică și Informatică a Universității din București de peste 15 ani. În anul 2011 a primit titlul de Profesor Bologna din partea asociațiilor studentești (ANOSR), la propunerea studenților de la cursul de Programare Orientată pe Obiecte (la acel moment 2 profesori din FMI dețineau această distincție). Din 2021, acesta este expert permanent în comisia C1 (Științe exacte și științele naturii) a ARACIS.

CIPRIAN-MIHAI CEAUȘESCU – asistent universitar doctorand (din anul 2023) la Facultatea de Matematică și Informatică a Universității din București, cu activitate de cercetare în domeniul vederii artificiale aplicată în domeniul medical. Este câștigătorul unei burse ANIS pentru Laboratorul de „Învățare Automată” (<https://unibuc.ro/cadre-didactice-si-doctoranzi-universitatii-din-bucuresti-printre-castigatorii-burselor-anis-premiile-acordate-in-cadrul-galei-bursele-anis-joi-7-noiembrie-2024/>). A predat cursuri de informatică și robotică în sistem Cambridge (IGCSE) la Cambridge School of Bucharest (2020-2023) având și diploma de QTS (profesor în sistem britanic din noiembrie 2021). Îmbină activitatea educațională cu cercetarea în domeniul inteligenței artificiale, participând constant la conferințe internaționale și proiecte care susțin dezvoltarea tehnologică și formarea noilor generații (<https://sites.google.com/view/ciprian-ceausescu/activity>).

PAUL ENACHE – profesor gradul I la Liceul Teoretic „Jean Monnet” din București. Având o dublă perspectivă – 20 de ani la catedră și 15 ani în industria IT ca programator/software developer –, aduce în sala de clasă conexiunea directă dintre teorie și tehnologia reală. Dedicat performanței, a ghidat generații de elevi în centre de excelență renumite, precum „Hai la Olimpiadă” și „Nerdvana/Nerdvana 2.0”, și a mentorat viitorii profesori prin coordonarea practicii pedagogice în parteneriat cu POLITEHNICA București. Implicat activ în comisiile de specialitate municipale și naționale, autorul este un promotor al modernizării educației.

Cuprins

Ghid de utilizare a manualului.....	5
Competențe generale și specifice conform Programei școlare	7
Evaluare inițială.....	8
CAPITOLUL 1 Strategii de rezolvare a problemelor	
1.1. Principii de elaborare a unui program.....	10
1.1.1. Noțiuni de bază	10
1.1.2. Modelul Intrare – Prelucrare – Ieșire	11
1.1.3. Gândirea computațională și proiectarea soluției	11
1.1.4. Etapele elaborării unui program și rolul fiecăreia.....	12
1.1.5. Reprezentarea algoritmilor	13
1.1.6. Introducere în limbajul Python.....	14
1.1.7. Citirea și afișarea datelor	16
1.1.8. Expresii și operatori	17
1.1.9. Structuri fundamentale de control.....	18
1.1.10. Urmărirea execuției algoritmilor și identificarea erorilor.....	21
1.1.11. Eficiența algoritmilor – abordare intuitivă.....	22
1.1.12. Proiectarea modulară a algoritmilor	24
1.2. Prelucrări ale datelor numerice	26
1.2.1. Operații cu cifrele unui număr.....	26
1.2.2. Divizori, multipli și CMMDC (cel mai mare divizor comun)	29
1.2.3. Prelucrarea numerelor și descompunerea în factori primi	33
1.2.4. Transformarea numerelor între baze de numerație	36
1.3. Recapitulare	39
1.4. Evaluare sumativă	41
CAPITOLUL 2 Tehnici de organizare a codului în limbaj de programare	
2.1. Subprograme	44
2.1.1. Subprograme – rol, caracteristici.....	44
2.1.2. Subprograme definite de utilizator – implementare în limbajul Python	47
2.1.3. Variabile locale, variabile globale	54
2.1.4. Funcții predefinite Python	57
2.2. Introducere în Programarea Orientată pe Obiecte	60
2.2.1. Motivația utilizării unui limbaj de programare orientat pe obiecte precum Python	60
2.2.2. Ce este Programarea Orientată pe Obiecte?	60
2.2.3. Programarea procedurală vs. Programarea Orientată pe Obiecte (POO)	62
2.2.4. Clase predefinite în Python	66
2.3. Recapitulare	68
2.4. Evaluare sumativă	69

CAPITOLUL 3 Memorarea și prezentarea datelor în limbaj de programare

3.1. Fișiere text	72
3.1.1. Caracteristici, principii de lucru	72
3.1.2. Implementarea fișierelor text în limbajul de programare Python	73
3.2. Biblioteca Tkinter pentru interfețe grafice	81
3.2.1. Motivația utilizării interfețelor grafice	81
3.2.2. Introducere. Ce este Tkinter și de ce îl folosim?	81
3.2.3. Componente de bază	83
3.2.4. Organizarea elementelor în fereastră: pack, grid, place	90
3.2.5. Variabile Tkinter: StringVar, IntVar	92
3.2.6. Evenimente: reacții la taste/click (bind)	93
3.2.7. Probleme rezolvate	94
3.3. Recapitulare	99
3.4. Evaluare sumativă	100

CAPITOLUL 4 Liste

4.1. Modelul conceptual liniar	102
4.1.1. Introducere	102
4.1.2. Lista	103
4.1.3. Stiva	105
4.1.4. Coadă	107
4.2. Clasa list din Python	110
4.2.1. Crearea unei liste	110
4.2.2. Indexarea elementelor din listă. Parcurgerea unei liste	110
4.2.3. Operatori	111
4.2.4. Metode ale clasei „list”	113
4.2.5. Funcții predefinite pentru liste	117
4.2.6. Stive și cozi	120
4.3. Metode de generare sistematică a elementelor unei liste	123
4.4. Metode de sortare	127
4.4.1. Introducere	127
4.4.2. Sortarea prin Selecție – Selection Sort	128
4.4.3. Metoda Bulelor (Bubble Sort)	131
4.4.4. Selection Sort vs. Bubble Sort	135
4.4.5. Sortare cu liste de frecvență	136
4.5. Recapitulare	139
4.6. Evaluare sumativă	142
Recapitulare finală	143
Evaluare finală	145

Soluții (selectiv)	147
Bibliografie	148

Ghid de utilizare a manualului

Manualul cuprinde variantele tipărită și digitală



Simboluri folosite
în varianta digitală

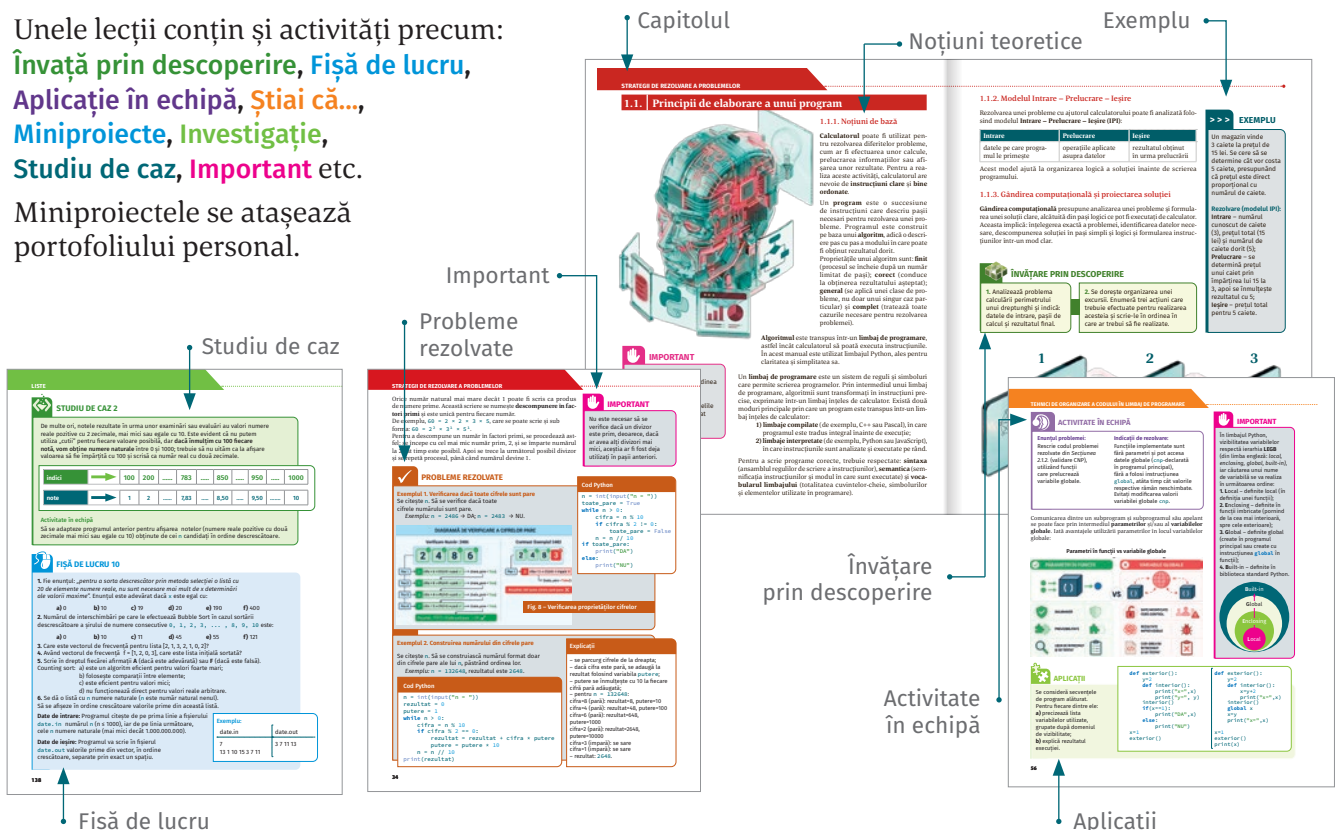


Manualul **Informatică**, curriculum de specialitate (CS), filiera teoretică, profilul real, specializarea matematică-informatică pentru clasa a IX-a este structurat în patru capitole, care respectă domeniile și conținuturile existente în programa școlară. Lecțiile sunt însoțite de un număr considerabil de exemple, care determină formarea competențelor specifice cu care acestea sunt corelate. Manualul cuprinde un număr mare de **probleme rezolvate**, probleme propuse și teste de evaluare/autoevaluare. La finalul manualului se regăsesc răspunsuri la testele de evaluare.

Unele lecții conțin și activități precum:

Învață prin descoperire, **Fișă de lucru**,
Aplicație în echipă, **Știi că...**,
Miniproiecte, **Investigație**,
Studiu de caz, **Important** etc.

Miniproiectele se atașează
portofoliului personal.



Recapitulare

2.3. Recapitulare

1. Scrie un program în Python care citește de la tastatură soldul unui client, al unei bănci comerciale, și definește două funcții prin care se determină soldul clientului, după depunerea unei sume în cont, respectiv o funcție care determină soldul clientului după retragerea unei sume din cont (dacă este posibil). Testează cele două funcții în programul principal.
2. Definește o funcție Python care să rezolve ecuația de gradul al doilea cu coeficienți reali, citiți de la tastatură. Funcția primește ca parametri coeficienții ecuației, iar apelul acesteia se va realiza în programul principal.
3. Se citește de la tastatură un număr natural (n - natural, $1 \leq n \leq 10$). Să se verifice dacă numărul format cu prima cifră a fiecărui număr citit este palindrom. Pentru rezolvare se vor defini cel puțin două funcții.
4. Se consideră următoarea clasă:

```
class Dreptunghi:
    def __init__(self, lungime, latime):
        self.lungime = lungime
        self.latime = latime

    def calculeaza_perimetrul(self):
        return self.lungime + self.latime
```

- a. Identifică atributele și metodele clasei.
- b. Ce reprezintă parametrul self?
- c. Definește metoda calculeaza_perimetrul() pentru un dreptunghi.
- d. Creează o instanță a clasei Dreptunghi și afișează mesajul Adevărat dacă perimetrul dreptunghiului este mai mare decât aria acestuia, altfel afișează mesajul Fals.
- e. Pentru obiectul instanțat anterior, verifică dacă este pătrat (lungimea este egală cu lățimea).
- f. Creează metoda calculeaza_diagonala(self) care returnează valoarea diagonalei (folosind teorema lui Pitagora).
- g. Pentru obiectul instanțat anterior, afișează pe ecran lungimea diagonalei.

Miniproiect

Miniproiecte – activitate în echipă de 2 elevi

1. Proiectează o aplicație modulară Python, de tip calculator matematic, pentru cele patru operații de bază: +, -, *, /, aplicate pe date numerice de tip real. Veți implementa un meniu care să permită selecția, în mod repetat, a operației dorite. Cerințe:
 - a) descompune problema în cel puțin 4 subprograme și stabilește specificațiile fiecărui subprogram: noi, parametri, valori returnate; realizează proiectarea modulară în limbaj pseudocod;
 - b) implementează, în limbajul Python, soluția problemei.
2. Se consideră următorul scenariu: știind datele a doi elevi (nume, clasă, nota 1 și nota 2), determină media notelor pentru fiecare elev, precum și numele elevului cu media cea mai mare. Cerințe:
 - a) implementează soluția problemei în variantă procedurală;
 - b) implementează soluția problemei în variantă POO (Programare Orientată pe Obiecte).

RECAPITULARE

2.4. Evaluare sumativă

Evaluare

I. Rezolvă:

- a. Este o variabilă specială care memorează valori reale. 0,5 p.
- b. Este un bloc de instrucțiuni care poate fi apelat de mai multe ori.
- c. Este o structură repetitivă.
- d. Este un tip de date numeric.

2. Stabilește valoarea de adevăr a următoarelor afirmații (Adevărat sau Fals).

- a. O funcție poate să nu aibă parametri. 0,25 p. x4=1 p.
- b. Definiția unei funcții trebuie să conțină instrucțiunea return.
- c. O funcție poate să returneze mai multe valori.
- d. O variabilă locală poate fi accesată în programul principal. 0,5 p.

3. Rezultatul execuției programului este:

```
def f(a, b=2, c=3):
    return a + b + c
print(f(1, c=5))
```

- a) 6 b) 8 c) 9 d) Eroare 0,5 p.

4. Stabilește rezultatul execuției programului:

```
nr=3
def f(nr):
    return >nr
print(f(nr), f(1))
```

5. Alege două dintre clasele date și completează, pentru fiecare dintre ele, cel puțin 3 atribute și cel puțin 3 metode. Exemple de clase: Telefon, Bicicletă, Profesor, Cont bancar, Animal de companie. 1 p.

6. Completează spațiile libere, astfel încât:

0,25 p. x2=0,5 p.

```
#1)
class Clasa1:
    x=10
    a1=...
    print(.....)

#2)
x = 200
def functia1():
    x = 200
    functia1()
    print(x)
```

- să definești a1 ca obiect al clasei Clasa1 utilizând obiectul a1,
- să afișezi valoarea lui x

Programul să afișeze valoarea 200

COMPETENȚE GENERALE ȘI SPECIFICE CONFORM PROGRAMEI ȘCOLARE

COMPETENȚE GENERALE

1. Identifică principalele caracteristici ale modelelor conceptuale și operaționale ale dezvoltării produselor software, pentru înțelegerea fundamentelor programării
2. Explică principii care stau la baza modelelor conceptuale și operaționale ale dezvoltării produselor software, pentru a fundamenta în mod logic proiectarea și implementarea soluțiilor informatice
3. Utilizează modele conceptuale și operaționale ale dezvoltării produselor software, în scopul obținerii de soluții informatice funcționale și eficiente
4. Analizează caracteristicile și aplicabilitatea modelelor conceptuale și operaționale ale dezvoltării produselor software, pentru a selecta soluțiile cele mai potrivite în funcție de contextul informatic dat
5. Evaluează corectitudinea și eficiența soluțiilor informatice, în vederea optimizării și asigurării funcționalității în diverse scenarii de utilizare
6. Elaborează algoritmi și programe personalizate, pentru a crea soluții informatice coerente și adaptate cerințelor

COMPETENȚE SPECIFICE

- 1.1. Identifică principalele caracteristici ale organizării datelor în cadrul unor modele conceptuale fundamentale – date simple sau liste, pentru a structura și accesa date în vederea prelucrării acestora
- 1.2. Identifică specificul, caracteristicile și etapele unor algoritmi specializați pe clase de probleme, pentru a îi putea utiliza în prelucrarea algoritmică a numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 1.3. Identifică specificul, caracteristicile și etapele unor strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 1.4. Identifică principalele elemente ale limbajului de programare utilizate pentru prelucrarea datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 1.5. Identifică elementele de sintaxă și componentele din definiția și apelul subprogramelor și subprogramelor predefinite pentru operații uzuale, pentru a le utiliza în implementarea algoritmilor în limbaj de programare

- 2.1. Explică principiile de organizare a datelor în cadrul unor modele conceptuale fundamentale – date simple sau liste, pentru a le gestiona eficient
- 2.2. Explică etapele algoritmilor specializați pe clase de probleme, pentru a îi putea utiliza în prelucrarea algoritmică a numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 2.3. Explică etapele unor strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 2.4. Explică rolul principalelor elemente ale limbajului de programare utilizate pentru prelucrarea datelor organizate în modele fundamentale – date simple sau liste, respectiv pentru transmiterea datelor de la și către un program în vederea rezolvării eficiente a problemelor informatice
- 2.5. Explică mecanismul de executare a subprogrameelor și subprogrameelor predefinite pentru operații uzuale, pentru a le utiliza în implementarea algoritmilor în limbaj de programare
- 3.1. Utilizează modul de organizare și prelucrare a datelor în cadrul unor modele conceptuale fundamentale - date simple sau liste, pentru a rezolva probleme de gestionare a datelor
- 3.2. Aplică algoritmi specializați pe clase de probleme, pentru a îi putea utiliza în prelucrarea algoritmică a numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 3.3. Aplică strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 3.4. Utilizează principalele elemente ale limbajului de programare utilizate pentru prelucrarea datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 3.5. Utilizează elementele de sintaxă și componentele din definiția și apelul subprogrameelor și subprogrameelor predefinite pentru operații uzuale, în implementarea algoritmilor în limbaj de programare
- 4.1. Analizează caracteristicile, modul de prelucrare, avantajele și dezavantajele utilizării unor modele conceptuale fundamentale – date simple sau liste, pentru a alege structura adecvată în rezolvarea unor probleme concrete
- 4.2. Analizează comportamentul, aplicabilitatea, avantajele și dezavantajele utilizării unor algoritmi specializați pe clase de probleme pentru prelucrarea numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 4.3. Analizează aplicabilitatea, avantajele și dezavantajele utilizării unor strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 4.4. Analizează aplicabilitatea, avantajele și dezavantajele utilizării unor elemente ale limbajului de programare, pentru a identifica soluția adecvată prelucrării datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 4.5. Analizează aplicabilitatea, avantajele și dezavantajele utilizării subprogrameelor, în implementarea algoritmilor în limbaj de programare
- 5.1. Argumentează alegerea unor modele conceptuale fundamentale – date simple sau liste – și a modurilor de prelucrare a acestora pentru rezolvarea unor probleme informatice concrete
- 5.2. Evaluează corectitudinea și eficiența soluțiilor cu algoritmi specializați pe clase de probleme pentru prelucrarea numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 5.3. Evaluează corectitudinea și eficiența algoritmilor care utilizează strategii de rezolvare a problemelor prin proiectarea modulară a algoritmilor
- 5.4. Evaluează corectitudinea și eficiența aplicațiilor care utilizează elemente specifice de limbaj de programare pentru prelucrarea datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea rezolvării eficiente a problemelor informatice
- 5.5. Evaluează corectitudinea și eficiența programelor care utilizează subprograme, pentru a rezolva probleme informatice
- 6.1. Proiectează algoritmi personalizați care utilizează modele conceptuale fundamentale - date simple sau liste pentru organizarea și prelucrarea eficientă a datelor, utilizând algoritmi de bază, în vederea rezolvării unor probleme
- 6.2. Proiectează soluții cu aplicarea personalizată a algoritmilor specializați pe clase de probleme pentru prelucrarea numerelor, sortarea sau generarea sistematică a unor secvențe de valori
- 6.3. Proiectează modular algoritmi personalizați pentru rezolvarea problemelor
- 6.4. Implementează aplicații care integrează în mod personalizat elemente specifice de limbaj de programare pentru prelucrarea datelor organizate în modele fundamentale - date simple sau liste, respectiv pentru transmiterea datelor de la și către un program, în vederea realizării unor soluții funcționale și performante
- 6.5. Implementează aplicații în limbaj de programare care integrează subprograme personalizate, pentru a modulariza și organiza eficient codul în cadrul soluțiilor informatice

Evaluare inițială

Timp de lucru: 50 minute

Toate subiectele sunt obligatorii. Se acordă 1 punct din oficiu.

Notății utilizate: $a \bmod b$ – restul împărțirii, $a \div b$ – câtul împărțirii întregi, unde a și b sunt numere întregi și $b \neq 0$.

Subiectul I – Întrebări de teorie

2 x 0,5 p. = 1 p.

1. Definește noțiunea de algoritm.
2. Numește două structuri repetitive studiate în gimnaziu.

Subiectul II – Citirea unor algoritmi în pseudocod

3 x 2 p. = 6 p.

1. Se consideră algoritmul:

```
citește a,b      // a, b numere reale
dacă a > b atunci
    scrie a
altfel
    scrie b
sfârșit_dacă
```

Ce se afișează dacă se citesc $a = 5$, $b = 8$? Ce rol are acest algoritm?

2. Se consideră algoritmul:

```
S ← 0
pentru i ← 1,5 execută
    S ← S+i
sfârșit_pentru
scrie S
```

Ce valoare are S la finalul algoritmului?

3. Se consideră algoritmul:

```
citește n      // n – număr natural nenul
cât_timp n ≠ 0 execută
    c ← n mod 10
    dacă c mod 2 = 1 atunci
        scrie c
    sfârșit_dacă
    n ← n div 10
sfârșit_cât_timp
```

Ce se afișează dacă se citește $n=27961$? Ce rol are acest algoritm?

Subiectul III – Scrierea unui algoritm

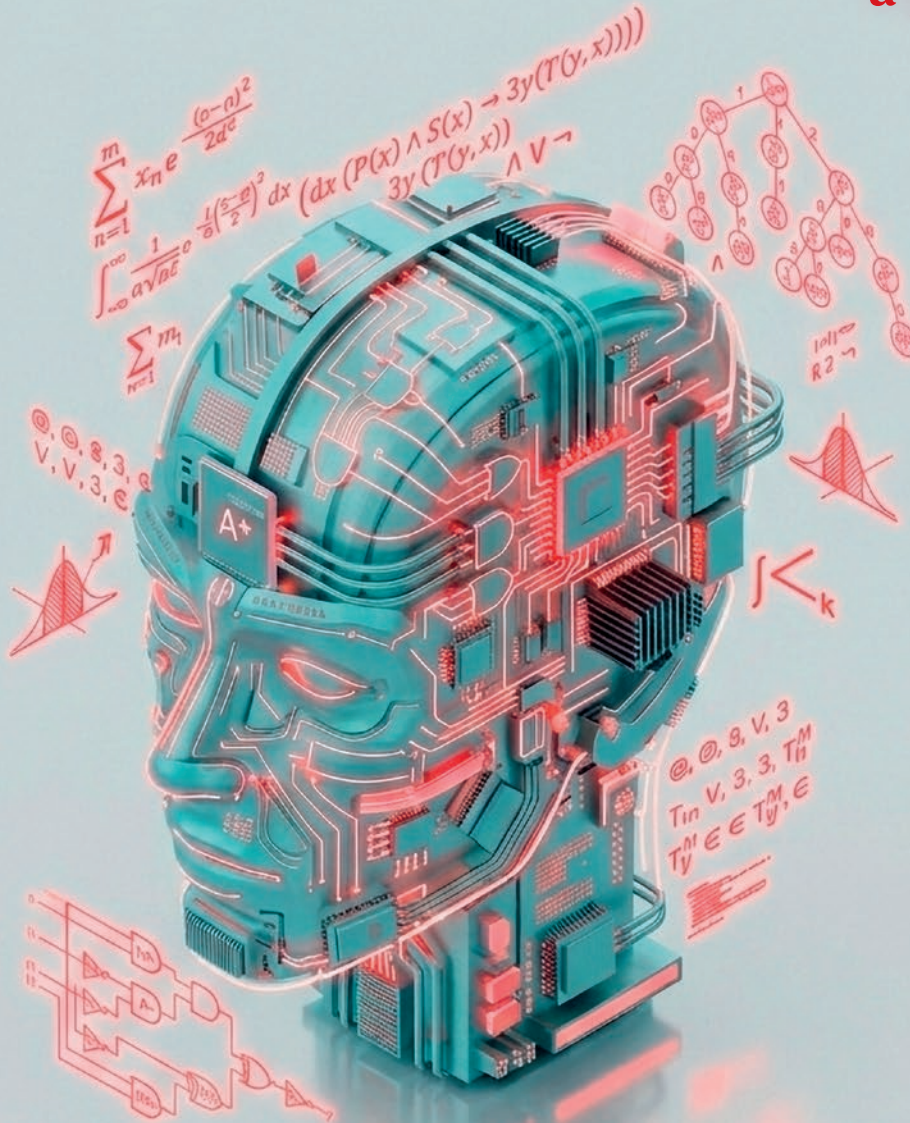
2 p.

Alege UNA dintre cerințele de mai jos și scrie algoritmul corespunzător în limbaj pseudocod:

1. Citește un număr natural nenul n și afișează suma numerelor naturale pare de la 1 la n .
2. Citește trei numere reale a , b , c și afișează cel mai mare dintre ele.

CAPITOLUL 1

Strategii de rezolvare a problemelor



COMPETENȚE SPECIFICE:

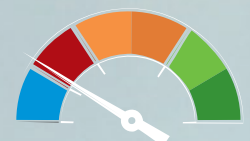
1.2, 1.3,
2.2, 2.3,
3.2, 3.3,
4.2, 4.3,
5.2, 5.3,
6.2, 6.3

1.1. Principii de elaborare a unui program

1.2. Prelucrări ale datelor numerice

1.3. RECAPITULARE

1.4. EVALUARE SUMATIVĂ



1.1. Principii de elaborare a unui program

1.1.1. Noțiuni de bază

Calculatorul poate fi utilizat pentru rezolvarea diferitelor probleme, cum ar fi efectuarea unor calcule, prelucrarea informațiilor sau afișarea unor rezultate. Pentru a realiza aceste activități, calculatorul are nevoie de **instrucțiuni clare și bine ordonate**.

Un **program** este o succesiune de instrucțiuni care descriu pașii necesari pentru rezolvarea unei probleme. Programul este construit pe baza unui **algoritm**, adică o descriere pas cu pas a modului în care poate fi obținut rezultatul dorit.

Proprietățile unui algoritm sunt: **fini**t (procesul se încheie după un număr limitat de pași); **corect** (conduce la obținerea rezultatului așteptat); **general** (se aplică unei clase de probleme, nu doar unui singur caz particular) și **complet** (tratează toate cazurile necesare pentru rezolvarea problemei).

Algoritmul este transpus într-un **limbaj de programare**, astfel încât calculatorul să poată executa instrucțiunile. În acest manual este utilizat limbajul Python, ales pentru claritatea și simplitatea sa.



IMPORTANT

Calculatorul execută instrucțiunile exact în ordinea în care sunt scrise. El nu poate deduce informații lipsă, nu corectează greșelile și produce același rezultat atunci când primește aceleași date de intrare.

Un **limbaj de programare** este un sistem de reguli și simboluri care permite scrierea programelor. Prin intermediul unui limbaj de programare, algoritmii sunt transformați în instrucțiuni precise, exprimate într-un limbaj înțeles de calculator. Există două moduri principale prin care un program este transpus într-un limbaj înțeles de calculator:

- 1) **limbaje compilate** (de exemplu, C++ sau Pascal), în care programul este tradus integral înainte de execuție;
- 2) **limbaje interpretate** (de exemplu, Python sau JavaScript), în care instrucțiunile sunt analizate și executate pe rând.

Pentru a scrie programe corecte, trebuie respectate: **sintaxa** (ansamblul regulilor de scriere a instrucțiunilor), **semantica** (semnificația instrucțiunilor și modul în care sunt executate) și **vocabularul limbajului** (totalitatea cuvintelor-cheie, simbolurilor și elementelor utilizate în programare).



1.1.2. Modelul Intrare – Prelucrare – Ieșire

Rezolvarea unei probleme cu ajutorul calculatorului poate fi analizată folosind modelul **Intrare – Prelucrare – Ieșire (IPI)**:

Intrare	Prelucrare	Ieșire
datele pe care programul le primește	operațiile aplicate asupra datelor	rezultatul obținut în urma prelucrării

Acest model ajută la organizarea logică a soluției înainte de scrierea programului.

1.1.3. Gândirea computațională și proiectarea soluției

Gândirea computațională presupune analizarea unei probleme și formularea unei soluții clare, alcătuită din pași logici ce pot fi executați de calculator. Aceasta implică: înțelegerea exactă a problemei, identificarea datelor necesare, descompunerea soluției în pași simpli și logici și formularea instrucțiunilor într-un mod clar.



ÎNVĂȚARE PRIN DESCOPERIRE

1. Analizează problema calculării perimetrului unui dreptunghi și indică: datele de intrare, pașii de calcul și rezultatul final.

2. Se dorește organizarea unei excursii. Enumeră trei acțiuni care trebuie efectuate pentru realizarea acesteia și scrie-le în ordinea în care ar trebui să fie realizate.

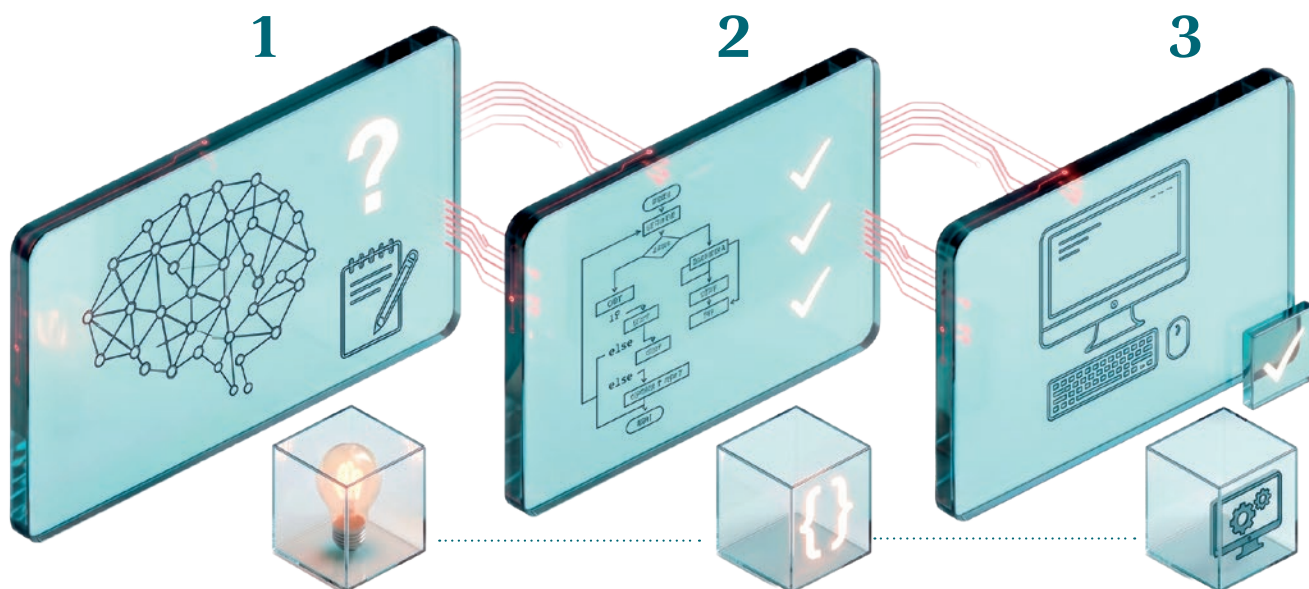
> > >

EXEMPLU

Un magazin vinde 3 caiete la prețul de 15 lei. Se cere să se determine cât vor costa 5 caiete, presupunând că prețul este direct proporțional cu numărul de caiete.

Rezolvare (modelul IPI):

Intrare – numărul cunoscut de caiete (3), prețul total (15 lei) și numărul de caiete dorit (5);
Prelucrare – se determină prețul unui caiet prin împărțirea lui 15 la 3, apoi se înmulțește rezultatul cu 5;
Ieșire – prețul total pentru 5 caiete.





1.1.4. Etapele elaborării unui program și rolul fiecăreia

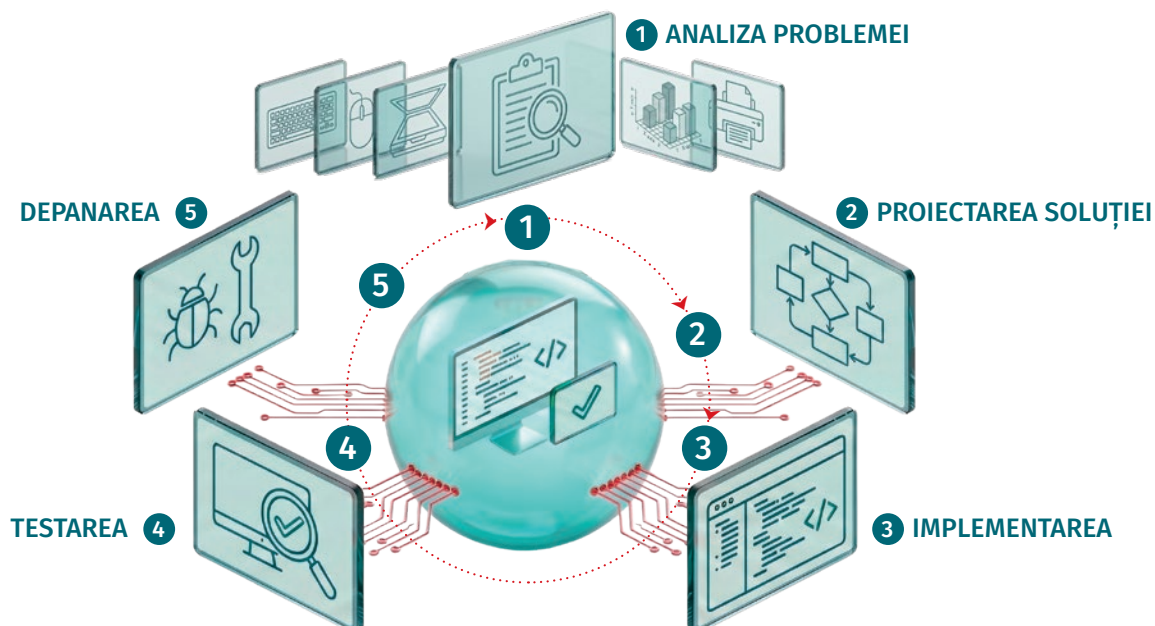
Rezolvarea unei probleme cu ajutorul calculatorului se realizează într-un mod organizat, urmând un set de etape bine definite. Acestea ajută la clarificarea problemei, la organizarea pas cu pas a soluției și la verificarea rezultatelor.



IMPORTANT

Etapele elaborării unui program sunt strâns legate între ele și pot fi parcurse de mai multe ori, până când soluția finală este corectă și completă. După testare și depanare, soluția poate fi îmbunătățită sau adaptată, reluând unele etape.

Etapa	Rol
1. Analiza problemei	Se clarifică cerința și se identifică datele de intrare (informațiile cunoscute; acestea sunt introduse în calculator cu ajutorul unor dispozitive de intrare, cum ar fi tastatura, mouse-ul sau scannerul), datele de ieșire (rezultatele dorite; acestea sunt prezentate utilizatorului cu ajutorul unor dispozitive de ieșire, precum monitorul sau imprimanta) și datele de manevră (valori intermediare folosite în calcule).
2. Proiectarea soluției	Se stabilesc pașii logici necesari, conducând la construirea algoritmului: se descrie algoritmul în cuvinte sau sub formă de pași, se stabilește ordinea operațiilor și se verifică dacă soluția este logică și completă.
3. Implementarea	Transpunerea algoritmului într-un limbaj de programare, respectând regulile de scriere ale acestuia.
4. Testarea	Se rulează programul și se compară rezultatele obținute cu cele așteptate, folosind exemple concrete.
5. Depanarea	Identificarea și corectarea erorilor. Poate necesita revenirea la implementare sau la proiectarea soluției, până când programul funcționează corect.





1.1.5. Reprezentarea algoritmilor

Un algoritm poate fi reprezentat în mai multe moduri, fiecare având rolul de a exprima pașii de rezolvare cu un grad diferit de precizie.

Mod de reprezentare	Caracteristici	Avantaje	Limite
Limbaj natural	Pași descriși în propoziții obișnuite.	Ușor de citit și de înțeles.	Ambiguu; nu poate fi executat direct de calculator.
Pseudocod	Formă intermediară: folosește cuvinte-cheie (citește, scrie, dacă, cât timp) fără reguli stricte de sintaxă.	Evidențiază clar datele și ordinea operațiilor; ușor de transpus în orice limbaj.	Nu se execută pe calculator; depinde de convenția autorului.
Diagramă de flux (schemă logică)	Reprezentare grafică prin simboluri și săgeți.	Vizualizează clar fluxul și deciziile.	Devine dificil de urmărit pentru algoritmi lungi.
Limbaj de programare	Transpunere într-un limbaj (Python, C++ etc.) cu sintaxă strictă. Programul este executat de calculator printr-un interpretor (Python) sau printr-un compilator (C++).	Poate fi executat direct; sintaxa garantează caracterul neambiguu.	Necesită cunoașterea regulilor limbajului.

Simbolurile uzuale în diagramele de flux

Simbol	Denumire	Descriere
	Bloc terminal	Marchează începutul (START) și sfârșitul (STOP) procesului.
	Linie de flux	Indică direcția execuției și leagă etapele algoritmului.
	Bloc de intrare/ieșire	Reprezintă citirea datelor sau afișarea rezultatelor.
	Bloc de proces	Reprezintă o operație sau un calcul efectuat.
	Bloc de decizie	Indică verificarea unei condiții și ramificarea (Da/Nu).
	Bloc conector	Conectează mai multe puncte ale schemei logice.
	Bloc de subprogram	Reprezintă un apel al unui subprogram.



STUDIUL DE CAZ

Se citesc trei note (n_1 , n_2 , n_3) și se cere afișarea mediei aritmetice a acestora.

Analiza problemei:

► Date de intrare: n_1 , n_2 , n_3 . ► Date de ieșire: *media*. ► Date de manevră: *suma*.

Limbaj natural	Proiectarea soluției (pseudocod)	Implementarea (Python)
Se cunosc trei note. Se calculează suma acestora și se împarte la 3. Rezultatul este media aritmetică.	<pre> citește n_1, n_2, n_3 suma ← $n_1 + n_2 + n_3$ media ← suma / 3 scrie media </pre>	<pre> n1 = int(input("n1 = ")) n2 = int(input("n2 = ")) n3 = int(input("n3 = ")) suma = n1 + n2 + n3 media = suma / 3 print(media) </pre>

Testarea (exemplu de rulare în consolă):

$n_1 = 9$; $n_2 = 10$; $n_3 = 8$; *media* = 9.0

Depanarea: Dacă media afișată nu corespunde celei așteptate, se verifică dacă împărțirea se face la 3 și dacă s-au folosit corect parantezele. O greșeală frecventă este scrierea $n_1 + n_2 + n_3 / 3$, care împarte doar pe n_3 la 3.



APLICAȚIE ÎN ECHIPĂ

Elevii lucrează în grupuri de 2-3. Se dorește organizarea unei excursii pentru elevii unei clase. Pentru participare, fiecare elev contribuie cu o sumă de bani.

Se cunosc: numărul de elevi participanți, suma plătită de fiecare elev și costul total al excursiei.

Stabiliți pașii necesari pentru a determina dacă suma strânsă este suficientă. Reprezentați algoritmul în pseudocod.

1.1.6. Introducere în limbajul Python

Python este un limbaj de programare modern, ușor de învățat datorită sintaxei sale simple și clare. Din acest motiv, este o alegere excelentă pentru începători. Cu ajutorul limbajului Python, se pot crea jocuri, aplicații web sau chiar sisteme de inteligență artificială.

Vocabularul limbajului Python este alcătuit din: setul de caractere (litere, cifre, simboluri), identificatori (nume de variabile, funcții), separatori (spații, paranteze, virgule), constante (valori fixe), cuvinte-cheie – cuvinte rezervate ale limbajului, care au o semnificație predefinită și nu pot fi folosite ca nume de variabile sau funcții (de exemplu: **if**, **while**, **for**, **def**, **return**, **import**) și operatori (de exemplu: **+**, **-**, *****, **/**). Comentariile în Python sunt texte care încep cu simbolul **#** și sunt ignorate de program la executare. Ele sunt utilizate pentru descrierea codului, astfel încât acesta să poată fi înțeles mai ușor.

În informatică, informațiile cu care lucrează un program poartă numele de **date**. Ele pot reprezenta valori numerice (vârsta, nota, temperatura), texte (nume, adresa) sau valori logice (adevărat/fals).

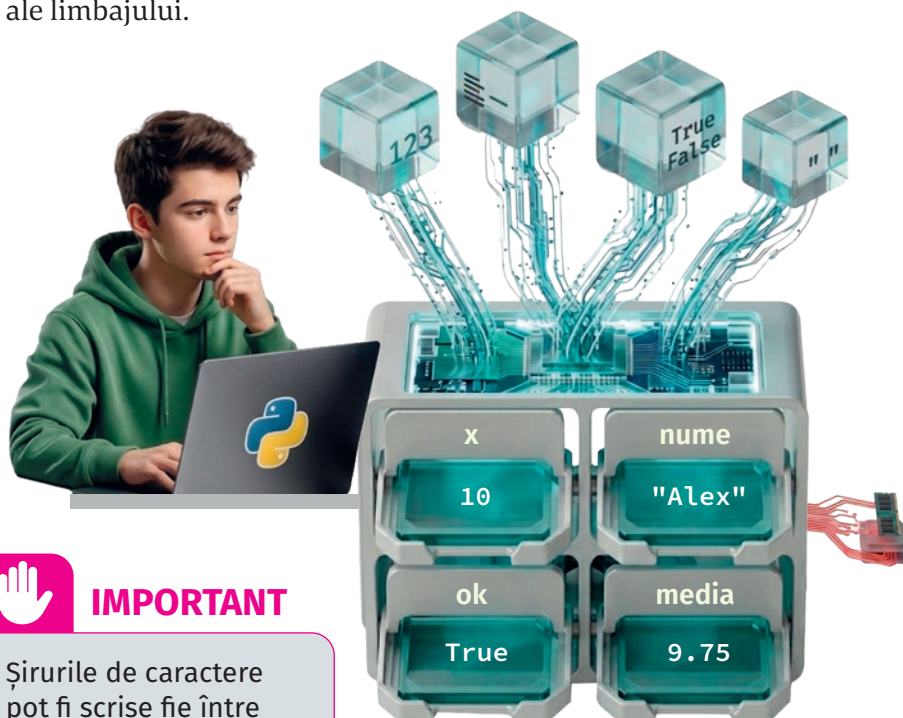
Pentru a fi utilizate de program, datele trebuie memorate. Memorarea se realizează cu ajutorul variabilelor. O **variabilă** este o zonă de memorie identificată printr-un nume, în care este stocată o valoare ce poate fi utilizată sau modificată pe parcursul execuției.

Memoria calculatorului este alcătuită dintr-un număr foarte mare de locații, fiecare având o adresă. Aceste locații pot fi comparate cu niște „căsuțe numerotate”. Atunci când unei variabile *i* se atribuie o valoare, calculatorul rezervă automat o astfel de căsuță și reține informația respectivă.

Identificatorii sunt numele atribuite variabilelor (și altor elemente: funcții, constante, clase).

Reguli de denumire a identificatorilor:

- ▶ numele trebuie să înceapă cu o literă sau cu simbolul underscore (`_`);
- ▶ nu sunt permise spațiile sau caracterele speciale;
- ▶ se pot utiliza litere, cifre și caracterul underscore (`_`);
- ▶ se recomandă folosirea unor denumiri sugestive;
- ▶ nu se pot folosi cuvintele-cheie (rezervate) ale limbajului.



IMPORTANT

Șirurile de caractere pot fi scrise fie între ghilimele (`" "`), fie între apostrofuri (`' '`). Cele două forme sunt echivalente. Alegerea uneia devine importantă atunci când textul conține deja unul dintre aceste simboluri, de exemplu: `mesaj = 'El a spus "Bună ziua"'`.



IMPORTANT

Operatorul `=` nu exprimă egalitatea matematică, ci operația de atribuire. O instrucțiune de forma `variabilă = expresie` se execută în doi pași: (1) se evaluează expresia din dreapta; (2) rezultatul este atribuit variabilei din stânga.



EXEMPLU

```
x = 1
x = x + 1
```

După executare, variabila `x` are valoarea **2**. Valoarea anterioară a variabilei este utilizată în calcul.

Constantele sunt variabile ale căror valori, prin convenție, nu trebuie să se modifice pe durata execuției și se scriu, de regulă, cu litere mari. De exemplu, `NUMAR_LUNI_DIN_AN = 12`.

Fiecare variabilă are asociat un tip de date, care indică natura informației stocate. Principalele tipuri din Python sunt prezentate în tabelul următor.

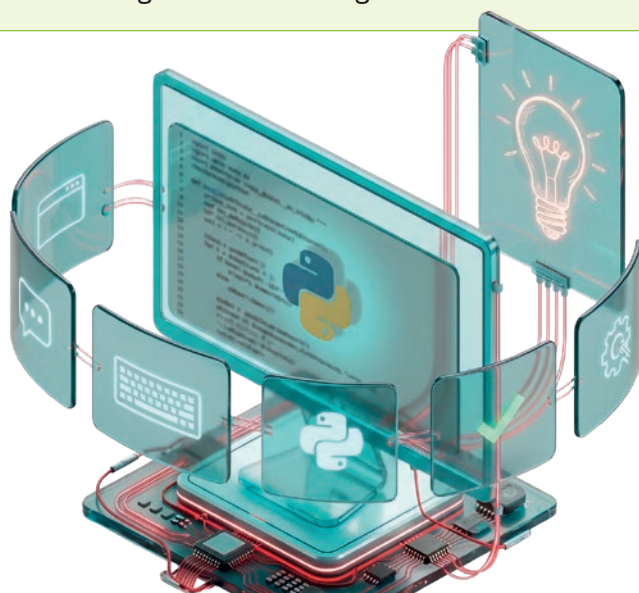
Tip	Semnificație	Exemplu
<code>int</code>	numere întregi (fără zecimale)	<code>nota = 10</code> <code>nivel = -1</code>
<code>float</code>	numere reale (cu zecimale)	<code>media = 9.75</code> <code>temperatura = -3.5</code>
<code>str</code>	șiruri de caractere (text), între <code>" "</code> sau <code>' '</code>	<code>oras = "București"</code> <code>mesaj = 'O zi bună'</code>
<code>bool</code>	valori logice: <code>True</code> / <code>False</code>	<code>este_prezent = True</code> <code>are_tema = False</code>



APLICAȚIE

Se consideră situația organizării unei excursii școlare. Pentru planificare sunt cunoscute următoarele informații: numărul de elevi: 28; costul unui bilet: 45.5 lei; numele orașului vizitat: Brașov; existența transportului asigurat: adevărat.

- Identifică datele utilizate în această situație.
- Propune pentru fiecare valoare un nume sugestiv de variabilă.
- Precizează tipul de date corespunzător fiecărei variabile.
- Explică de ce este utilă folosirea unor variabile cu denumiri sugestive într-un algoritm.



1.1.7. Citirea și afișarea datelor

Pentru scrierea și rularea programelor se utilizează un mediu de dezvoltare integrat (IDE – Integrated Development Environment), precum **PyCharm**, care permite scrierea, verificarea și rularea programelor. Rezultatele se afișează în **consolă** – o interfață de tip text integrată în IDE, prin care programul comunică cu utilizatorul: afișează mesaje și preia date introduse de la tastatură.

Citirea datelor se realizează cu ajutorul funcției `input()`. Aceasta afișează un mesaj și așteaptă ca utilizatorul să introducă o valoare, care este preluată și memorată într-o variabilă. Valoarea citită este inițial de tip `str` (șir de caractere). Pentru a o utiliza în calcule, se realizează **conversia de tip**: `int()` pentru numere întregi și `float()` pentru numere reale.

Afișarea datelor se realizează cu funcția `print()`. Aceasta primește una sau mai multe valori (texte, numere, expresii) separate prin virgulă și le afișează pe ecran, urmate de trecerea la linia următoare.

EXEMPLE



Cod Python	Explicații
<code>print("O zi bună!")</code>	Afișează un text fix.
<code>nume = input("Nume: ")</code> <code>print("Salut,", nume)</code>	Citește un text și afișează un mesaj personalizat.
<code>a = int(input("a= "))</code> <code>b = int(input("b= "))</code> <code>print("Suma:", a + b)</code>	Citește două numere întregi și afișează suma.
<code>print("Raport:")</code> <code>print("Expresie: (12-4)*3")</code> <code>print("Rezultat:", (12-4)*3)</code>	Afișează un mic raport de calcul pe trei linii.
<code>varsta = int(input("Introduceți vârsta: "))</code> <code>print(f"Vârsta este: {varsta}")</code>	Citește un număr întreg și îl afișează folosind un f-string – șir prefixat cu <code>f</code> , în care expresia dintre <code>{}</code> este înlocuită automat cu valoarea variabilei <code>varsta</code> .

1.1.8. Expresii și operatori

O **expresie** este o **combinație de operatori și operanzi** legați între ei după reguli specifice limbajului de programare. Operanzii pot fi: variabile, constante, funcții. Rezultatul evaluării unei expresii este o valoare numerică (expresii aritmetice) sau logică, de tip **True/False** (expresii logice).

Operatorii aritmetici permit efectuarea calculului matematice.

Operator	+	-	*	/	//	%	**
Semnificație	adunare	scădere	înmulțire	împărțire	împărțire întreagă	rest	putere

Operatorii relaționali (de comparație) – rezultatul este **True** sau **False**:

Operator	<	<=	>	>=	==	!=
Semnificație	mai mic	mai mic sau egal	mai mare	mai mare sau egal	egal	diferit

Operatorii logici – combină valori adevărate/false:

Operator	Semnificație	Exemplu
and	și (ambele condiții adevărate)	<code>(24 > 0) and (24 < 100)</code>
or	sau (cel puțin o condiție adevărată)	<code>(1 == 1) or (1 == 5)</code>
not	negație	<code>not (1 == 0)</code>

Tabelul de adevăr – AND / OR:

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Prioritatea operatorilor:

Prioritate	1	2	3	4	5	6	7	8
Operator	()	**	*, /, //, %	+, -	relaționali (<, <=, >, >=, ==, !=)	not	and	or

Pentru claritate, se recomandă folosirea parantezelor.



IMPORTANT

În Python, operația de împărțire (/) produce un rezultat de tip **float**, chiar dacă împărțirea este exactă (de exemplu, `10 / 2` este `5.0`). Pentru împărțirea cu rezultat întreg se folosește `//`.





PROBLEMĂ REZOLVATĂ

În programul de mai jos sunt utilizate mai multe tipuri de date, expresii aritmetice și logice.

Cod Python

```
nume = input("Introduceți numele elevului: ")
nota1 = float(input("Introduceți prima notă: "))
nota2 = float(input("Introduceți a doua notă: "))
prezent = True

media = (nota1 + nota2) / 2
promovat = media >= 5 and prezent == True

print("Elev:", nume)
print("Media:", media)
print("Este promovat:", promovat)
```

Explicații

- variabila `nume` citește și memorează un text introdus de la tastatură;
- variabilele `nota1` și `nota2` citesc de la tastatură valori numerice, convertite cu `float()` din șirul returnat de `input()`;
- variabila `prezent` memorează o valoare logică (`True`);
- expresia aritmetică calculează media notelor;
- expresia logică verifică dacă elevul este promovat;
- rezultatele sunt afișate în consolă cu `print()`.



APLICAȚIE

Se consideră un dreptunghi cu lungimea de 12 cm și lățimea de 5 cm. Să se declare variabile pentru memorarea acestor valori, apoi să se calculeze perimetrul și aria dreptunghiului. Folosind o expresie logică, să se verifice dacă aria este mai mare decât 50 și să se memoreze rezultatul într-o variabilă de tip logic. La final, să se afișeze perimetrul, aria și rezultatul verificării.

1.1.9. Structuri fundamentale de control

În practică, execuția unui algoritm poate depinde de anumite condiții sau poate presupune repetarea unor pași. Pentru astfel de situații, se folosesc **structurile fundamentale de control**, care stabilesc modul de parcurgere a pașilor de rezolvare: decizia, alternativa, alegerea multiplă și repetarea.



IMPORTANT

În Python, blocurile de cod sunt delimitate prin indentare (un TAB sau 4 spații spre dreapta), nu prin acolade. Indentarea corectă este esențială: toate instrucțiunile scrise la același nivel fac parte din același bloc. Amestecarea spațiilor și a tab-urilor în același bloc de cod poate produce eroarea *TabError*.

1.1.9.1. Decizia, alternativa și alegerea multiplă: **if**, **if-else**, **elif**

Instrucțiunea **if** – decizia.

Permite executarea unui bloc de instrucțiuni doar dacă o condiție este adevărată.

```
if condiție:  
    instrucțiuni
```

Instrucțiunea **if-else** – alternativa.

Algoritmul alege între două acțiuni diferite: dacă condiția este **True** se execută ramura **if**, altfel se execută ramura **else**.

```
if condiție:  
    instrucțiuni_1  
else:  
    instrucțiuni_2
```

Instrucțiunea **elif** – alegerea multiplă.

Verifică succesiv mai multe condiții și execută blocul corespunzător primei condiții adevărate. Dacă niciuna nu este adevărată, se execută ramura **else** (opțională).

```
if condiție_1:  
    instrucțiuni_1  
elif condiție_2:  
    instrucțiuni_2  
else:  
    instrucțiuni_finale
```



PROBLEMĂ REZOLVATĂ

Exemplu – în limbaj natural

Se citește nota unui elev. Se cere afișarea calificativului:

- “Insuficient” dacă nota este sub 5;
- “Suficient” dacă nota este 5 sau 6;
- “Bine” dacă nota este 7 sau 8;
- “Foarte bine” dacă nota este 9 sau 10.

Pseudocod

```
citește nota  
dacă nota < 5 atunci  
    afișează "Insuficient"  
altfel  
    dacă nota <= 6 atunci  
        afișează "Suficient"  
    altfel  
        dacă nota <= 8 atunci  
            afișează "Bine"  
        altfel  
            afișează "Foarte bine"  
sfârșit_dacă  
sfârșit_dacă  
sfârșit_dacă
```

Cod Python

```
nota = int(input("Nota: "))  
if nota < 5:  
    print("Insuficient")  
elif nota <= 6:  
    print("Suficient")  
elif nota <= 8:  
    print("Bine")  
else:  
    print("Foarte bine")
```

Explicații

- se citește variabila **nota** (de exemplu 8);
- prima condiție verificată este **nota < 5** → fals;
- a doua condiție **nota <= 6** → fals;
- a treia condiție **nota <= 8** → adevărat;
- se execută instrucțiunea corespunzătoare; se afișează mesajul “Bine”;
- verificările ulterioare nu mai sunt realizate.

**IMPORTANT**

O buclă **while** trebuie să aibă o **condiție de oprire**, bazată de obicei pe o **variabilă de control** care trebuie modificată în corpul buclei. Dacă variabila nu este actualizată, bucla devine infinită și programul nu se mai termină.

1.1.9.2. Repetarea — bucle: `while` și `for`

O buclă este o structură de control care permite **executarea repetată** a unui bloc de instrucțiuni atât timp cât este îndeplinită o anumită condiție sau pentru un număr cunoscut de pași. Instrucțiunile care se repetă formează **corpul buclei**, iar fiecare parcurgere a corpului se numește **iterație**.

Instrucțiunea `while` – repetare cu test inițial. Se folosește atunci când nu se cunoaște dinainte numărul de repetări. La fiecare iterație condiția este evaluată; dacă este **True**, se execută instrucțiunile; dacă este **False**, bucla se oprește.

`while` condiție:
 instrucțiuni

Instrucțiunea `for` – repetare cu număr cunoscut de pași. Parcurge valorile unei secvențe, de exemplu generată cu **`range()`**.

`for` variabilă în secvență:
 instrucțiuni

> > > EXEMPLU

Se cere afișarea numerelor de la 1 la 5.

Cod Python – folosind <code>while</code>	Cod Python – folosind <code>for</code>	Exemplu de rulare
<pre>i = 1 while i <= 5: print(i) i = i + 1</pre>	<pre>for i in range(1, 6): print(i)</pre>	<pre>1 2 3 4 5</pre>

**PROBLEMĂ REZOLVATĂ – șiruri de valori cu număr necunoscut de pași**

Se citesc note de la tastatură până la introducerea valorii 0. Să se calculeze media acestora.

Cod Python	Explicații
<pre>suma = 0 numar_note = 0 nota = int(input("Introdu o notă: ")) while nota != 0: suma = suma + nota numar_note = numar_note + 1 nota = int(input("Introdu o notă: ")) if numar_note > 0: media = suma / numar_note print("Media notelor este:", media) else: print("Nicio notă introdusă")</pre>	– se inițializează suma cu 0; – se inițializează numărul de note cu 0; – se citește prima notă de la tastatură; – funcția <code>input()</code> este utilizată pentru a prelua valorile introduse de utilizator; – funcția <code>int()</code> transformă aceste valori din text în numere întregi; – se repetă cât timp nota este diferită de 0; – se adaugă nota la sumă; – se crește numărul de note; – se citește următoarea notă; – se verifică dacă există note introduse; – se calculează media; – se afișează media, folosind instrucțiunea <code>print()</code>; – altfel, se tratează cazul fără note; – se afișează mesaj corespunzător.



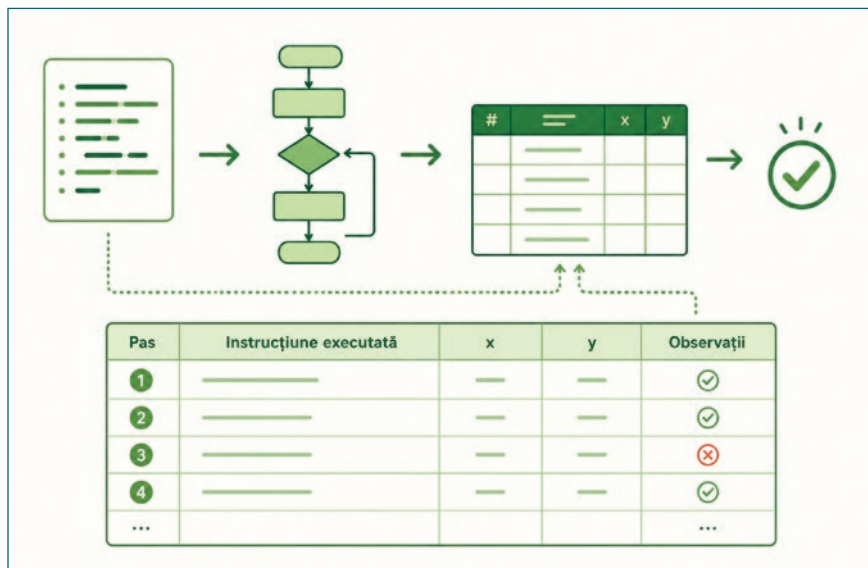
APLICAȚIE

Se citesc numere întregi până la introducerea valorii 0. Să se determine suma numerelor pare și suma numerelor impare, apoi să se afișeze care sumă este mai mare.

1.1.10. Urmărirea execuției algoritmilor și identificarea erorilor

Pentru a înțelege corect un algoritm, se urmărește pas cu pas evoluția valorilor variabilelor, folosind un tabel de urmărire (pas, instrucțiune executată, valorile variabilelor după execuție).

Acest tabel ajută la identificarea erorilor și la verificarea rezultatului.



EXEMPLU

>>>

În exemplul următor, în stânga este codul Python, iar în dreapta tabelul de execuție.

```
s = 0
for i in range(1, 4):
    s = s + i
print(s)
```

Pas	Instrucțiune	i	s
1	s = 0	-	0
2	i = 1	1	0
3	s = s + i	1	1
4	i = 2	2	1
5	s = s + i	2	3
6	i = 3	3	3
7	s = s + i	3	6

Tip eroare	Descriere	Exemplu
Eroare de sintaxă	Instrucțiunile nu respectă regulile limbajului; programul nu poate fi rulat.	<pre>if x > 5 print("x mai mare") # lipsește simbolul :</pre>
Eroare de logică	Programul rulează, dar rezultatul este greșit (algoritm greșit).	<pre>x = 10 y = 5 media = x + y / 2 # corect: (x+y)/2</pre>
Eroare de execuție	Apare în timpul rulării; programul se oprește brusc.	<pre>x = 10 y = 0 print(x / y) # împărțire la 0</pre>



ÎNVĂȚARE PRIN DESCOPERIRE

Analizează programul de mai jos și explică, pe baza codului și a observațiilor făcute la rulare, cum sunt utilizate structurile de control învățate. Rulează programul în PyCharm, observă rezultatul obținut și explică:

- cum se modifică valorile variabilelor pe parcursul execuției;
- în ce situație se afișează mesajul "buget sigur" și în ce situație se afișează "buget critic";
- câte cheltuieli mari există, știind că o cheltuială este considerată mare dacă valoarea ei este mai mare decât 70.

După rulare, programul poate fi salvat. Un **fișier de tip program sursă** conține instrucțiuni scrise într-un limbaj de programare (de exemplu, Python), pe care calculatorul le poate executa. Extensia sa este, de obicei, **.py**.

Cod Python	Explicații
<pre> buget = 400 cheltuiala = 70 pas = 1 cheltuieli_mari = 0 while (buget >= 50) and (pas <= 8): buget = buget - cheltuiala if cheltuiala > 70: cheltuieli_mari = cheltuieli_mari + 1 cheltuiala = cheltuiala + 15 pas = pas + 1 if buget >= 100: mesaj = "buget sigur" else: mesaj = "buget critic" print("Buget rămas:", buget) print("Număr pași:", pas - 1) print("Cheltuieli mari:", cheltuieli_mari) print("Mesaj:", mesaj) </pre>	– buget = 400 – suma inițială – cheltuiala = 70 – prima cheltuială – pas = 1 – numără pași – cheltuieli_mari = 0 – numără cheltuielile > 70 Bucula rulează cât timp: buget >= 50 și pas <= 8. La fiecare pas: – se scade cheltuiala din buget; – dacă cheltuiala > 70, se incrementează cheltuieli_mari – cheltuiala crește cu 15 – pas crește cu 1 (asigură oprirea buclei) După buclă: – dacă buget >= 100 → "buget sigur", altfel → "buget critic" La final se afișează rezultatele.

1.1.11. Eficiența algoritmilor – abordare intuitivă

În rezolvarea unei probleme pot exista mai mulți algoritmi corecți care conduc la același rezultat, dar care diferă prin numărul de pași necesari și prin modul de organizare. Un algoritm este considerat eficient atunci când rezolvă problema folosind cât mai puține operații și cât mai puțină memorie.

Eficiența se analizează din două puncte de vedere:

- 1. eficiența în timp** – numărul de operații efectuate de calculator pentru obținerea rezultatului într-un timp cât mai scurt;
- 2. eficiența în memorie (spațiu)** – cantitatea de memorie utilizată pentru stocarea datelor (variabile, liste) pe durata execuției.



Se cere calcularea sumei primelor 4 numere naturale.

Algoritm A – se adună numerele pe rând, folosind o variabilă **s** și o variabilă contor **i** (ex: pentru $n = 4$: $0+1 \rightarrow 1$; $1+2 \rightarrow 3$; $3+3 \rightarrow 6$; $6+4 \rightarrow 10$).

```
s = 0
for i in range(1, 4 + 1):
    s = s + i
```

Algoritm B – se aplică Formula lui Gauss:

```
s = n * (n + 1) // 2
s = 4 * (4 + 1) // 2 = 10
```

Ambii algoritmi produc același rezultat, însă B efectuează un număr **fix** de operații (indiferent de n), pe când A efectuează n adunări. Prin urmare, B este mai **eficient în timp**.

Eficiența în spațiu – ambii algoritmi utilizează un număr constant de variabile (algoritmul A folosește variabilele **s** și **i**, iar algoritmul B doar variabila **s**), astfel că memoria utilizată nu depinde de valoarea lui n , fiind de tip constant (**$O(1)$**). În schimb, dacă s-ar memora mai întâi toate cele n valori într-o listă și abia apoi s-ar calcula suma, memoria utilizată ar crește proporțional cu n , ceea ce ar face soluția mai puțin eficientă din punct de vedere al spațiului.

Numărul de pași și dimensiunea datelor

În practică, eficiența depinde și de dimensiunea datelor de intrare. Dacă un algoritm parcurge n valori (de exemplu, n note), numărul de pași crește odată cu n . Un astfel de algoritm se notează intuitiv cu **$O(n)$** – numărul de operații este proporțional cu volumul datelor. Un algoritm care execută un număr fix de operații (ca Algoritmul B) se notează cu **$O(1)$** . Există și algoritmi care au complexitate **polinomială**, în care numărul de pași crește ca o putere a lui n , de exemplu **$O(n^2)$** sau **$O(n^3)$** . Două bucle imbricate pot duce, în anumite cazuri, la o complexitate **$O(n^2)$** .



APLICAȚIE

Se citesc de la tastatură trei numere întregi distincte: **a**, **b**, **c**. Se propun două variante de algoritm pentru determinarea valorii maxime.

Algoritm A	Algoritm B
– se presupune că a este maximul; – se compară b cu maximul și, dacă este mai mare, maximul devine b; – se compară c cu maximul și, dacă este mai mare, maximul devine c; – se afișează maximul.	– se verifică dacă a este mai mare decât b și c; dacă da, se afișează a; – altfel, se verifică dacă b este mai mare decât a și c; dacă da, se afișează b; – altfel, se afișează c.

- Implementează în pseudocod ambii algoritmi.
- Implementează în Python ambii algoritmi.
- Precizează care algoritm este mai eficient.
- Compară numărul de comparații efectuate în fiecare caz.
- Dacă problema ar fi pentru n numere, precizează (intuitiv) care algoritm ar fi de tip $O(n)$.

1.1.12. Proiectarea modulară a algoritmilor

Un **modul** reprezintă o parte a algoritmului care realizează o sarcină bine definită. În **proiectarea modulară**, problema este împărțită în mai multe subprobleme (module), fiecare rezolvând o etapă distinctă. Această abordare face algoritmul mai ușor de înțeles, urmărit, verificat și modificat.

Analiză comparativă: soluție compactă vs. modulară

Soluție compactă	Soluție modulară
Toți pașii sunt descriși într-o singură secvență: citire, calcul, afișare.	Problema este separată în etape distincte: citire, calcul parțial, calcul final, afișare. Fiecare etapă are un rol clar.
Mai scurtă; devine dificil de urmărit când problema este complexă.	Mai bine organizată; erorile se identifică mai rapid, fiecare parte este analizată separat.

>>> **EXEMPLU**

Se citesc prețul unui produs și procentul de reducere. Să se calculeze prețul final.

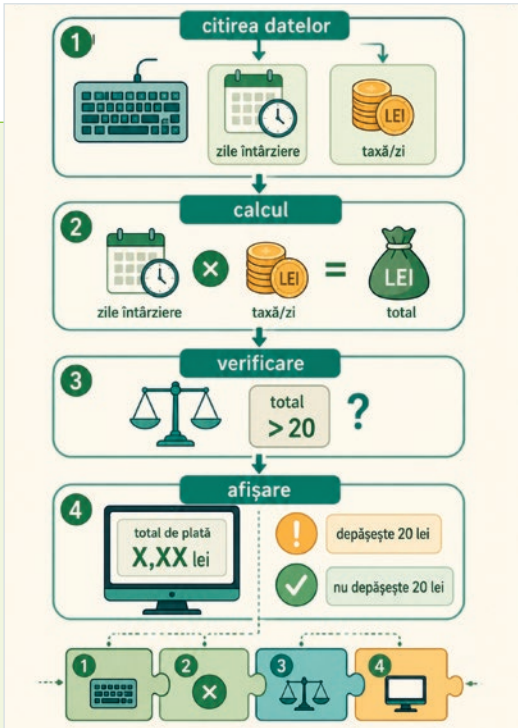
Cod Python	Explicații
<pre>pret = float(input("Preț inițial: ")) reducere = float(input("Reducere (%): ")) valoare_reducere = pret * reducere / 100 pret_final = pret - valoare_reducere print("Preț final:", pret_final)</pre>	În acest program se pot identifica modulele: - citirea datelor (preț și reducere); - calculul valorii reducerii; - determinarea prețului final; - afișarea rezultatului.

 APLICAȚIE

Se consideră o bibliotecă școlară. Un elev împrumută mai multe cărți, iar pentru fiecare zi de întârziere trebuie să plătească o taxă fixă (în lei).

Se citesc de la tastatură numărul de zile de întârziere și taxa pentru o zi de întârziere.

- (a) descrie algoritmul în variantă modulară, identificând etapele principale;
- (b) precizează ce calcule se realizează în fiecare etapă;
- (c) scrie programul în Python;
- (d) testează programul pentru o taxă de 2 lei pe zi și verifică dacă suma totală de plată depășește 20 lei;
- (e) explică de ce este utilă împărțirea problemei în module.





FIȘĂ DE LUCRU 1

1. Se consideră următorul algoritmul scris în Python:

```
x = 1
suma = 0

while x <= 5:
    suma = suma + x
    if x % 2 == 0:
        suma = suma + 2
    x = x + 1
```

- a) Să se identifice variabila de control și condiția de oprire a buclei.
- b) Să se completeze un tabel de urmărire a execuției pentru primele trei iterații, indicând valorile variabilelor **x** și **suma**.
- c) Să se precizeze valoarea finală a variabilei **suma**.
- d) Să se explice rolul instrucțiunii **if** din interiorul buclei.

2. Se consideră următorul program:

```
numar = 10
contor = 1
rezultat = 0

while contor <= numar:
    if contor % 3 == 0:
        rezultat = rezultat + contor
    contor = contor + 1
```

- a) Să se precizeze ce valori sunt adunate în variabila **rezultat**, în funcție de condiția din instrucțiunea **if**.
- b) Să se precizeze numărul de iterații ale buclei **while**.
- c) Să se determine valoarea finală a variabilei **rezultat**.

3. Se citește de la tastatură un număr natural **n**, cu **n > 0**. Apoi se citesc **n** numere întregi.

Să se realizeze un program care determină câte dintre numere sunt pare, câte sunt impare și câte sunt negative, calculează suma numerelor pare și media numerelor negative, apoi afișează rezultatele obținute și un mesaj care indică dacă există mai multe numere pare decât impare. Dacă nu există numere negative, se va afișa un mesaj corespunzător.

- a) Identifică datele de intrare, datele de ieșire și descrie rezolvarea folosind modelul Intrare–Prelucrare–Ieșire;
- b) Precizează cele 5 etape ale elaborării unui program;
- c) Scrie algoritmul în pseudocod și în limbajul Python;
- d) Pentru valorile **n** = 4 și numerele 2, -5, 4, -1 urmărește execuția algoritmului și precizează valorile variabilelor: variabila de control, valoarea curentă citită, numărul de valori pare, numărul de valori negative și sumele calculate.



1.2. Prelucrări ale datelor numerice

1.2.1. Operații cu cifrele unui număr

Numerele naturale sunt formate din cifre, iar în programare apare frecvent necesitatea de a lucra cu cifrele individuale ale unui număr: aflarea ultimei cifre, numărarea cifrelor sau construirea unui număr nou. Prin operații simple de împărțire, putem accesa fiecare cifră a unui număr și putem construi numere noi, cifră cu cifră.

Cele două operații esențiale pentru lucrul cu cifrele unui număr sunt **împărțirea întreagă** și **restul împărțirii**:

- `n % 10` – returnează ultima cifră a numărului `n`;
- `n // 10` – returnează numărul obținut prin eliminarea ultimei cifre.

De exemplu, pentru `n = 4527`: operația `4527 % 10` produce `7` (ultima cifră), iar `4527 // 10` produce `452` (numărul fără ultima cifră). Aceste două operații, folosite într-o structură repetitivă, permit parcurgerea tuturor cifrelor unui număr, de la dreapta la stânga.

Pentru a construi un număr nou prin **adăugarea de cifre**, se folosesc operații inverse extragerii cifrelor:

- **adăugarea unei cifre la dreapta** se realizează prin formula: `rezultat = nr * 10 + cifra`. Înmulțirea cu 10 deplasează cifrele numărului la stânga, iar cifra se adaugă pe ultima poziție.
- **adăugarea unei cifre la stânga** necesită determinarea unei puteri a lui 10 corespunzătoare numărului de cifre din `nr`. Se folosește o variabilă `putere`, inițializată cu 1, care se înmulțește cu 10 pentru fiecare cifră. Apoi se calculează: `rezultat = cifra * putere + nr`



IMPORTANT

În Python, operatorul `/` realizează împărțirea și returnează un rezultat cu zecimale (de exemplu, `4527 / 10` produce `452.7`). Pentru a obține direct partea întreagă a câtului se folosește operatorul `//`, numit împărțire întreagă: `4527 // 10` produce `452`.



PROBLEME REZOLVATE

Exemplul 1. Aflarea ultimei cifre a unui număr

Se citește un număr natural `n`. Să se afișeze ultima cifră a numărului.

Exemplu: `n = 4527`. Ieșire așteptată: `7`.

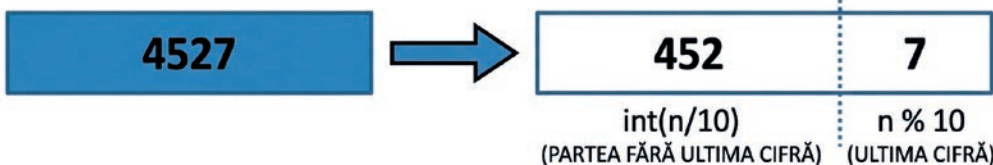
Cod Python

```
n = int(input("Numărul n este: "))
uc = n % 10
print("Ultima cifră este:", uc)
```

PAS 1

`n = 4527` → Operație: `4527 % 10`

Împărțim: `4527 ÷ 10 = 452` rest `7`



PAS 2

Rezultat ultima cifră = 7

Fig. 1 – Extragerea ultimei cifre cu operația MODULO



PROBLEME REZOLVATE

Exemplul 2. Numărarea cifrelor unui număr

Se citește un număr natural n ($n > 0$).

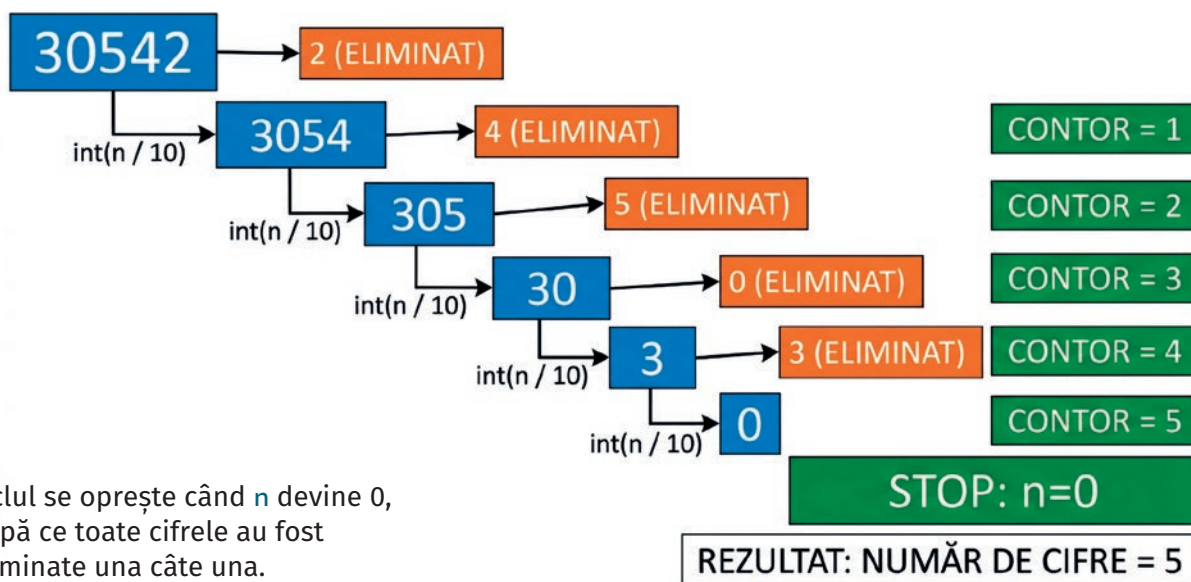
Să se determine câte cifre are numărul.

Exemplu: $n = 30542$. Ieșire așteptată: 5.

Cod Python

```
n = int(input("Numărul n este: "))
contor = 0
while n > 0:
    n = n // 10
    contor = contor + 1
print("Număr de cifre =", contor)
```

Fig. 2 – Numărarea cifrelor prin eliminare repetată



Ciclul se oprește când n devine 0, după ce toate cifrele au fost eliminate una câte una.

Exemplul 3. Adăugarea unei cifre la dreapta unui număr

Se citesc un număr natural n și o cifră c . Să se construiască numărul obținut prin adăugarea cifrei c la dreapta lui n .

Exemplu: $n = 123$, $c = 7$.

Ieșire așteptată: 1237.

Cod Python

```
n = int(input("Numărul n este: "))
c = int(input("Cifra c este: "))
rezultat = n * 10 + c
print("Rezultat =", rezultat)
```

Fig. 3 – Adăugarea unei cifre la dreapta

Adăugarea cifrei 7 la dreapta numărului 123

Număr inițial:

1 2 3

Pas 1: $123 \times 10 = 1230 \rightarrow$ se „face loc” la dreapta

1 2 3 0

Pas 2: $1230 + 7 = 1237 \rightarrow$ se „pune” cifra 7 pe locul liber

1 2 3 7



PROBLEME REZOLVATE

Exemplul 4. Adăugarea unei cifre la stânga unui număr

Se citește un număr natural n și o cifră c . Să se construiască numărul obținut prin adăugarea cifrei c la stânga lui n .

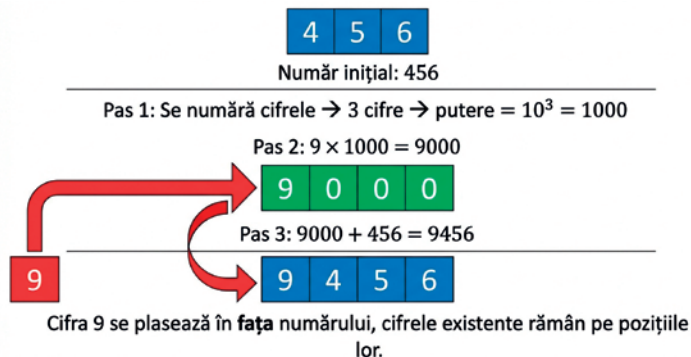
Exemplu: $n = 456$, $c = 9$. Ieșire așteptată: 9456.

Cod Python

```
n = int(input("n = "))
c = int(input("c = "))
putere = 1
copie = n
while copie > 0:
    putere = putere * 10
    copie = copie // 10
    rezultat = c * putere + n
    print("Rezultat =", rezultat)
```

Fig. 4 – Adăugarea unei cifre la stânga

Adăugarea cifrei 9 la stânga numărului 456



Se observă cum variabila **putere** crește de la 1 la 1000, parcurgând cifrele lui 456. Astfel, cifra 9 este plasată corect la stânga numărului.



APLICAȚII

1. Se citește un număr natural n . Să se calculeze suma cifrelor sale. Indicații: se inițializează suma cu 0, se extrage ultima cifră cu $n \% 10$, se adaugă la sumă, se elimină ultima cifră cu $n // 10$ și se repetă cât timp $n > 0$.
2. Se citește un număr natural n . Să se construiască și să se afișeze *oglinditul* (inversarea cifrelor). Exemplu: $n = 1234$, oglinditul este 4321.

Cod Python

```
n = int(input("n = "))
oglindit = 0
while n > 0:
    cifra = n % 10
    oglindit = oglindit * 10 + cifra
    n = n // 10
print("Oglindit =", oglindit)
```

Explicații

- se inițializează **oglindit** = 0;
- la fiecare pas se extrage ultima cifră și se adaugă la dreapta lui oglindit;
- pentru $n = 1234$:
 - pas 1: $oglindit = 0 \cdot 10 + 4 = 4$
 - pas 2: $oglindit = 4 \cdot 10 + 3 = 43$
 - pas 3: $oglindit = 43 \cdot 10 + 2 = 432$
 - pas 4: $oglindit = 432 \cdot 10 + 1 = 4321$
- pentru $n = 50810$, oglinditul este 1805: zeroul interior se păstrează, dar cel de la final dispare.

3. Se citește un număr natural n ($n > 0$). Să se determine cea mai mică cifră nenulă a numărului. Exemplu: $n = 30542$, rezultatul este 2.
4. Se citește un număr natural n . Să se afișeze câte cifre pare are numărul. Exemplu: $n = 284613$, rezultatul este 4.

1.2.2. Divizori, multipli și CMMDC (cel mai mare divizor comun)

1.2.2.1. Divizori ai unui număr

Un număr d este **divizor** al lui n dacă restul împărțirii lui n la d este zero, ceea ce înseamnă că $n \% d == 0$. Echivalent, n este **multiplu** al lui d .

Divizibilitatea se verifică în Python folosind operatorul **% (modulo)**. Dacă $n \% d == 0$, atunci d este divizor al lui n . De exemplu, 3 este divizor al lui 12 deoarece $12 \% 3 = 0$.

Pentru a găsi toți divizorii unui număr n , se parcurg toate valorile de la 1 la n și se verifică condiția de divizibilitate. Un caz special important este cel al **numerelor prime**: un număr natural n ($n \geq 2$) este **prim** dacă are exact doi divizori: 1 și n însuși.

>>>

EXEMPLE



Divizorii apar în multe situații practice. Să ne imaginăm că avem o bandă de hârtie cu lungimea de 30 cm și dorim să o tăiem în bucăți egale, fără să rămână rest. Cum procedăm? Trebuie să găsim toți divizorii lui 30: aceștia sunt 1, 2, 3, 5, 6, 10, 15 și 30. Fiecare dintre aceste valori reprezintă o lungime posibilă pentru bucăți. Dacă dorim bucăți cât mai lungi, alegem cel mai mare divizor mai mic decât 30, adică 15, și obținem 2 bucăți de câte 15 cm fiecare.

Un alt exemplu: la distribuirea a 24 de caiete în grupuri egale, posibilitățile sunt date de divizorii lui 24: 1, 2, 3, 4, 6, 8, 12, 24. Putem forma 6 grupuri a câte 4 caiete sau 8 grupuri a câte 3 caiete și așa mai departe.



PROBLEME REZOLVATE

Tăierea unei benzi în bucăți egale

Se citește lungimea L a unei benzi. Să se afișeze toate posibilitățile de tăiere în bucăți egale (adică toți divizorii lui L). Date de intrare: $L = 30$. Date de ieșire: 1 2 3 5 6 10 15 30.

Cod Python

```
L = int(input("Lungime bandă: "))
for d in range(1, L + 1):
    if L % d == 0:
        print(d, end=" ")
```

Explicații

- se citește lungimea benzii L ;
- se parcurg valorile d de la 1 la L ;
- pentru fiecare d , se testează dacă $L \% d == 0$ (dacă d divide pe L);
- dacă da, d este o lungime posibilă pentru bucăți egale;
- pentru $L = 30$:
 - $d=1$: $30 \% 1=0$, se afișează 1
 - $d=2$: $30 \% 2=0$, se afișează 2
 - $d=5$: $30 \% 5=0$, se afișează 5
 - ...
 - $d=30$: $30 \% 30=0$, se afișează 30

1.2.2.2. Multiplii unui număr

Cel mai mic multiplu comun (CMMMC) a două numere este cel mai mic număr natural nenul care este multiplu al ambelor numere. De exemplu, multiplii lui 4 sunt: 4, 8, 12, 16, 20, 24, ..., iar multiplii lui 6 sunt: 6, 12, 18, 24, ... Cel mai mic număr care apare în ambele șiruri este 12, deci **CMMMC(4, 6) = 12**.

Există o legătură directă între CMMDC și CMMMC, exprimată prin formula:

CMMMC(a, b) = a * b // CMMDC(a, b).



>>> EXEMPLE

Exemplul 1: Se consideră numerele $a = 12$ și $b = 18$. Se determină mai întâi cel mai mare divizor comun: $\text{CMMDC}(12, 18) = 6$. Se aplică formula pentru cel mai mic multiplu comun: $\text{CMMC}(12, 18) = (12 * 18) // 6 = 216 // 6 = 36$. Se observă că 36 se împarte exact atât la 12, cât și la 18, deci reprezintă cel mai mic multiplu comun.

Exemplul 2: Se consideră două semafoare într-o intersecție. Primul are un ciclu de a secunde, iar al doilea un ciclu de b secunde. Presupunând că ambele semafoare pornesc în același moment, se cere să se determine după câte secunde cele două semafoare vor afișa simultan aceeași culoare. Această situație se modelează folosind cel mai mic multiplu comun. Se consideră $a = 40$ și $b = 60$. Se determină mai întâi cel mai mare divizor comun: $\text{CMMDC}(40, 60) = 20$. Se aplică formula: $\text{CMMC}(40, 60) = (40 * 60) // 20 = 2400 // 20 = 120$. Așadar, cele două semafoare vor afișa simultan aceeași culoare după **120 de secunde**.

Cod Python

```
# a și b numere naturale,  
# nenule
```

```
a = int(input("a = "))  
b = int(input("b = "))  
while a != b:  
    if a > b:  
        a = a - b  
    else:  
        b = b - a  
print("CMMDC =", a)
```

1.2.2.3. Cel mai mare divizor comun a două numere naturale nenule (CMMDC) și Algoritmul lui Euclid

Cel mai mare divizor comun (CMMDC) a două numere naturale nenule este cel mai mare număr natural care le divide pe ambele. De exemplu, $\text{CMMDC}(12, 18) = 6$, deoarece 6 este cel mai mare număr care divide și pe 12, și pe 18.

» Algoritmul lui Euclid – varianta cu scăderi repetate

Acest algoritm se bazează pe o proprietate matematică simplă: dacă două numere a și b au un divizor comun d , atunci și diferența lor ($a - b$) este divizibilă cu d . De aceea, se poate înlocui numărul mai mare cu diferența celor două, fără să se modifice CMMDC-ul. Matematic, se scrie:

$$\text{CMMDC}(a, b) = \text{CMMDC}(a - b, b),$$

pentru $a > b$. Concret, algoritmul funcționează astfel: se compară cele două numere; din cel mai mare se scade cel mai mic. Se repetă acest pas până când cele două numere devin egale. Valoarea comună obținută este CMMDC-ul.

În figura 5 sunt ilustrați pașii algoritmului pentru $a = 48$ și $b = 18$. Rezultatul este 6. Acest rezultat are și o interpretare practică: dacă dorim să acoperim o podea de $48 \text{ m} \times 18 \text{ m}$ cu plăci de gresie pătrate cât mai mari, latura maximă a plăcii este 6 m.

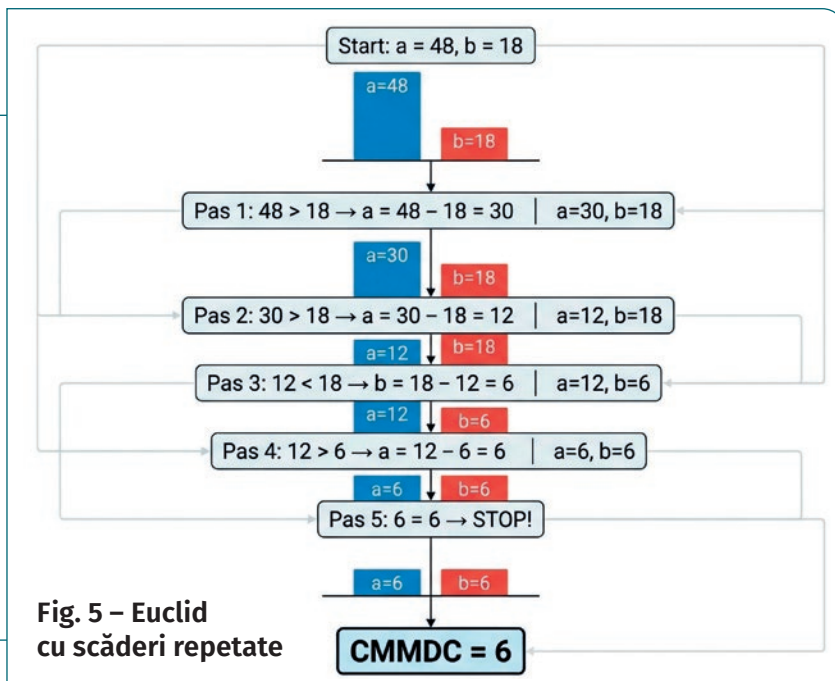


Fig. 5 – Euclid cu scăderi repetate



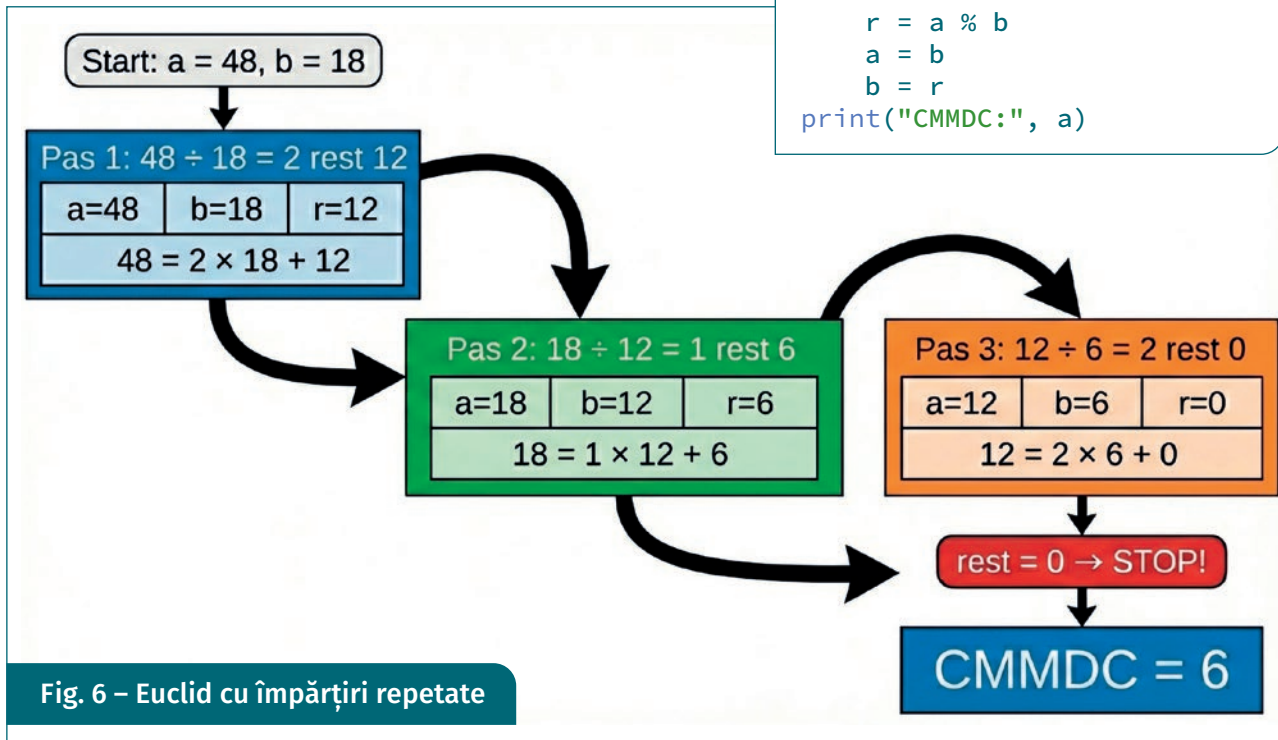
» Algoritmul lui Euclid – varianta cu împărțiri repetate

Această variantă este mai eficientă deoarece în loc să se efectueze scăderi repetate, se înlocuiește numărul mai mare cu restul împărțirii. Proprietatea folosită este: $\text{CMMDC}(a, b) = \text{CMMDC}(b, a \% b)$. Procesul se repetă până când restul devine 0, iar CMMDC-ul este ultimul rest nenul.

Să se urmărească algoritmul pentru $a = 48$ și $b = 18$. Se observă că metoda bazată pe împărțiri necesită un număr mai mic de pași decât metoda bazată pe scăderi.

Cod Python

```
a = int(input("a = "))
b = int(input("b = "))
while b != 0:
    r = a % b
    a = b
    b = r
print("CMMDC:", a)
```



PROBLEME REZOLVATE

Exemplul 1. Verificarea dacă un număr este divizor al altuia

Se citesc două numere naturale a și b ($b \neq 0$). Să se verifice dacă b este divizor al lui a .

Exemplu: $a = 15, b = 3$. Ieșire: 3 este divizor al lui 15.

Cod Python

```
a = int(input("a = "))
b = int(input("b = "))
if a % b == 0:
    print(b, "este divizor al lui", a)
else:
    print(b, "nu este divizor al lui", a)
```

Explicații

- se citesc două numere a și b ;
- se verifică dacă restul împărțirii lui a la b este 0;
- dacă $a \% b == 0$, atunci b divide pe a exact (fără rest);
- pentru $a = 15, b = 3$: $15 \% 3 = 0$, deci 3 este divizor al lui 15.



PROBLEME REZOLVATE

Exemplul 2. Verificarea dacă un număr este prim

Se citește un număr natural n ($n \geq 2$).

Să se verifice dacă n este prim. Un număr este prim dacă are exact 2 divizori: 1 și el însuși.

Exemplu: $n = 13$. Ieșire: 13 este număr prim.

Fig. 7 – Verificarea primalității



Divizorii lui 13: 1, 13

$d = 1 \rightarrow 13 \% 1 = 0$ ✓ (contor = 1)

$d = 2 \rightarrow 13 \% 2 = 1$ ✗

$d = 3 \rightarrow 13 \% 3 = 1$ ✗

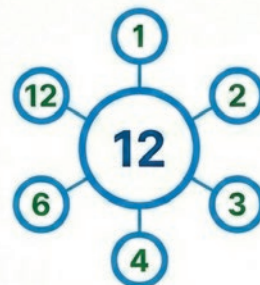
$d = 4 \rightarrow 13 \% 4 = 1$ ✗

...

$d = 12 \rightarrow 13 \% 12 = 1$ ✗

$d = 13 \rightarrow 13 \% 13 = 0$ ✓ (contor = 2)

Concluzie: contor = 2 $\rightarrow$ exact 2 divizori $\rightarrow$ PRIM ✓



Divizorii lui 12:
1, 2, 3, 4, 6, 12

Cod Python

```
n = int(input("n = "))
contor = 0
for d in range(1, n + 1):
    if n % d == 0:
        contor = contor + 1
if contor == 2:
    print("Este număr prim")
else:
    print("Nu este număr prim")
```

Exemplul 3. Numărarea pașilor algoritmului lui Euclid

Se citesc două numere naturale a și b . Să se afișeze câți pași necesită algoritmul lui Euclid cu împărțiri repetate pentru a determina CMMDC-ul lor.

Exemplu: $a = 48$, $b = 18$. Ieșire: CMMDC = 6, pași = 3.

Cod Python

```
a = int(input("a = "))
b = int(input("b = "))
pasi = 0
while b != 0:
    r = a % b
    a = b
    b = r
    pasi = pasi + 1
print("CMMDC =", a)
print("Pași:", pasi)
```

Explicații

- se adaugă o variabilă `pasi` care se incrementează la fiecare iterație;
- pentru $a = 48$, $b = 18$:
 - pas 1: $r=48\%18=12$, $a=18$, $b=12$
 - pas 2: $r=18\%12=6$, $a=12$, $b=6$
 - pas 3: $r=12\%6=0$, $a=6$, $b=0$
- CMMDC = 6, obținut în 3 pași.



PROBLEME REZOLVATE

Exemplul 4. Operații cu fracții

Proiectează un program care simplifică o fracție la forma ireductibilă, folosind algoritmul lui Euclid pentru calculul CMMDC-ului. De exemplu, fracția $12/18$ se simplifică la $2/3$.

Exemplu: $a = 12, b = 18$. Ieșire: $2/3$.

Cod Python

```
# a și b numere naturale, b nenul
a = int(input("a = "))
b = int(input("b = "))

x, y = a, b
while y != 0:
    r = x % y
    x = y
    y = r
cmmdc = x
print(a // cmmdc, "/", b // cmmdc)
```

Explicații

- se salvează copii ale lui a și b în x și y pentru a calcula CMMDC;
- se aplică Euclid cu împărțiri repetate:
 - $x=12, y=18 \rightarrow r=12, x=18, y=12$
 - $x=18, y=12 \rightarrow r=6, x=12, y=6$
 - $x=12, y=6 \rightarrow r=0, x=6, y=0$
- CMMDC = 6;
- se împart a și b la CMMDC: $12/6 = 2, 18/6 = 3$;
- rezultat: fracția $2/3$.



APLICAȚII

1. Extinde **Exemplul 4** pentru a realiza și adunarea a două fracții: $a_1/b_1 + a_2/b_2 = (a_1*b_2 + a_2*b_1) / (b_1*b_2)$, apoi simplifică rezultatul.
2. O cameră are dimensiunile a și b (în metri). Se dorește acoperirea podelei cu plăci de gresie pătrate cât mai mari. Determină latura maximă a plăcii (CMMDC) și numărul de plăci necesare. Exemplu: $a = 36, b = 24$, latura = CMMDC(36, 24) = 12, număr plăci = $(36/12) * (24/12) = 6$
3. Se citesc două numere a și b . Calculează CMMMC-ul celor două numere folosind formula CMMMC(a, b) = $a * b // \text{CMMDC}(a, b)$ și afișează rezultatul.
Exemplu: $a = 12, b = 18 \rightarrow \text{CMMMC} = 36$.
4. Evaluează diferența de complexitate între parcurgerea divizorilor de la 1 la n , de la 1 la $n/2$ și de la 1 la $\sqrt{n}$. Pentru $n = 100$, câte verificări se fac în fiecare caz?

1.2.3. Prelucrarea numerelor și descompunerea în factori primi

În lecțiile anterioare s-au studiat modalitățile de acces la cifrele unui număr și lucrul cu divizorii. În lecția curentă, aceste cunoștințe sunt utilizate împreună pentru rezolvarea unor probleme mai complexe, precum verificarea proprietăților cifrelor, construirea de numere noi și descompunerea în factori primi.

Pentru a prelucra cifrele unui număr, se extrage pe rând ultima cifră ($n \% 10$), apoi aceasta se elimină din număr prin împărțire la 10. Procesul se repetă cât timp numărul este mai mare decât 0. Pentru a determina divizorii unui număr, se verifică fiecare număr de la 1 până la n . Un număr d este divizor dacă $n \% d == 0$.



IMPORTANT

Atunci când este necesară atât parcurgerea cifrelor, cât și păstrarea valorii originale a numărului, se salvează o copie a numărului înainte de parcurgere, deoarece operația de eliminare a cifrelor modifică valoarea variabilei.



Orice număr natural mai mare decât 1 poate fi scris ca produs de numere prime. Această scriere se numește **descompunere în factori primi** și este unică pentru fiecare număr.

De exemplu, $60 = 2 \times 2 \times 3 \times 5$, care se poate scrie și sub forma: $60 = 2^2 \times 3^1 \times 5^1$.

Pentru a descompune un număr în factori primi, se procedează astfel: se începe cu cel mai mic număr prim, 2, și se împarte numărul la 2 cât timp este posibil. Apoi se trece la următorul posibil divizor și se repetă procesul, până când numărul devine 1.



IMPORTANT

Nu este necesar să se verifice dacă un divizor este prim, deoarece, dacă ar avea alți divizori mai mici, aceștia ar fi fost deja utilizați în pașii anteriori.



PROBLEME REZOLVATE

Exemplul 1. Verificarea dacă toate cifrele sunt pare

Se citește n . Să se verifice dacă toate cifrele numărului sunt pare.

Exemplu: $n = 2486 \rightarrow \text{DA}$; $n = 2483 \rightarrow \text{NU}$.

DIAGRAMĂ DE VERIFICARE A CIFRELOR PARE

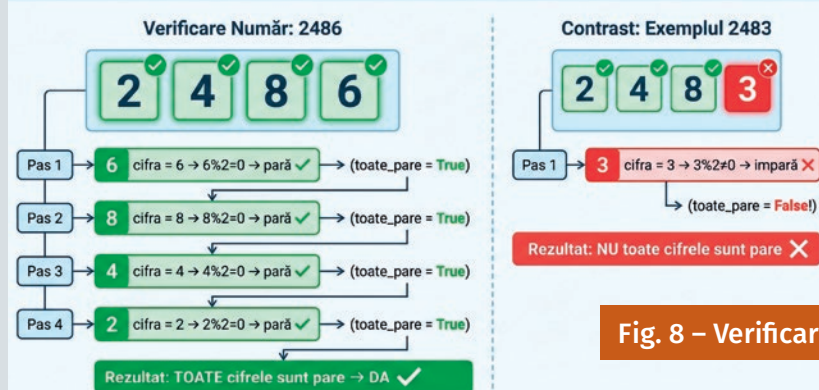


Fig. 8 – Verificarea proprietăților cifrelor

Cod Python

```
n = int(input("n = "))
toate_pare = True
while n > 0:
    cifra = n % 10
    if cifra % 2 != 0:
        toate_pare = False
    n = n // 10
if toate_pare:
    print("DA")
else:
    print("NU")
```

Exemplul 2. Construirea numărului din cifrele pare

Se citește n . Să se construiască numărul format doar din cifrele pare ale lui n , păstrând ordinea lor.

Exemplu: $n = 132648$, rezultatul este 2648.

Cod Python

```
n = int(input("n = "))
rezultat = 0
putere = 1
while n > 0:
    cifra = n % 10
    if cifra % 2 == 0:
        rezultat = rezultat + cifra * putere
        putere = putere * 10
    n = n // 10
print(rezultat)
```

Explicații

- se parcurg cifrele de la dreapta;
- dacă cifra este pară, se adaugă la rezultat folosind variabila **putere**;
- putere se înmulțește cu 10 la fiecare cifră pară adăugată;
- pentru $n = 132648$:
cifra=8 (pară): rezultat=8, putere=10
cifra=4 (pară): rezultat=48, putere=100
cifra=6 (pară): rezultat=648, putere=1000
cifra=2 (pară): rezultat=2648, putere=10000
cifra=3 (impară): se sare
cifra=1 (impară): se sare
- rezultat: 2648.



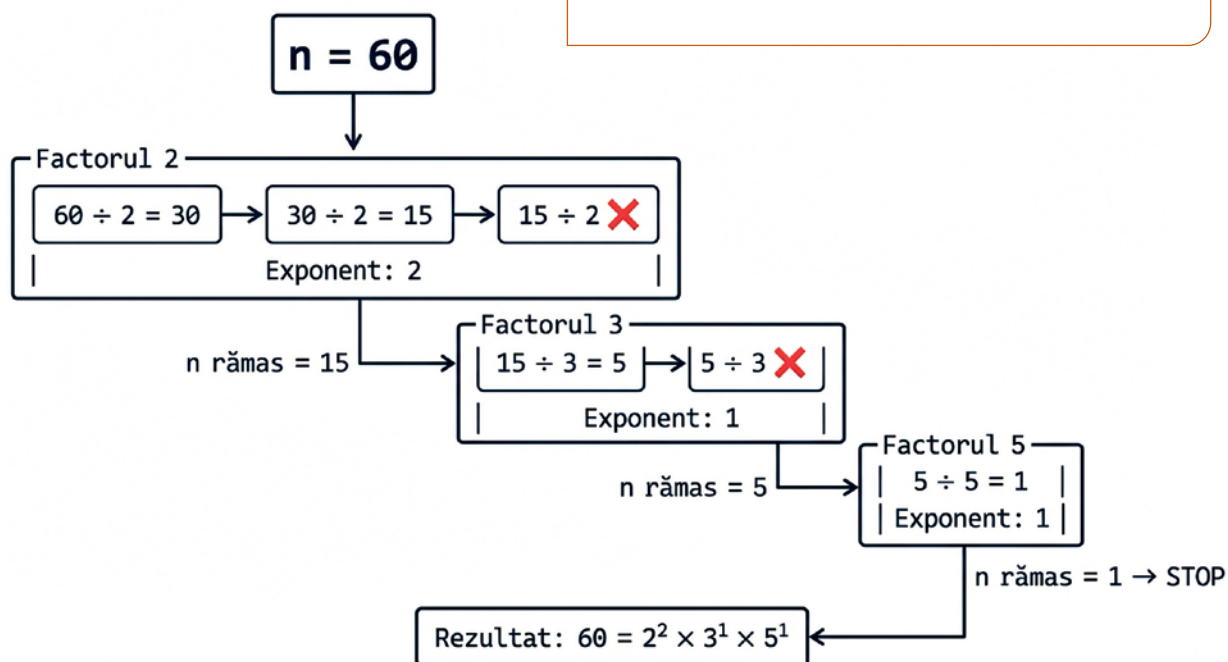
PROBLEME REZOLVATE

Exemplul 3. Descompunerea în factori primi

Se citește n ($n \geq 2$). Să se afișeze descompunerea lui n în factori primi.

Exemplu: $n = 60$. Ieșire: $2^2, 3^1, 5^1$ (în consolă, afișarea este pe verticală).

Fig. 9 – Descompunerea în factori primi



Cod Python

```
n = int(input("n = "))
d = 2
while n > 1:
    e = 0
    while n % d == 0:
        n = n // d
        e = e + 1
    if e > 0:
        print(str(d) + "^" + str(e))
    d = d + 1
```



IMPORTANT

Instrucțiunea `print(str(d) + "^" + str(e))` afișează valorile variabilelor `d` și `e` sub forma d^e , folosind `str()` cu rolul de a transforma numerele în text, pentru a fi concatenate cu operatorul `+`.



APLICAȚII

1. Se citește n . Să se numere câte cifre nenule ale lui n sunt și divizori ai lui n . De exemplu, pentru $n = 1232$, cifrele 1, 2, 2 sunt divizori, iar 3 nu este → rezultat: 3.
2. Se citește n ($n \geq 2$). Să se determine cel mai mare divizor prim al lui n . Indicații: se parcurg toți divizorii și se verifică dacă sunt primi.

3. Se citește n . Să se calculeze produsul cifrelor impare. Dacă nu există cifre impare, se afișează 0. Exemplu: $n = 23456 \rightarrow 3 * 5 = 15$.
4. Se citesc n ($n \geq 2$) și un număr prim p . Să se determine de câte ori apare p în descompunerea lui n . Exemplu: $n = 72, p = 2 \rightarrow$ rezultat: 3.



1.2.4. Transformarea numerelor între baze de numerație



În viața de zi cu zi folosim baza 10 (sistemul zecimal), dar calculatoarele lucrează în baza 2 (sistemul binar). Există și alte baze utilizate frecvent în informatică: baza 8 (octal) și baza 16 (hexazecimal). De exemplu, codul ASCII al literei „A” este 65 în baza 10.

Un număr reprezentat într-o bază b folosește cifre de la 0 la $b - 1$. Valoarea unui număr $c_3c_2c_1c_0$ scris în baza b se calculează ca: $c_3 \times b^3 + c_2 \times b^2 + c_1 \times b^1 + c_0 \times b^0$.

Pentru a transforma un număr **din baza 10 în baza b** , se aplică împărțiri succesive la b ($2 \leq b \leq 9$): la fiecare pas, restul ($n \% b$) furnizează o cifră,

iar câtul ($n // b$) devine noul număr. Cifrele obținute, citite în ordine inversă, formează reprezentarea în baza b .

Transformarea inversă, **din baza b ($2 \leq b \leq 9$) în baza 10**, se realizează prin parcurgerea cifrelor numărului și calcularea sumei ponderate: fiecare cifră se înmulțește cu puterea corespunzătoare a bazei.

Pentru a transforma un număr **dintr-o bază b_1 ($2 \leq b_1 \leq 9$) într-o altă bază b_2 ($2 \leq b_2 \leq 9$)**, se folosește baza 10 ca intermediar: mai întâi se transformă din baza b_1 în baza 10, apoi din baza 10 în baza b_2 .



PROBLEME REZOLVATE

Exemplul 1. Transformarea din baza 10 în baza 2

Se citește n în baza 10. Să se afișeze reprezentarea binară.

Exemplu: $n = 13$. Ieșire: 1101.

Verificare: $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$.

Cod Python

```
n = int(input("n = "))
rezultat = 0
putere = 1
while n > 0:
    cifra = n % 2
    rezultat = rezultat + cifra * putere
    putere = putere * 10
    n = n // 2
print(rezultat)
```

Fig. 10 – Transformarea din baza 10 în baza 2

Baza 10 → Baza 2 (Împărțiri repetate)

$$13 \div 2 = 6 \text{ rest } 1$$

$$6 \div 2 = 3 \text{ rest } 0$$

$$3 \div 2 = 1 \text{ rest } 1$$

$$1 \div 2 = 0 \text{ rest } 1$$

Citește
resturile
de jos în sus

Rezultat: 1 1 0 1 (binar)

Verificare (Baza 2 → Baza 10)

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 8 + 4 + 0 + 1$$

$$= 13 \checkmark$$



PROBLEME REZOLVATE

Exemplul 2. Transformarea din baza 10 în baza b oarecare

Se citesc n și o bază b ($2 \leq b \leq 9$).
Să se afișeze reprezentarea lui n în baza b .

Exemplu: $n = 100, b = 8$. Ieșire: 144.

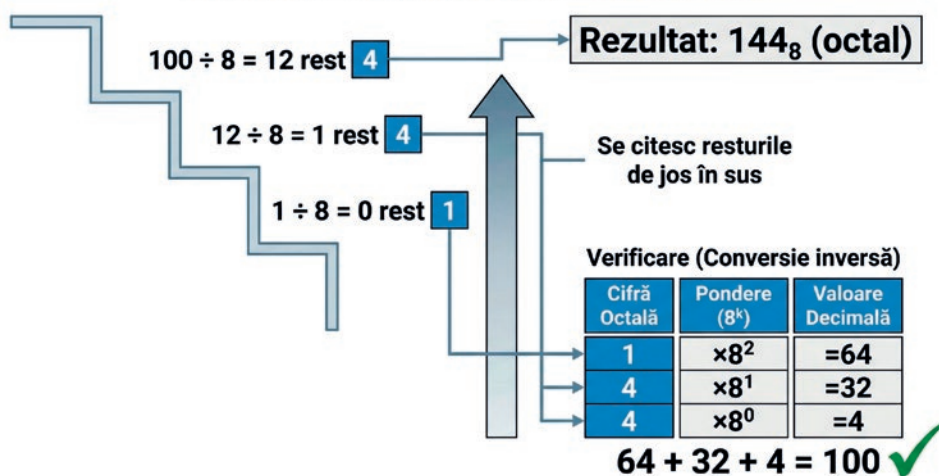
Verificare: $1 \times 8^2 + 4 \times 8^1 + 4 \times 8^0 = 100$.

Fig. 11 – Transformarea din baza 10 în baza b

Cod Python

```
n = int(input("n = "))
b = int(input("baza = "))
rezultat = 0
putere = 1
while n > 0:
    cifra = n % b
    rezultat = rezultat + cifra * putere
    putere = putere * 10
    n = n // b
print(rezultat)
```

Conversie: 100 (Baza 10) → Baza 8 (Octal)



Exemplul 3. Transformarea din baza b în baza 10

Se citesc un număr n (ale cărui cifre reprezintă un număr în baza b) și baza b ($2 \leq b \leq 9$). Să se calculeze valoarea în baza 10.

Exemplu: $n = 1101, b = 2$. Ieșire: 13.

Cod Python

```
n = int(input("n = "))
b = int(input("baza = "))
rezultat = 0
putere = 1
while n > 0:
    cifra = n % 10
    rezultat = rezultat + cifra * putere
    putere = putere * b
    n = n // 10
print(rezultat)
```

Explicații

- se parcurg cifrele numărului;
- fiecare cifră se înmulțește cu puterea corespunzătoare a bazei;
- putere se înmulțește cu b la fiecare pas ($1, b, b^2, b^3, \dots$);
- pentru $n=1101, b=2$:
 - cifra=1: rezultat=1*1=1, putere=2
 - cifra=0: rezultat=1+0*2=1, putere=4
 - cifra=1: rezultat=1+1*4=5, putere=8
 - cifra=1: rezultat=5+1*8=13, putere=16
- rezultat: 13.



ÎNVĂȚARE PRIN DESCOPERIRE

Exemplul 4. Transformarea din baza b_1 în baza b_2

Se citește n (în baza b_1), b_1 și b_2 . Să se transforme numărul n din baza b_1 în baza b_2 , realizând conversia în doi pași: mai întâi din baza b_1 în baza 10, apoi din baza 10 în baza b_2 . Pentru implementare, se vor urma etapele studiate anterior: extragerea cifrelor folosind operațiile $\%$ și $//$, utilizarea unei variabile putere pentru calculul valorii poziționale și construirea treptată a rezultatului, conform figurii 12.

Exemplu: $n = 1101$ (baza 2), $b_2 = 8$. Rezultat: 15 (baza 8).

Transformare în doi pași: BAZA 2 → BAZA 10 → BAZA 8

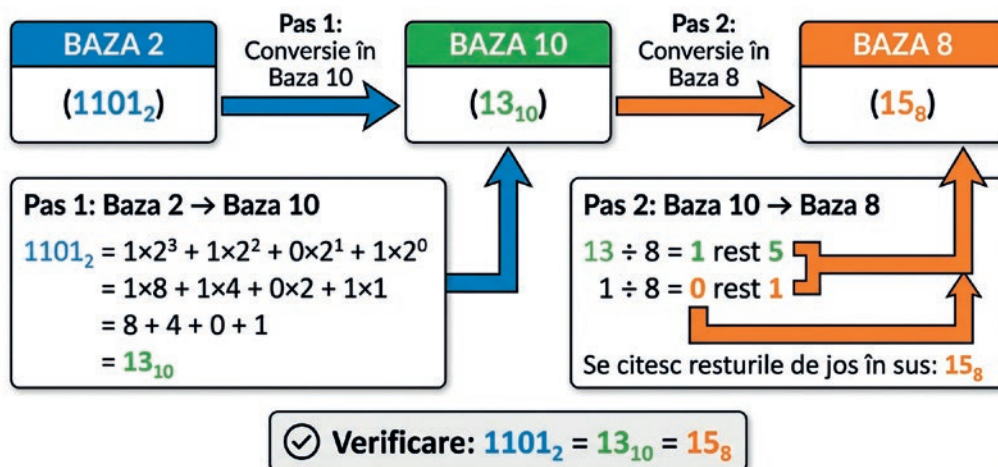


Fig. 12 – Transformarea între două baze de numerație



FIȘĂ DE LUCRU 2

1. Se citește n . Să se verifice dacă reprezentarea binară este palindrom.

Exemplu: $n = 9 \rightarrow$ binar $1001 \rightarrow$ DA.

2. Se citește un număr n (ale cărei cifre reprezintă un număr într-o bază b) și baza b , unde $2 \leq b \leq 9$. Să se calculeze valoarea numărului în baza 10, apoi să se determine cel mai mare divizor comun dintre această valoare și suma cifrelor lui n .

Exemplu: $n = 231$, $b = 4$.

Transformarea în baza 10: $231_4 = 2 \times 4^2 + 3 \times 4^1 + 1 \times 4^0 = 2 \times 16 + 3 \times 4 + 1 = 45$,
suma cifrelor lui n este 6, iar $\text{CMMDC}(45, 6) = 3$.

3. Se citește un număr natural n . Să se verifice dacă n este palindrom, adică se citește la fel în ambele sensuri. Pentru aceasta, se construiește oglinditul numărului și se compară cu numărul inițial. De exemplu, pentru $n = 12321$, oglinditul este tot 12321, deci numărul este palindrom. În schimb, pentru $n = 1200$, oglinditul este 21, iar numărul nu mai coincide cu cel inițial, deci nu este palindrom. Se observă că aplicarea de două ori a operației de oglindire nu conduce întotdeauna la numărul inițial, deoarece zerourile de la finalul numărului se pierd în procesul de construire a oglinditului.

RECAPITULARE

1.3. Recapitulare

1. Pentru fiecare afirmație de mai jos, să se stabilească dacă este adevărată sau falsă.

- ☐ Operația $n \% 10$ returnează prima cifră a numărului n .
- ☐ Funcția `input()` returnează întotdeauna o valoare numerică.
- ☐ Algoritmul lui Euclid cu împărțiri repetate este mai eficient decât cel cu scăderi.
- ☐ Operatorul `=` realizează atribuirea unei valori unei variabile.
- ☐ Un număr prim are exact doi divizori: 1 și el însuși.
- ☐ Instrucțiunile dintr-un program sunt executate la întâmplare.
- ☐ Pentru a transforma din baza 3 în baza 7, se folosește baza 10 ca intermediar.
- ☐ `Oglindit(oglindit(n))` este întotdeauna egal cu n .
- ☐ Instrucțiunea `while` se execută cel puțin o dată, indiferent de condiție.
- ☐ Numerele trebuie scrise între ghilimele.

2. Se consideră următorul program. Ce se afișează pentru `nr = 1234`?

- a) Urmărește execuția pas cu pas, completând un tabel cu valorile variabilelor `nr`, `c` și `suma`.
- b) Precizează ce calculează algoritmul.
- c) Modifică algoritmul astfel încât să calculeze produsul cifrelor impare.

Cod Python

```
nr = int(input("nr = "))
suma = 0
while nr > 0:
    c = nr % 10
    if c % 2 == 0:
        suma = suma + c
    nr = nr // 10
print(suma)
```

3. Se consideră următorul algoritm scris în pseudocod. Ce se va afișa în urma executării instrucțiunilor? Operatorul `%` reprezintă restul împărțirii.

```
start
  a ← 10
  b ← 2

  cât_timp a < 15 executa
    dacă a mod b = 0 atunci
      scrie '#'
    sfârșit_dacă
    a ← a + 1
    b ← b + 1
  sfârșit_cât_timp
stop
```

```
a. #
#

b. #

c. #
#
#
```

4. Se citește un număr natural n , format din cel puțin două cifre. Construiește un nou număr obținut prin interschimbarea primei cifre cu ultima cifră a lui n .

Exemplu: $n = 1234 \rightarrow$ rezultatul este 4231.

5. Se citesc două numere naturale nenule a și b . Afișează cifrele comune celor două numere, o singură dată și în ordine crescătoare. Cifrele se vor afișa pe un singur rând, despărțite de câte un spațiu, iar dacă cele două numere citite nu au cifre comune se va afișa mesajul "Nu există".

Exemplu: dacă $a=522781$ și $b=327822$, se vor afișa cifrele 2 7 8

6. Se citește un număr natural n . Determină cifra de control a numărului, obținută prin însumarea repetată a cifrelor sale, până când rezultatul devine o singură cifră.

Exemplu: $n = 9875 \rightarrow 9+8+7+5 = 29 \rightarrow 2+9 = 11 \rightarrow 1+1 = 2$. Rezultat: 2.

7. Se citesc trei numere naturale a , b și k , cu $a \leq b$. Scrie un program care determină cel mai mare divizor comun al tuturor numerelor din intervalul $[a, b]$ care sunt multipli ai lui k .

Exemplu: $a = 10, b = 30, k = 6 \rightarrow$ multiplii lui 6 din $[10, 30]$ sunt: 12, 18, 24, 30 $\rightarrow$ CMMDC = 6.

MINIPROIECTE – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVI

1. Se citesc două numere naturale a și b . Verifică dacă cele două numere sunt **prietene**, adică suma divizorilor lui a (mai puțin a însuși) este egală cu b , iar suma divizorilor lui b (mai puțin b însuși) este egală cu a .

Exemplu: $a = 220, b = 284$. Divizorii lui 220, mai puțin 220: 1, 2, 4, 5, 10, 11, 20, 22, 44, 55, 110 $\rightarrow$ suma = 284. Divizorii lui 284, mai puțin 284: 1, 2, 4, 71, 142 $\rightarrow$ suma = 220. Rezultat: cele două numere sunt prietene.

- Realizează analiza problemei, identificând datele de intrare și datele de ieșire, apoi scrie algoritmul în pseudocod;
- Completează un tabel de urmărire pentru $a = 220$;
- Implementează algoritmul în Python.

2. Se citește un număr natural n . Să se construiască un nou număr format doar din cifrele impare ale lui n , păstrând ordinea în care acestea apar în număr.

Exemplu: pentru $n = 123456$, rezultatul este 135.

- Realizează analiza problemei, identificând datele de intrare și datele de ieșire, apoi scrie algoritmul în pseudocod;
- Completează un tabel de urmărire pentru $n = 123456$;
- Implementează algoritmul în Python.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

R E C A P I T U L A R E

1.4. Evaluare sumativă

I. Rezolvă.

1. Asociază fiecare termen din coloana **A** cu definiția corespunzătoare din coloana **B**.

5x0,1 p. = 0,5 p.

Coloana A	Coloana B
Pseudocod	Împărțirea unui algoritm în subprobleme independente, fiecare cu un rol clar.
Constantă	Numărul de operații crește proporțional cu dimensiunea datelor de intrare.
Expresie logică	O valoare care nu se modifică pe parcursul execuției programului.
Proiectare modulară	O combinație de valori și operatori care produce un rezultat de tip True sau False .
Complexitate $O(n)$	O formă intermediară de reprezentare a unui algoritm, între limbajul natural și un limbaj de programare.

2. Evaluează următoarele expresii Python, precizând valoarea și tipul rezultatului:

5x0,1 p. = 0,5 p.

- a) `int(3.9) + float(2)`
- b) `(7 + 3) * 2 // 6`
- c) `100 % 7 % 3`
- d) `not (10 == 10) or 3 != 4`
- e) `20 // 3 > 6 or 20 % 3 == 0`

3. Se citește n ($n > 1$). Verifică dacă n este **număr perfect**, adică dacă suma divizorilor lui n , mai puțin n însuși, este egală cu n . Implementează verificarea.

1 p.

Exemplu: $n = 6$, divizorii lui 6 mai puțin 6 sunt 1, 2, 3, iar $1 + 2 + 3 = 6 \rightarrow$ DA, este perfect.

4. Următorul program conține trei erori (de sintaxă, de logică sau de execuție). Identifică fiecare eroare, precizează tipul ei și scrie varianta corectă.

0,5 p.

```
x = int(input("Introduceti x: "))
y = int(input("Introduceti y: "))
if x = y:
    print("Numerele sunt egale")
else
    medie = x + y / 2
    print("Media:", medie)
```

R E C A P I T U L A R E

5. Se consideră următorul program scris în limbajul Python.
Să se determine ce valoare se afișează la finalul execuției programului
și să se precizeze rolul variabilei **ok** în cadrul algoritmului.

0,5 p.

```
x = 1
y = 0
ok = True
while x <= 5:
    if x % 2 == 0:
        y = y + x
    else:
        ok = not ok
    if ok == True:
        y = y + 1
    x = x + 1
print(y)
```

- a) 5
b) 6
c) 7
d) 8

6. Se citesc două numere naturale **a** și **b** ($a \leq b$). Scrie un program care calculează suma tuturor numerelor prime din intervalul **[a, b]**.

- a) Implementează algoritmul.
b) Descrie rezolvarea folosind modelul IPI.
c) Precizează complexitatea intuitivă a algoritmului și propune o optimizare.

1 p.
0,5 p.
0,25 p.
0,25 p.

7. Se citesc două numere **a** și **b** scrise în baza **b1**, precum și bazele **b1** și **b2**. Scrie un program care calculează suma **a + b** și afișează rezultatul în baza **b2**, folosind baza 10 ca intermediar. Mai exact, programul convertește mai întâi pe **a** și pe **b** din baza **b1** în baza 10, apoi calculează suma celor două valori obținute (în baza 10), iar la final convertește rezultatul sumei din baza 10 în baza **b2** și îl afișează.

Exemplu: **a = 1101** (baza 2), **b = 11** (baza 2), **b1 = 2**, **b2 = 8**.

Convertim mai întâi cele două numere în baza 10 și obținem **a = 13** și **b = 3**.

Calculăm apoi suma în baza 10: $13 + 3 = 16$.

În final, convertim 16 din baza 10 în baza 8 și obținem 20, care este rezultatul afișat. 1 p.

II. Miniproiect. La alegere, unul din cele două miniproiecte rezolvate în cadrul secțiunii de *Recapitulare*.

1x2 p. = 2 p.

III. Portofoliu. La finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul *Unitate1_NumePrenumeElev*, următoarele rezolvări: *Fișa de lucru 1* și *Fișa de lucru 2*.

2 x 1 p. = 2 p.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*. Se acordă 1 punct din oficiu.

CAPITOLUL 2

Tehnici de organizare a codului în limbaj de programare

COMPETENȚE
SPECIFICE:

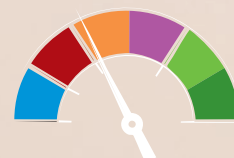
1.5,
2.3, 2.5,
3.3, 3.5,
4.3, 4.5,
5.3, 5.5,
6.3, 6.5

2.1. Subprograme

2.2. Introducere în Programarea Orientată pe Obiecte

2.3. RECAPITULARE

2.4. EVALUARE SUMATIVĂ



2.1. Subprograme

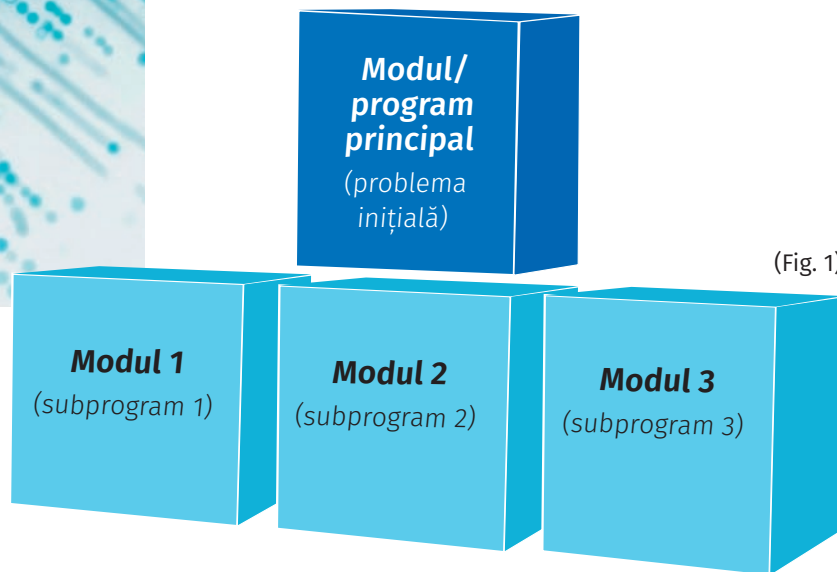
2.1.1. Subprograme – rol, caracteristici

În dezvoltarea aplicațiilor informatice, pe măsură ce problemele devin tot mai complexe, este necesară organizarea eficientă a codului pentru a-l face mai ușor de înțeles, de testat și de reutilizat.

Strategia de bază este de a descompune problema inițială în subprobleme (module), fiecare dintre ele având responsabilități unice și bine delimitate. Descompunerea poate continua până când se obține un modul cu rezolvare imediată sau un modul de complexitate mică. Această tehnică de proiectare modulară a algoritmilor poartă denumirea de **tehnică top-down**.

Fiecare subproblemă va fi descrisă printr-un **subprogram**.

În exemplul de mai jos, o problemă a fost descompusă în trei subprograme (Fig. 1).



(Fig. 1)

2.1.1.1. Clasificarea modulelor (subprogramelor) – din punct de vedere al ierarhizării

Modul (subprogram) apelant – este modulul care, pentru rezolvarea propriei probleme, apelează la alte module (fiecare rezolvând o anumită sarcină). În momentul apelului, modulul apelant *transferă controlul* modulului apelat pentru a fi executat. În diagrama anterioară, *Modulul principal* este modulul apelant.

Modul (subprogram) apelat – este modulul care este apelat de un alt modul pentru a-i rezolva sarcina. După ce își termină execuția, modulul apelat *redă controlul* modulului apelant. În diagrama anterioară *Modul 1*, *Modul 2* și *Modul 3* sunt module apelate.



STUDIU DE CAZ 1

Enunțul problemei: Utilizează rețeta „Sos de roșii” pentru a prepara o pizza.

Analiza problemei: Rețeta „Sos de roșii” are rolul de a prepara acest tip de sos utilizând un set de pași clar și logic. Sosul este o „parte” folosită în prepararea mai multor feluri de mâncare (pizza, paste, lasagna) și are o rețetă cunoscută (este scrisă deja).

Analogia cu informatica: Rețeta „Sos de roșii” = subprogram (modul apelat); rețeta de pizza (paste/ lasagna) = program (modul apelant).



STUDIU DE CAZ 2

Enunțul problemei: Se consideră un șir de n (n – număr natural nenul) valori de numere naturale nenule. Determină numărul de valori din șir care au suma cifrelor un număr prim.

Analiza problemei:

- date de intrare: n și elementele șirului (fiecare element notat cu a);
- date de ieșire: numărul nr al valorilor din șir care respectă condiția din enunț.

Proiectarea modulară a algoritmului: Pentru fiecare valoare din șir, avem același tip de calcul: suma cifrelor, iar pentru fiecare sumă obținută – se verifică dacă este număr prim sau nu.

Descompunerea problemei în subprograme

- **Modul1** (subprogram 1) – calculează suma cifrelor unui număr natural nenul dat.
- **Modul2** (subprogram 2) – verifică dacă un număr natural nenul dat este număr prim.
- **Modul principal** (program principal) – pentru fiecare valoare citită, se utilizează Modul1 pentru a calcula suma cifrelor, iar pentru suma obținută se utilizează Modul2 – pentru a testa dacă suma este un număr prim.

Proiectare

– reprezentarea algoritmului în limbaj pseudocod

```
Modul principal – definiție modul apelant
//date de intrare – n și a: naturale
//date de ieșire – nr: natural
//date de manevră – x, y, i: naturale

citește n
nr ← 0
pentru i ← 1, n execută
    citește a
    //apelează Modul1 pentru valoarea a,
    //iar suma cifrelor lui a se memorează
    //în variabila x
    x ← apel Modul1(a)
    //apelează Modul2 pentru valoarea x,
    //iar y reține rezultatul
    //subprogramului (1 sau 0) după cum
    //x este număr prim sau nu
    y ← apel Modul2(x)
    dacă y = 1 atunci
        nr ← nr + 1
    sfârșit_dacă
sfârșit_pentru
scrie nr
```

Modul1(a) – definiție modul apelat

```
// date de intrare – a: natural
//date de ieșire – suma: natural

suma ← 0
cât_timp a ≠ 0 execută
    suma ← suma + a mod 10
    a ← a div 10
sfârșit_cât_timp
Ieșire cu suma din Modul1
```

Modul2(x) – definiție modul apelat

```
//date de intrare – x: natural
//date de ieșire – prim: natural

prim ← 1
dacă x < 2
    atunci prim ← 0
sfârșit_dacă
d ← 2
cât_timp d * d ≤ x AND prim = 1 execută
    dacă x mod d = 0
        atunci prim ← 0
    sfârșit_dacă
    d ← d + 1
sfârșit_cât_timp
Ieșire cu prim din Modul2
```

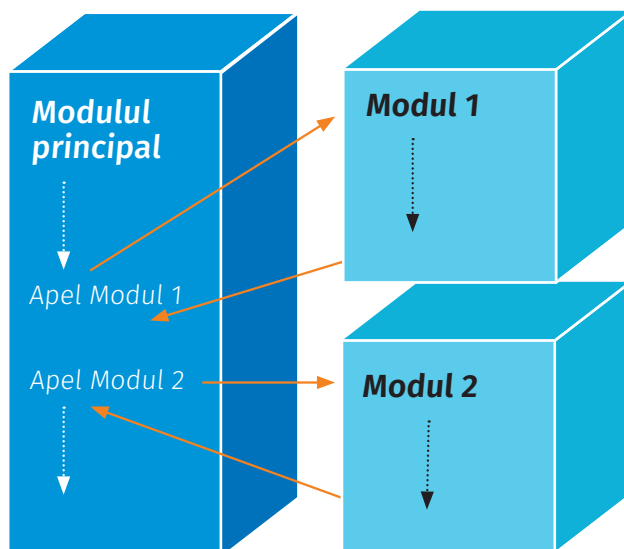




Fluxul de execuție (ordinea executării instrucțiunilor) (fig. 2)

Fluxul de execuție urmează o structură logică precisă, de sus în jos, dar cu „sărituri” între liniile de cod atunci când subprogramele sunt apelate. Iată etapele principale ale acestui proces:

1. **Execuția modulului principal** – instrucțiunile din interior sunt executate secvențial (una după alta).
2. **Apelarea unui subprogram** – când întâlnește un apel de subprogram (de exemplu, *Apel Modul1(a)*): execuția în punctul curent este suspendată temporar și controlul algoritmului „sare” la prima linie din interiorul subprogramului apelat.
3. **Execuția corpului subprogramului** – instrucțiunile din interiorul subprogramului sunt executate secvențial.
4. **Returnarea controlului** – are loc în două situații:
 - ▶ returnează o valoare și oprește imediat execuția subprogramului;
 - ▶ ajunge la final: subprogramul se termină după executarea ultimei instrucțiuni.



(Fig. 2)

În ambele situații, controlul programului se întoarce exact în locul de unde a fost făcut apelul, continuând cu următoarea instrucțiune a modulului principal.



APLICAȚII

Propune o variantă de rezolvare modulară, în limbaj pseudocod, pentru următoarele probleme.

1. Să se calculeze numerele „mistice” mai mici decât un număr n natural nenul dat (numărul natural x se numește număr „mistic” dacă suma cifrelor care îl compun este egală cu produsul lor; exemple: 22, 13131).
2. Se consideră datele despre n cumpărături (n – număr natural nenul), iar pentru fiecare cumpăratură se cunosc **prețul** (un număr real nenul) și **numărul de bucăți** (un număr natural nenul). Să se calculeze costul total al cumpărăturilor.

2.1.1.2. Concluzii și caracteristici generale

- ▶ Subprogramul **este reprezentat de un set de instrucțiuni care rezolvă o anumită sarcină**, este identificabil printr-un **nume** și poate fi **lansat în execuție (apelat)** ori de câte ori este cazul.
- ▶ Pentru a lucra cu un subprogram avem nevoie de: **definiția subprogramului** și de **apelul** acestuia.
- ▶ Un subprogram este lansat în execuție (apelat) în cod executabil.
- ▶ Cele două subprograme (apelat și apelant) pot comunica prin intermediul unor **parametri de comunicare**.
- ▶ În cele mai multe dintre limbajele de programare, subprogramele sunt plasate „înaintea” secțiunii de instrucțiuni a programului propriu-zis (numit, de obicei, **program principal**).
- ▶ Un subprogram are un grad mare de generalitate pentru problema pe care o rezolvă. El poate fi utilizat în mai multe aplicații și poate fi înglobat într-o bibliotecă a limbajului de programare.

2.1.2. Subprograme definite de utilizator – implementare în limbajul Python

2.1.2.1. Noțiuni teoretice

În limbajul Python, subprogramele se numesc **funcții**.

Clasificarea subprogramelor – în funcție de autorul lor

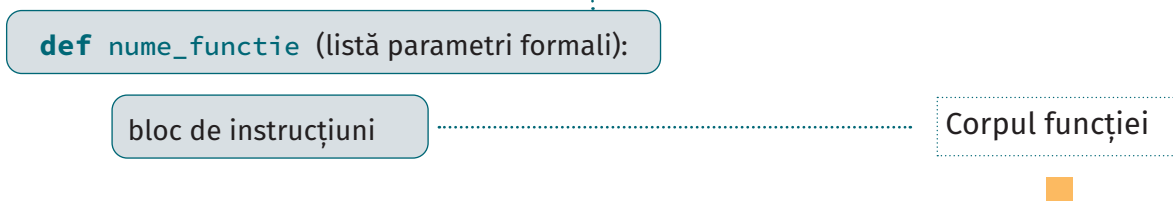
► **Subprograme predefinite (standard)** – sunt subprograme care sunt deja definite în bibliotecile limbajului de programare. Programatorul apelează (utilizează) aceste subprograme.

► **Subprograme definite de utilizator (nstandard)** – sunt subprograme definite și apelate de către programator.

Pentru a lucra cu funcții definite de utilizator, avem nevoie de:

- Definiția funcției
- Apelul funcției

Definiția unei funcții respectă următoarea sintaxă:



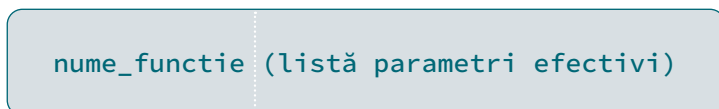
def – cuvânt rezervat, introduce definiția funcției;

nume_funcție – este numele funcției; este stabilit de programator și respectă regula formării unui identificador în limbajul Python;

listă parametri formali – este o listă de parametri prin care subprogramul comunică cu subprogramul apelant. Această listă este opțională, iar în cazul în care lipsește, atunci setul de caractere **()** este obligatoriu.

bloc de instrucțiuni – alcătuit dintr-un set de instrucțiuni care este executat în momentul în care funcția este apelată. Fiecare linie, aparținând blocului de instrucțiuni, păstrează o indentare de aceeași lungime.

Apelul unei funcții respectă următoarea sintaxă:



nume_funcție – este numele funcției stabilit la definiția acesteia;

listă parametri efectivi (sau argumente) – la apel, valorile acestora sunt cunoscute.

La apel, parametrii formali primesc, de la stânga la dreapta, valorile din lista parametrilor efectivi.

Return

O funcție poate returna o valoare, sau mai multe valori, către subprogramul apelant,

utilizând instrucțiunea **return <expresii>**.

Instrucțiunea **return** oprește execuția unei funcții, controlul este transferat subprogramului apelant. Dacă o funcție nu conține nicio instrucțiune **return** sau conține doar **return** fără a indica o expresie, atunci funcția returnează valoarea specială **None** (constantă care indică absența unei valori; este definită în biblioteca interpretorului Python).

> > > **EXEMPLUL 1****Cod Python**

```
#Definiție funcție
def Functie_1():
    print("Prima mea funcție")

#Program principal
Functie_1()
```

Exemplu de rulare

Prima mea funcție

Explicații

Antetul funcției este `def Functie_1():`

Numele funcției este `Functie_1`

Corpul funcției conține o instrucțiune de afișare.

Atenție la indentarea instrucțiunilor ce alcătuiesc corpul funcției!!

Nu există o listă a parametrilor formali, ca atare, nu există o listă a parametrilor efectivi.

Definiția funcției este plasată înaintea subprogramului care conține instrucțiunea de apel.

Apelul funcției `Functie_1()` apare în corpul subprogramului apelant (aici programul **principal**). La întâlnirea instrucțiunii de apel are loc executarea funcției.

> > > **EXEMPLUL 2****Enunțul problemei:**

Scrie în limbajul Python un program care convertește, în grade Celsius (notat cu °C), **trei valori reale citite de la tastatură**, reprezentând grade Fahrenheit (notat cu °F). Rezolvarea se va realiza în două variante: **una cu funcții și alta fără funcții**.

Analiza problemei:

► *Date de intrare* – valorile reale `f1`, `f2`, `f3` (reprezintă valori în °F).

► *Date de ieșire* – trei valori reale (reprezintă conversia datelor de intrare în °C).

Gradul Fahrenheit și gradul Celsius sunt unități de măsură pentru temperatură.

Formule de conversie ale unei valori reale `x`:

Din grade Fahrenheit în grade Celsius	Din grade Celsius în grade Fahrenheit
$x \text{ } ^\circ\text{F} = (x - 32) * 5/9 \text{ } ^\circ\text{C}$	$x \text{ } ^\circ\text{C} = (x * 9/5 + 32) \text{ } ^\circ\text{F}$

**Rezolvare
fără funcții**

```
#Program principal
f1=float(input("Prima valoare in grade F: "))
f2=float(input("A doua valoare in grade F: "))
f3=float(input("A treia valoare in grade F: "))

celsius1=(f1 - 32)*5/9
celsius2=(f2 - 32)*5/9
celsius3=(f3 - 32)*5/9

print("Prima valoare in grade C:",celsius1)
print("A doua valoare in grade C:",celsius2)
print("A treia valoare in grade C:",celsius3)
```



Rezolvare cu funcții

```
def citeste():
    a1=float(input("Prima valoare in grade F: "))
    a2=float(input("A doua valoare in grade F: "))
    a3=float(input("A treia valoare in grade F: "))
    return a1,a2,a3

def Functie_2(fahr):
    celsius=(fahr - 32)*5/9
    return celsius

#Program principal
f1,f2,f3=citeste()
print("Prima valoare in grade C:",Functie_2(f1))
print("A doua valoare in grade C:",Functie_2(f2))
print("A treia valoare in grade C:",Functie_2(f3))
```

Prima valoare in grade F: 95
A doua valoare in grade F: 32
A treia valoare in grade F: 50
Prima valoare in grade C: 35.0
A doua valoare in grade C: 0.0
A treia valoare in grade C: 10.0

Explicații:

Problema enunțată are un grad mic de complexitate, iar rezolvarea cu funcții este o abordare mai degrabă didactică. S-a dorit exemplificarea implementării funcțiilor cu parametri, a funcțiilor care returnează un rezultat și a funcțiilor care returnează mai multe rezultate. *Rezolvarea fără funcții* – s-a aplicat de trei ori aceeași formulă de conversie; aici observăm un dezavantaj: codul este mai greu de actualizat (exemplu: dacă se modifică formula de calcul). *Rezolvarea cu funcții* – au fost definite două funcții

► Funcția `citeste()` – nu are parametri, citește de la tastatură trei valori reale `a1`, `a2` și `a3` și returnează (transmite prin instrucțiunea `return`) programului principal aceste valori, care vor fi memorate, în ordine, în variabilele `f1`, `f2` și, respectiv, `f3`. Variabilele `a1`, `a2` și `a3` sunt proprii funcției `citeste()` și nu pot fi utilizate din exteriorul funcției.

- Funcția `Functie_2(fahr)` aplică formula de conversie pentru parametrul formal `fahr`, codul este scris o singură dată, iar funcția este apelată de trei ori.
- În variabila `celsius` se calculează rezultatul funcției, rezultat care este returnat (întors) către programul principal prin instrucțiunea `return celsius`
- Funcția are un parametru formal (`fahr`), ca atare va avea și un parametru efectiv (exemplu `f1`). **Nu este obligatoriu ca denumirile acestora să fie identice.**
- La întâlnirea apelului `Functie_2(f1)`, se execută următorii pași: parametrul formal `fahr` primește valoarea parametrului efectiv `f1`, apoi are loc executarea propriu-zisă a funcției, cu această valoare, și returnarea rezultatului funcției către programul principal. Acest tip de transmitere a parametrilor se numește **transmitere prin valoare**.

Exemplu numeric:

f1	Funcție_2(fahr)	fahr	celsius	Operație
95	-	-	-	Citire din subprogramul <code>citeste()</code>
95	<code>Funcție_2(95)=?</code>	95	-	Apel <code>Funcție_2(f1)</code>
95	<code>Funcție_2(95)=?</code>	95	35.0	Calculul <code>celsius=35.0</code>
95	35.0	-	-	Întoarcere în programul principal și scrie 35.0

- ▶ Aceiași pași se vor executa și în cazul apelurilor `Funcție_2(f2)` și `Funcție_2(f3)`.
- ▶ Observăm câteva avantaje: codul care aplică formula de conversie este scris o singură dată și utilizat de trei ori. Atunci când executăm (testăm) secvențe mici de cod (subprograme), erorile pot fi depistate mai ușor.

Mecanismul de executare a unui program care conține funcții definite de utilizator:

- ▶ La întâlnirea definiției unei funcții: interpretorul Python traduce codul într-un cod cunoscut de calculator (bytecode), având ca rezultat înregistrarea numelui funcției, fără a executa corpul funcției;
- ▶ La întâlnirea apelului unei funcții: interpretorul Python execută corpul funcției.



IMPORTANT

- » În mod implicit, numărul parametrilor formali este egal cu numărul parametrilor efectivi.
- » Transmiterea parametrilor de tip `int`, `float`, `str` se realizează **prin valoare**: parametrul formal primește o copie a valorii parametrului efectiv astfel încât, orice modificare a parametrului formal nu afectează valoarea parametrului efectiv din subprogramul apelant.

>>> EXEMPLUL 3

Se consideră următoarea definiție:

```
def F(a,b):
    return a+b
```

Instrucțiuni de apel:

<code>x=7</code> <code>y=3</code> <code>print(F(x,y))</code>	<code>x=7.6</code> <code>y=9.4</code> <code>print(F(x,y))</code>	<code>x="car"</code> <code>y="te"</code> <code>print(F(x,y))</code>	<code>x="car"</code> <code>y=10</code> <code>print(F(x,y))</code>
Rezultat: 10, tip <code>int</code>	Rezultat: 17.0, tip <code>float</code>	Rezultat: <i>carte</i> , tip <code>str</code>	Eroare de execuție!
Parametrii efectivi sunt de tip întreg (<code>int</code>)	Parametrii efectivi sunt de tip real (<code>float</code>)	Parametrii efectivi sunt de tip șir de caractere (<code>str</code>).	Operatorul "+" nu se poate aplica pe un <code>str</code> și un <code>int</code>

Explicații: Un parametru efectiv, **neavând tipul declarat**, poate fi de orice tip, atâta timp cât instrucțiunile/operațiile din definiția funcției pot fi executate.



Enunțul problemei: Se citește, de la tastatură, un număr natural. Verifică dacă numărul poate fi un CNP valid, afișând un mesaj corespunzător (condiții verificate: numărul trebuie să conțină 13 cifre, iar luna de naștere să fie validă).

Analiza problemei:

date de intrare: valoarea naturală **CNP**;

date de ieșire: mesaj de validare.

CNP – Cod Numeric Personal, este un cod numeric unic atribuit fiecărei persoane din România; are următoarea formă:

S	A	A	L	L	Z	Z	X	X	X	X	X	X
Secol	An	An	Lună	Lună	Zi	Zi	X	X	X	X	X	X

Proiectare modulară:

Numele funcției	Rolul funcției – indicații de rezolvare
<code>citire_CNP()</code>	Citește un număr natural și returnează valoarea citită. Acesta va fi reținut în variabila <code>cnp</code> declarată în programul principal.
<code>validare_lungime(c)</code>	Funcția primește, prin parametrul <code>c</code> , un număr natural primit la apel în subprogramul <code>validare_CNP</code> . Funcția calculează numărul de cifre ale valorii memorate în parametrul <code>c</code> și returnează 1 sau 0 după cum lungimea acestuia este egală cu 13 sau nu.
<code>validare_luna(c)</code>	Funcția primește, prin parametrul <code>c</code> , un număr natural, primit la apel în subprogramul <code>validare_CNP</code> . Funcția returnează 1 dacă luna este validă, altfel, funcția returnează 0.
<code>validare_CNP(c)</code>	Funcția primește, prin parametrul <code>c</code> , numărul natural citit, prin apel în programul principal. Integrează toate validările și afișează rezultatul validării. Folosește o variabilă declarată în această funcție, inițial cu valoarea 1; valoarea acesteia se modifică la valoarea 0 dacă: – lungimea <code>cnp</code> este diferită de 13 sau – lungimea <code>cnp</code> este egală cu 13, dar luna nu este validă.

În exemplul de mai jos: CNP - invalid (luna nu este validă)

<code>validare_lungime(c)</code>												
13	12	11	10	9	8	7	6	5	4	3	2	1
1	7	8	1	5	2	4	2	4	0	0	5	2
<code>validare_luna(c)</code>												

> > >

Program Python

```

#Definiții funcții
def citire_CNP():
    return int(input("Introduceți CNP: "))

def validare_lungime(c):
    nr_cifre=0
    while(c>0):
        nr_cifre+=1
        c=c//10
    if(nr_cifre==13):
        return 1
    else:
        return 0

def validare_luna(c):
    ll=(c//10**8)%100
    if ll<1 or ll>12:
        return 0
    return 1

def validare_CNP(c):
    lungime=validare_lungime(c)
    luna=validare_luna(c)
    validare=1
    if lungime==0:
        print("CNP nu are 13 cifre")
        validare=0
    if lungime==1 and luna==0:
        print("CNP nu are luna valida")
        validare=0
    if validare == 1:
        print("CNP valid")
    else:
        print("CNP invalid")

#Program principal
cnp = citire_CNP()
validare_CNP(cnp)

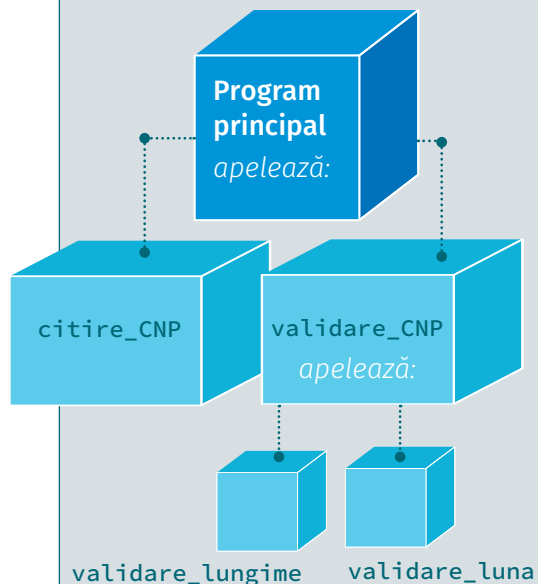
```

Exemplu de rulare

Introduceți CNP: 2781230500342
CNP valid

Introduceți CNP: 1801024780
CNP nu are 13 cifre
CNP invalid

Reprezentarea grafică de mai jos evidențiază structura logică a aplicației, dar și dependența și relația de subordonare între subprograme.



Activitate în echipă

Integrează alte două funcții programului prezentat anterior:
a) o funcție pentru validarea primei cifre (cuprinsă între 1 și 8);
b) o funcție pentru validarea zilei de naștere (în funcție de luna nașterii).



2.1.2.2. Tipuri de parametri efectivi (argumente) ce pot fi transmiși către funcții

Parametri poziționali – lista parametrilor efectivi coincide cu lista parametrilor formali ca număr și ordine. <i>Exemplu:</i>	Parametri cuvinte-cheie (keyword) – asocierea <i>cuvinte-cheie=valoare</i> permite schimbarea ordinii parametrilor la apel. <i>Exemplu:</i>
<pre># parametri poziționali def f(a,b,c): print(a,b,c) f(1,2,3) # 1 2 3 f(8,7,6) # 8 7 6</pre>	<pre># parametri keyword def f(a,b,c): print(a,b,c) f(a=1,b=2,c=3) # 1 2 3 f(b=8,c=7,a=6) # 6 8 7</pre>
Valori implicite ale parametrilor formali – se utilizează valorile implicite pentru argumentele netransmise. <i>Exemplu:</i>	Exemplu – combinație parametri poziționali și parametri keyword
<pre># valori implicite pentru a,b,c def f(a=1,b=2,c=3): print(a,b,c) f() # 1 2 3 f(7) # 7 2 3 f(2,4) # 2 4 3</pre>	<pre>def f(a,b,c): print(a,b,c) f(1,c=3, b=2) # 1 2 3</pre>



IMPORTANT

Ca urmare a complexității modului de transmisie a argumentelor către funcții, trebuie respectată o anumită **ordine a parametrilor** atât la apel, cât și la definirea funcției:

În antet	La apel
1) parametri poziționali 2) parametri cu valori implicite	1) parametri poziționali 2) parametri keyword

AVANTAJELE utilizării subprogramelor:

- ▶ **Reutilizarea codului** – aspect extrem de important în eficientizarea dezvoltării aplicațiilor, codul este scris o singură dată, dar subprogramul poate fi apelat de câte ori este nevoie.
- ▶ **Depanarea și actualizarea aplicației se fac mai ușor** – este mult mai ușor să se efectueze modificări la nivelul unui subprogram, decât la nivelul întregii aplicații.
- ▶ **Favorizează lucrul în echipă** - membrii echipei pot lucra simultan la sarcini diferite.
- ▶ **Crearea de aplicații lizibile** - codul devine mai ușor de urmărit și de înțeles.
- ▶ Crește **portabilitatea programelor** – subprogramele realizate pot fi preluate de alte programe/aplicații care necesită rezolvarea de sarcini similare.

Câteva DEZAVANTAJE ale utilizării subprogramelor:

- ▶ Necesită o proiectare mai atentă.
- ▶ Mai multe funcții pot părea greu de urmărit pentru începători.



APLICAȚII

Implementează, în limbajul Python, din **Secțiunea 2.1.1:**

- algoritmul prezentat în cadrul **Studiului de caz 2**.
- cei doi algoritmi, propuși spre rezolvare modulară, de la finalul secțiunii 2.1.1.

2.1.3. Variabile locale, variabile globale

2.1.3.1. Noțiuni teoretice

În cadrul organizării modulare a unui program, este foarte importantă zona din program în care o variabilă *acționează*, este *recunoscută și poate fi folosită*. Această zonă (porțiune) de program poartă numele de **domeniu de vizibilitate (scop)**.

Din acest punct de vedere, în limbajul Python, variabilele se pot clasifica astfel:

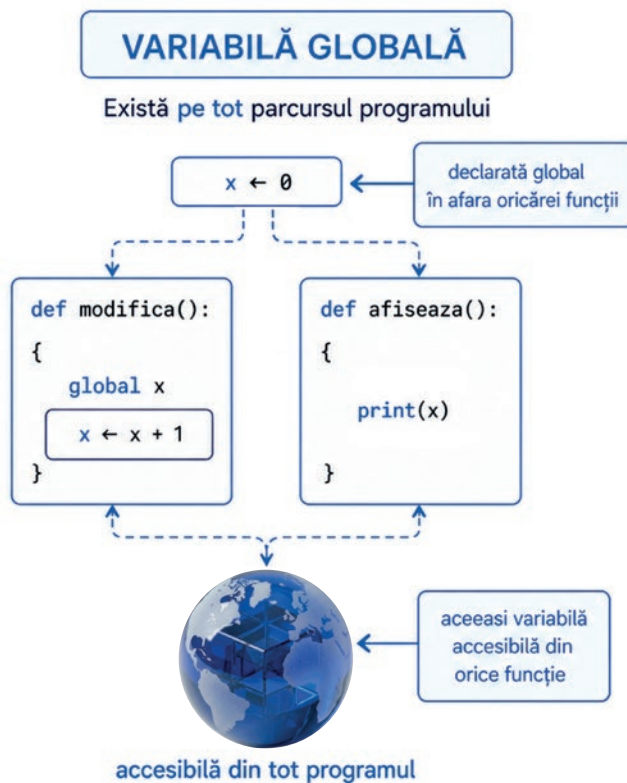
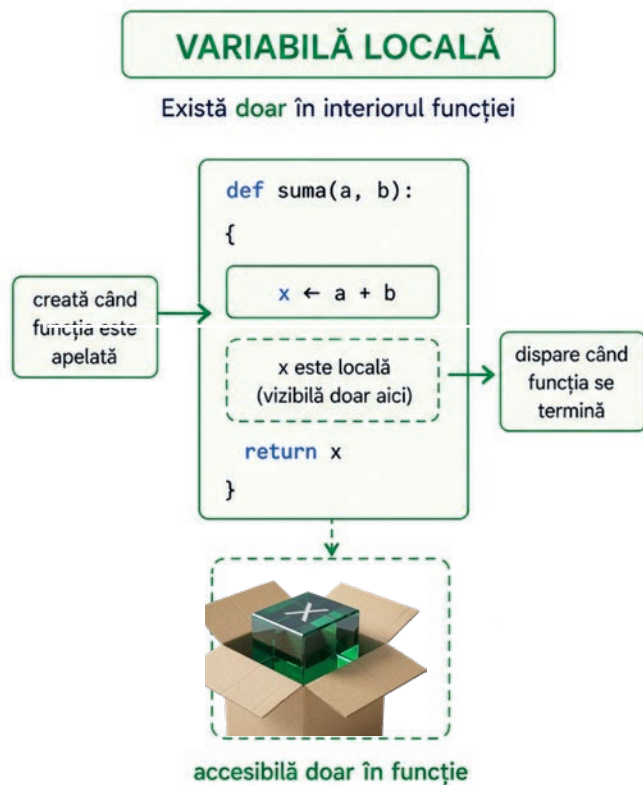
a) VARIABLE LOCALE – așa cum am văzut într-o lecție anterioară, sunt create în corpul unei funcții și nu pot fi utilizate din exteriorul funcției. Parametrii unei funcții sunt considerați variabile locale.

Segmentul de stivă este o zonă specială a memoriei interne unde sunt memorate variabilele locale, precum și adresa de revenire (adresa instrucțiunii ce trebuie executată după întoarcerea din subprogram). La finalul execuției subprogramului, acest spațiu este eliberat (variabilele locale sunt distruse).

b) VARIABLELE GLOBALE – sunt create în exteriorul definiției oricărei funcții, pot fi utilizate de programul principal, dar și de toate funcțiile definite în program.

Există, însă, un caz în care o variabilă globală poate fi creată **în cadrul definiției unei funcții**: dacă variabila este declarată cu instrucțiunea **global**.

Variabilele globale se memorează într-o zonă specială a memoriei interne, denumită **segment de date** și pot fi utilizate pe parcursul execuției întregului program.



2.1.3.2. Reguli de acces

- O funcție are acces la toate variabilele globale (pentru modificare se utilizează instrucțiunea **global**; se păstrează orice modificare efectuată asupra variabilelor globale).
- O funcție are acces la toate variabilele locale funcției exterioare (pentru *modificare* se utilizează instrucțiunea **nonlocal**).
- Dacă într-o funcție se declară o variabilă cu *același nume* ca și cea globală, atunci în acea funcție va funcționa variabila declarată local (pentru acces la variabila globală se poate defini variabila împreună cu instrucțiunea **global**).
- Dacă într-o funcție se declară o variabilă cu *același nume* ca și cea declarată în funcția exterioară, atunci în acea funcție va funcționa variabila declarată local (pentru acces la variabila locală declarată în funcția exterioară se poate defini variabila împreună cu instrucțiunea **nonlocal**).



IMPORTANT

Spunem că **două funcții sunt imbricate (încapsulate)** atunci când definiția unei funcții (o *denumim funcție interioară*) este inclusă în corpul definiției altei funcții (o *denumim funcție exterioară*).



>>> EXEMPLE

```
x=1                #x var. globală
def ex1(a):        #a var. locală
    a=a+5
    print(a)
x=x+2              #var. globală
ex1(x)             #afișează 8
print(x)           #afișează 3
```

#a)

```
x=1                #x var. globală
def ex2(a):        #a var. locală
    x=a+2
    print(x)
ex2(x)             #afișează 3
print(x)           #afișează 1
```

#b)

```
x=1                #x var. globală
def ex3():
    global x        #x var. globală
    x=x+1
    print(x)
ex3()              #afișează 2
print(x)           #afișează 2
```

#c)

```
x=1                #x var. globală
def ex3_1():
    x=x+1           #EROARE la execuție
    print(x)
ex3_1()
print(x)
```

#d)

```
x=1                #x var. globală
#Funcții imbricate:
def ex4():
    x=2             #x var. locală
    def ex5():
        x=3         #x var. locală
        print(x)
    ex5()           #afișează 3
    print(x)        #afișează 2
ex4()
print(x)           #afișează 1
```

#e)

```
x=1                #x var. globală
#Funcții imbricate:
def ex4_1():
    x=2             #x var. locală
    def ex5_1():
        nonlocal x  #x var. locală in ex4_1()
        x=3
        print(x)
    ex5_1()         #afișează 3
    print(x)        #afișează 3
ex4_1()
print(x)           #afișează 1
```

#f)

ACTIVITATE ÎN ECHIPĂ

Enunțul problemei:

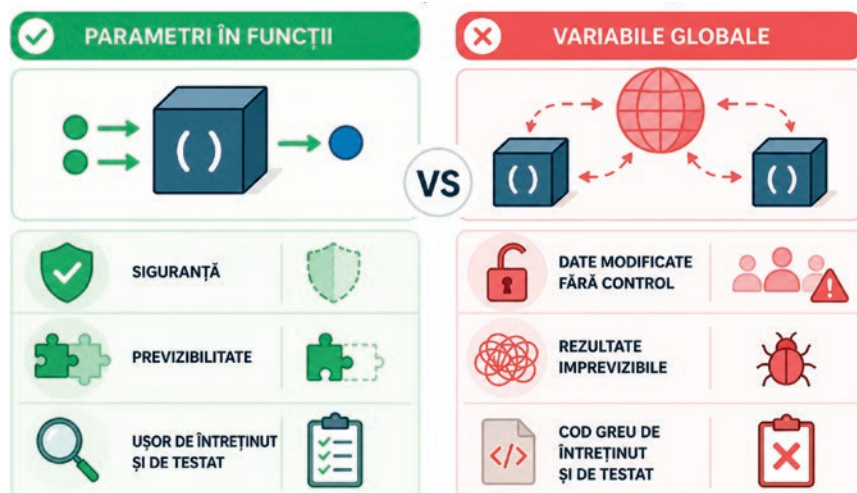
Rescrie codul problemei rezolvate din Secțiunea 2.1.2. (validare CNP), utilizând funcții care prelucrează variabile globale.

Indicații de rezolvare:

Funcțiile implementate sunt fără parametri și pot accesa datele globale (`cnp`-declarată în programul principal), fără a folosi instrucțiunea `global`, atâta timp cât valorile respective rămân neschimbate. Evitați modificarea valorii variabilei globale `cnp`.

Comunicarea dintre un subprogram și subprogramul său apelant se poate face prin intermediul **parametrilor** și/sau al **variabilelor globale**. Iată avantajele utilizării parametrilor în locul variabilelor globale:

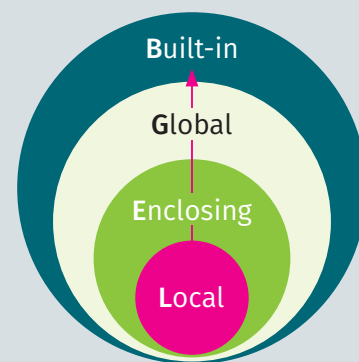
Parametri în funcții vs variabile globale



IMPORTANT

În limbajul Python, vizibilitatea variabilelor respectă ierarhia **LEGB** (din limba engleză: *local, enclosing, global, built-in*), iar căutarea unui nume de variabilă se va realiza în următoarea ordine:

1. **Local** – definite local (în definiția unei funcții);
2. **Enclosing** – definite în funcții imbricate (pornind de la cea mai interioară, spre cele exterioare);
3. **Global** – definite global (create în programul principal sau create cu instrucțiunea `global` în funcții);
4. **Built-in** – definite în biblioteca standard Python.



APLICAȚII

Se consideră secvențele de program alăturat. Pentru fiecare dintre ele:

- a) precizează lista variabilelor utilizate, grupate după domeniul de vizibilitate;
- b) explică rezultatul execuției.

```
def exterior():
    y=2
    def interior():
        print("x=",x)
        print("y=", y)
    interior()
    if(x==1):
        print("DA",x)
    else:
        print("NU")
x=1
exterior()
```

```
def exterior():
    y=2
    def interior():
        x=y+2
        print("x=",x)
    interior()
    global x
    x=y
    print("x=",x)
x=1
exterior()
print(x)
```



2.1.4. Funcții predefinite Python

Documentația Bibliotecii Standard oferă o prezentare detaliată a diferitelor componente ale interpretorului Python, incluzând tipurile de date, funcțiile, modulele și alte elemente recunoscute de acesta.

Funcțiile predefinite, pe care le regăsim în biblioteca Python, le putem clasifica în:

1 Funcții încorporate (*built-in*) în biblioteca standard

Ai utilizat (apelat) deja unele funcții de acest tip:
`print()`, `int()`, `range()`.

2 Funcții definite în *module încorporate (built-in)* în biblioteca standard (exemplu modulul `math`).

În limbajul Python, *un modul* este o colecție de instrucțiuni și definiții de funcții, organizate într-un fișier. Pentru a accesa o funcție, definită dintr-un modul Python, avem nevoie să utilizăm, în programul nostru, directiva `import` urmată de numele modulului, iar apelul unei funcții va fi de forma `nume_modul.nume_funcție()`.

2.1.4.1. Funcții predefinite utile în *prelucrări matematice*

Nume funcție	Tip argument	Tip rezultat	Rezultatul funcției. <i>Funcție built-in</i> sau funcție în modulul <i>math</i> .	Exemple
<code>sqrt(arg)</code>	<code>int/float</code> (nenegative)	<code>float</code>	Rădăcină pătrată din <i>arg</i> . <i>Modul math</i> .	<code>math.sqrt(16) → 4.0</code> <code>math.sqrt(2.25) → 1.5</code>
<code>abs(arg)</code>	<code>int/float</code>	<code>int/float</code>	Valoarea absolută a <i>arg</i> . <i>Funcție built-in</i> .	<code>abs(-9.5) → 9.5</code> <code>abs(9) → 9</code>
<code>fabs(arg)</code>	<code>float</code>	<code>float</code>	Valoarea absolută a <i>arg</i> . <i>Modul math</i> .	<code>math.fabs(-2.5) → 2.5</code> <code>math.fabs(2) → 2.0</code>
<code>round(arg1 [, arg2])</code>	<code>arg1: int/float</code> <code>arg2: int</code> (poate lipsi)	<code>int/float</code>	Numărul <i>arg1</i> rotunjit la <i>arg2</i> zecimale. Dacă <i>arg2</i> lipsește, se obține cel mai apropiat întreg de <i>arg1</i> . <i>Funcție built-in</i> .	<code>round(2.67, 1) → 2.7</code> <code>round(-2.19, 1) → -2.2</code> Caz particular - la valori exact la jumătate, rotunjește către cel mai apropiat întreg par: <code>round(2.5) → 2</code> <code>round(1.5) → 2</code>
<code>floor(arg)</code>	<code>float</code>	<code>int</code>	Cel mai mare număr întreg mai mic sau egal cu <i>arg</i> . <i>Modul math</i> .	<code>math.floor(2.9) → 2</code> <code>math.floor(-2.4) → -3</code>
<code>ceil(arg)</code>	<code>float</code>	<code>int</code>	Cel mai mic număr întreg mai mare sau egal cu <i>arg</i> . <i>Modul math</i> .	<code>math.ceil(2.9) → 3</code> <code>math.ceil(-2.4) → -2</code>
<code>trunc(arg)</code>	<code>float</code>	<code>int</code>	Partea zecimală se elimină, lăsând partea întreagă. <i>Modul math</i> .	<code>math.trunc(-2.8) → -2</code> <code>math.trunc(2.15) → 2</code>

2.1.4.2. Funcții predefinite *built-in de conversie explicită*, pe care le-ai studiat deja în Capitolul 1:

`int()`, `float()` și `str()`.

- *Conversia explicită*: utilizează funcții predefinite de conversie. Este posibilă numai dacă se respectă formatul tipului de dată țintă.
- *Conversia implicită*: convertește automat un tip de date în altul, fără utilizarea funcțiilor predefinite de conversie (exemplu: de la `int` la `float`, de la `bool` la `int`). Acest tip de conversie se realizează de obicei către tipul de date mai larg.

#pentru a concatena un numar la un str #conversie explicită: din int în str <code>a = str(35)+"xy" #a devine 35xy</code>	#Conversie implicită <code>a = 3 #a de tip int</code> <code>b = 3.14 #b de tip float</code> <code>c = a + b #c de tip float</code>
#pentru a aduna un str la un numar #conversie explicită: din str în float <code>a = float('12.5')+1.5</code>	
#Eroare la conversie explicită <code>a = int('3.14')</code>	

2.1.4.3. Funcții predefinite *built-in* utile în prelucrarea secvențelor de date

Nume funcție	Tip argumente	Tip rezultat	Rezultatul funcției	Exemple
<code>min(arg1, arg2, ...)</code>	<code>int/float</code>	<code>int/float</code>	Cea mai mică valoare dintre 2 sau mai multe argumente.	<code>min(6,7,2.1) → 2.1</code>
<code>max(arg1, arg2, ...)</code>	<code>int/float</code>	<code>int/float</code>	Cea mai mare valoare dintre 2 sau mai multe argumente.	<code>max(6,7,2.1) → 7</code>

>>> EXEMPLU

Enunțul problemei:

Scrie un program, în limbajul Python, care să calculeze distanța dintre două puncte în planul axelor carteziane: $A(x_a, y_a)$, $B(x_b, y_b)$. Coordonatele reale ale celor două puncte se citesc de la tastatură.

Cod Python

```
#instrucțiune de import
import math
xa=float(input("xa = "))
ya=float(input("ya = "))
xb=float(input("xb = "))
yb=float(input("yb = "))
d=math.sqrt((xa-xb)**2+(ya-yb)**2)
print("Distanța:", d)
```

Exemplu de rulare

```
xa = 3
ya = 4
xb = 0
yb = 0
Distanța: 5.0
```



Avantajele utilizării subprogramelor predefinite: crește lizibilitatea codului, reduce dimensiunea codului, funcțiile predefinite sunt optimizate și testate.



1. Rulează, în limbajul Python, secvențele de program de mai jos și justifică rezultatul:

```
# punctul a)
x = -2.5
y = 1.5
print(round(abs(x)+float(int(y))))
```

```
# punctul c)
import math
print(math.trunc(2.59))
print(int(2.59),math.floor(2.59))
print(round(2.59,1),math.ceil(2.59))
```

```
# punctul b)
a=str(1)+str(2)
b=int("1")+int("2")
print("a=",a)
print("b=",b)
```

```
# punctul d)
a=int(input("a="))
b=int(input("b="))
x=max(a-b,b-a)==abs(a-b)
print(x)
```

2. Se consideră codul Python de mai jos:

```
def cifre_distincte():
    y=n
    while y > 0:
        c = -----
        x = y // 10
        while x > 0:
            if x % 10 == c:
                return ----
            x = -----
        y = -----
    return ----

# program principal
n = int(input("n = "))
if -----:
    -----
else:
    -----
```

a) Completează spațiile libere astfel încât programul să afișeze mesajul **Adevărat**, dacă numărul natural n , citit de la tastatură, are toate cifrele distincte. Dacă nu îndeplinește această condiție, să afișeze mesajul **Fals**.

b) Modifică programul obținut la punctul a, astfel încât programul să afișeze suma tuturor numerelor naturale $\leq n$ care au toate cifrele distincte. Funcția `cifre_distincte` va primi, prin parametrul d , un număr natural $\leq n$.

3. Citește scorurile (valori naturale) obținute de trei elevi la un campionat de șah.

Să se determine scorul maxim și scorul minim (cu și fără aplicarea funcțiilor `max()` și `min()`).

4. Se consideră trei puncte în planul axelor carteziane: $A(x_a, y_a)$, $B(x_b, y_b)$ și $C(x_c, y_c)$. Verifică dacă cele trei puncte formează un triunghi, utilizând două funcții Python: o funcție care citește coordonatele reale ale unui punct în plan și returnează datele citite și o funcție care returnează distanța dintre două puncte.

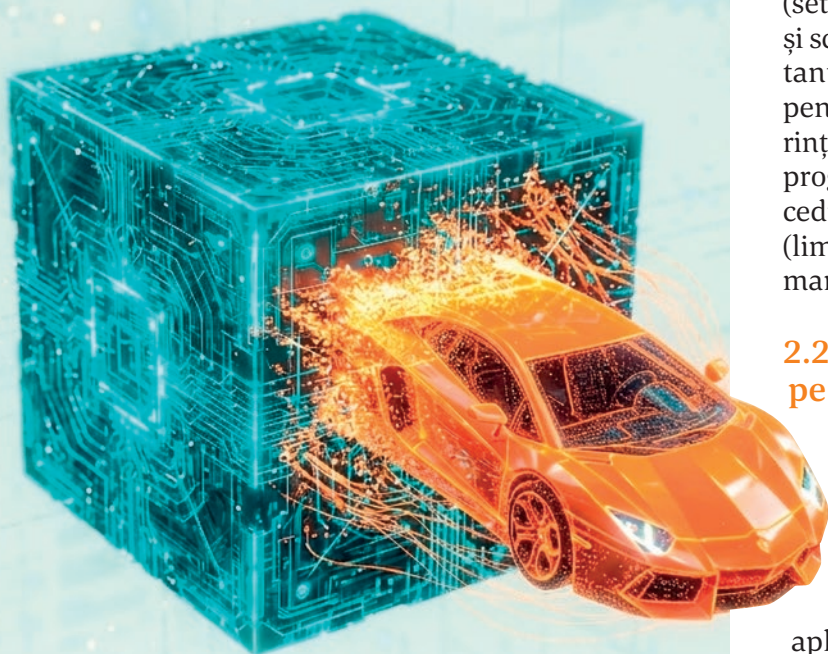
5. Scrie un program în Python care citește de la tastatură trei note, numere naturale.

Definește două funcții: `f1(a,b,c)` – returnează media aritmetică a celor trei note, cu două zecimale, fără rotunjire, și `f2(a,b,c)` – returnează media aritmetică a celor trei note, cu rotunjire la cel mai apropiat întreg.

Apelează și testează funcțiile în programul principal, pentru seturile de note (9,9,8), (7,7,8).



2.2. Introducere în Programarea Orientată pe Obiecte



2.2.1. Motivația utilizării unui limbaj de programare orientat pe obiecte precum Python

În dezvoltarea software modernă, alegerea limbajului de programare și a paradigmei de programare (set de reguli și principii după care este structurat și scris un program de calculator) este foarte importantă pentru eficiența procesului de dezvoltare, pentru corectitudinea programelor și pentru ușurința întreținerii codului. Printre acestea se numără programarea în limbaj *masina*, programarea procedurală, *programarea în limbaje multiparadigmă* (limbaje care suportă mai multe stiluri de programare), precum Python.

2.2.2. Ce este Programarea Orientată pe Obiecte?

2.2.2.1. Introducere

Programarea orientată pe obiecte a apărut din necesitatea de a dezvolta programe mai clare, mai bine organizate și mai ușor de întreținut pe termen lung, capabile să gestioneze aplicații complexe, prin modelarea elementelor din lumea reală. Noutatea adusă de acest concept constă în organizarea problemei sub forma unor **obiecte** care înglobează atât **caracteristici**, cât și **comportamente**.



ȘTIAI CĂ...?

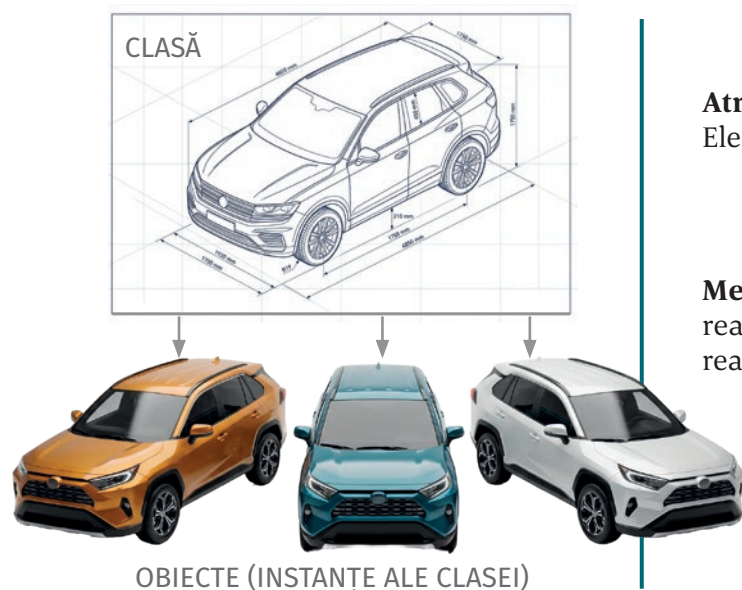
Primul nivel istoric al programării este reprezentat de programarea în limbaj de asamblare. Pentru a simplifica procesul de dezvoltare, au apărut limbajele de nivel înalt și programarea procedurală. În acest model, programul este organizat sub forma unor funcții care rezolvă anumite sarcini.

2.2.2.2. Tranziția de la programarea procedurală la programarea orientată pe obiecte

Într-un program procedural care gestionează informații despre elevi, am putea avea variabile separate pentru nume, vârstă și note, iar mai multe funcții ar modifica aceste variabile. Din acest motiv, programarea procedurală poate deveni inefficientă în ceea ce privește corectitudinea și organizarea codului. Pentru a rezolva aceste probleme, a apărut **programarea orientată pe obiecte (POO)**. În acest model, programul este organizat în jurul unor obiecte care combină datele și funcțiile ce operează asupra acestor date. Conceptul de bază este **clasa**, care reprezintă un model pentru **obiecte**. Obiectele create dintr-o clasă conțin atât atribute (date), cât și metode (funcții).

În prezent, multe limbaje moderne sunt considerate limbaje multiparadigmă. Aceasta înseamnă că permit utilizarea mai multor stiluri de programare: procedural, orientat pe obiecte sau chiar funcțional. Python este un exemplu foarte bun de astfel de limbaj.

Datorită acestei flexibilități, limbajele multiparadigmă sunt în prezent cele mai eficiente din toate punctele de vedere. Ele permit programatorului să combine avantajele mai multor paradigme și să evite limitările fiecăreia. În plus, Python are o sintaxă simplă și clară, ceea ce îl face ușor de învățat pentru începători, dar suficient de puternic pentru dezvoltarea unor aplicații complexe, inclusiv în domenii precum inteligența artificială, analiza datelor sau dezvoltarea web.



2.2.2.3. Noțiuni de bază în POO

Clasa – este un **șablon** sau un model după care se creează obiecte. Ea descrie ce **atribute** (date, caracteristici) și ce **metode** (acțiuni, comportamente) vor avea obiectele.

Exemplul 1: Clasa **Masina** descrie toate mașinile, nu o mașină anume;

Exemplul 2: Clasa **Elev** descrie toți elevii, la modul general;

Obiectul – este o **instanță a unei clase**, adică o realizare concretă a șablonului. Obiectele sunt cele care există efectiv în program.

Exemplul 1: O mașină verde, marca Dacia, carburant Diesel – este un obiect al clasei **Masina**.

Exemplul 2: Un elev din clasa a IX-a, numit Andrei, în vârstă de 15 ani – este un obiect al clasei **Elev**.

Atributele – sunt **caracteristicile** unui obiect. Ele rețin informațiile utile ale obiectelor.

Exemplu: pentru **Masina**: **culoare**, **carburant**, **marca** etc; pentru **Elev**: **clasa**, **nume**, **varsta**

Metodele – **acțiunile** pe care un obiect le poate realiza. Sunt similare cu funcțiile din programarea procedurală.

Exemplul 1: (pentru clasa **Masina**) **porneste()**, **acelereaza()**, **opreste()** etc.

Exemplul 2: (pentru clasa **Elev**) **mergeLaScoala()**, **invata()**, **participaLaOre()** etc.

2.2.2.4. Modelarea unui exemplu din viața reală

În diagrama următoare, prezentăm clasa **Mașină**:

Mașină	numele clasei	Pentru a modela clasa Mașină , trebuie să răspundem la următoarele puncte: 1. Identificarea numelui clasei (Mașină). 2. Identificarea atributelor clasei – „Ce informații sunt necesare pentru a o descrie?” (firmă , model , an). 3. Identificarea metodelor clasei - „Ce acțiuni trebuie să poată realiza această clasă?” (pornire() , oprire() , informatii()).
firmă	atribute	
model		
an		
pornire()	metode	
oprire()		
informatii()		



APLICAȚII



1. Modelează clasa „Elev”, completând următoarele informații:

- a) Numele clasei: _____
- b) Atribute (minim 3): _____, _____, _____
- c) Metode (minim 3): _____(), _____(), _____()

Creează două obiecte de tip „Elev” (două instanțe), folosind valori concrete pentru fiecare atribut în parte:

- a) `elev1`: atribut 1 = ___, atribut 2 = ___, atribut 3 = ___
- b) `elev2`: atribut 1 = ___, atribut 2 = ___, atribut 3 = ___

2. Se consideră următorul scenariu:

Într-o școală se organizează o **bibliotecă** pentru elevi. Biblioteca are mai multe **cărți**, iar fiecare carte are un titlu, un autor și un cod unic. Elevii pot veni la bibliotecă să împrumute cărți pentru o perioadă de timp. Un **elev** are nume, clasă și vârstă. Când un elev împrumută o carte, biblioteca trebuie să înregistreze împrumutul: cine a împrumutat cartea și data până la care trebuie returnată. Dacă o carte este deja împrumutată, elevul nu o poate împrumuta din nou până nu este returnată. Bibliotecarul poate verifica dacă o carte este disponibilă, poate împrumuta o carte unui elev și poate primi o carte înapoi (returnare). De asemenea, biblioteca poate afișa lista de cărți disponibile.

Pornind de la scenariul de mai sus:

- a. Identifică **minimum 3 clase** care apar în scenariul dat.
- b. Pentru fiecare clasă, identifică: **minimum 3 atribute** (caracteristici/date) și **minimum 3 metode** (acțiuni/comportamente)
- c. Prezintă colegilor soluția propusă.



2.2.3. Programarea procedurală vs. Programarea Orientată pe Obiecte (POO)

2.2.3.1. Programarea procedurală

Idee principală:
programul este construit ca o succesiune de pași, organizați în funcții sau proceduri.

Caracteristici	Folosești variabile (date) și instrucțiuni: <code>if</code> , <code>for</code> , <code>while</code> , <code>calcule</code> , <code>print</code> . Programul este o succesiune de comenzi executate în ordine. Datele și funcțiile sunt, de obicei, separate (sau lucrezi direct cu variabile).
Avantaje	Simplu de înțeles la început. Bun pentru programe mici, exerciții scurte.
Dezavantaje	Când programul crește, devine mai greu de organizat. Dacă ai multe „entități” (de exemplu: mulți elevi), ajungi să repeți cod sau să gestionezi multe variabile.

2.2.3.2. Programarea orientată pe obiecte (POO)

Idee principală:
programul este organizat în jurul unor obiecte care au atribute/date (nume, note) și metode/acțiuni/funcții (calculează media, afișează rezultatul)

Caracteristici	Datele și funcțiile sunt grupate în același loc (în clasă). Codul devine mai ușor de extins (adaugi ușor noi metode: absențe, teză).
Avantaje	Organizare mai bună pentru programe mai mari. Ușor de lucrat cu multe entități similare (elevi, produse, conturi). Reutilizare: aceeași clasă poate fi folosită în mai multe programe.
Dezavantaje	La început pare mai „mult de scris”. Trebuie înțelese clasele/obiectele.

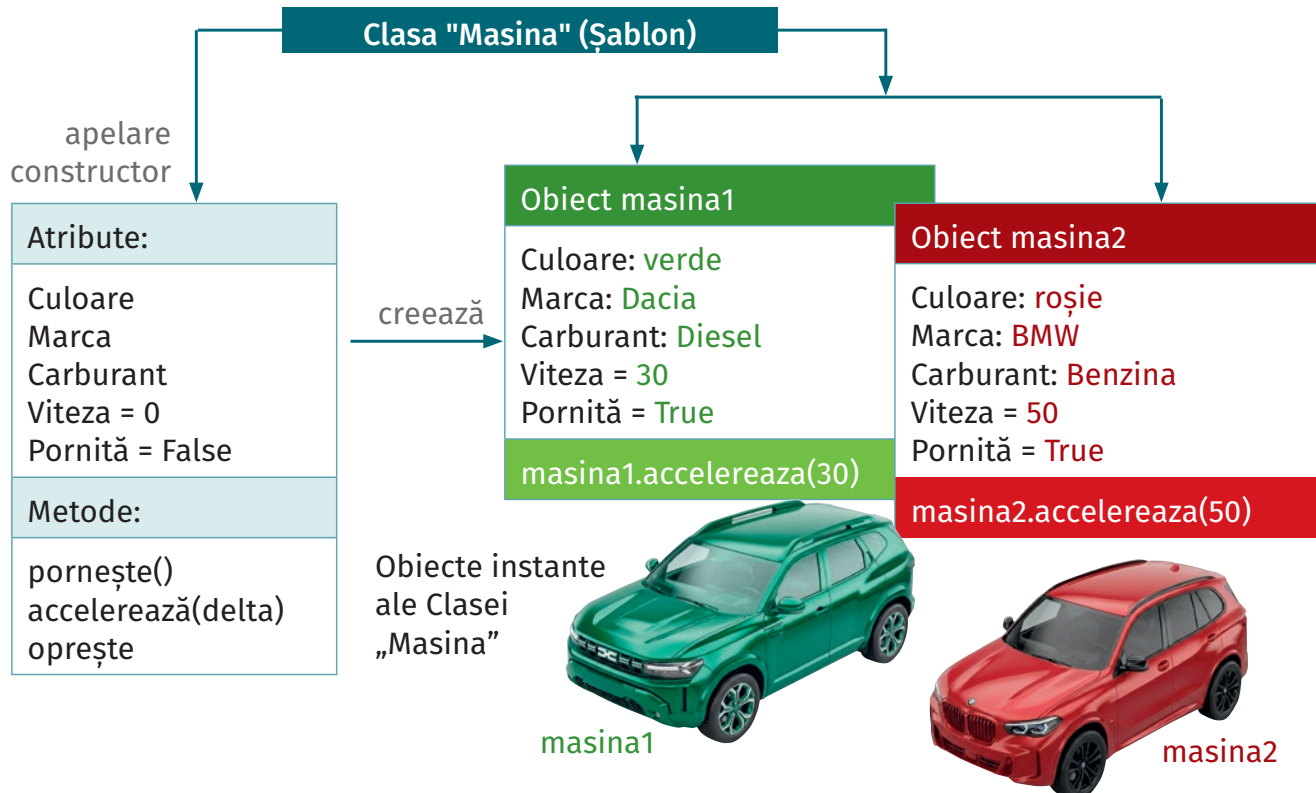
2.2.3.3. Implementarea exemplurilor pentru ambele paradigme de programare

Se dă următorul scenariu. Există două mașini:

- **mașina1:** culoare verde, marca Dacia, carburant Diesel;
- **mașina2:** culoare roșie, marca BMW, carburant Benzină.

Acestea au următoarele acțiuni:

- **pornește mașina1** și o accelerează cu 30 km/h;
- **pornește mașina2** și o accelerează cu 50 km/h;
- **oprește mașina1** (viteza devine 0).



În continuare vom implementa acest scenariu, pentru fiecare paradigmă în parte, folosind limbajul Python.



Cod Python – Implementarea procedurală

```
# Date pentru masina1
culoare1 = "verde"
marca1 = "Dacia"
carburant1 = "Diesel"
viteza1 = 0
pornita1 = False

def porneste_masina1():
    global pornita1
    pornita1 = True
    print(f"{marca1} a pornit.")

def accelereaza_masina1(delta):
    global viteza1
    if not pornita1:
        print("Nu poti accelera: masina este oprita.")
        return
    viteza1 += delta
    print(f"{marca1} accelereaza la {viteza1} km/h.")

def opreste_masina1():
    global viteza1, pornita1
    pornita1 = False
    viteza1 = 0
    print(f"{marca1} s-a oprit.")

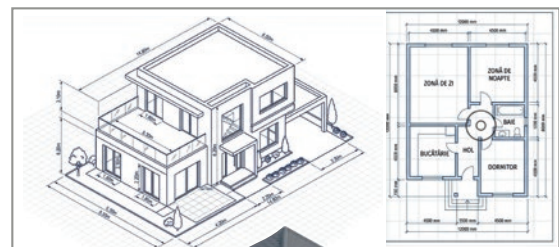
# masina1
porneste_masina1()
accelereaza_masina1(30)
opreste_masina1()

# Similar trebuie să implementăm pentru masina2.
# Această abordare nu este scalabilă (inițializarea unui volum mare de date),
# întrucât avem nevoie de metode diferite pentru fiecare mașină în parte.
```



APLICAȚII

1. Descrie pe scurt cele două paradigme de programare (procedural și POO)
2. Cum sunt organizate datele și acțiunile pentru fiecare paradigmă de programare?
 - a) Procedural: date = _____,
acțiuni = _____
 - b) POO: date = _____,
acțiuni = _____
3. Ce probleme pot apărea în programarea procedurală dacă, pentru stocarea datelor unor elevi, folosim liste independente (pentru note, nume, medii etc)? De ce?





```

class Masina:
    # Constructorul: se apelează când creăm un obiect nou
    def __init__(self, culoare, marca, carburant):
        # Atribute (caracteristici) ale obiectului
        self.culoare = culoare
        self.marca = marca
        self.carburant = carburant
        # Stari initiale
        self.viteza = 0
        self.pornita = False
    def porneste(self):
        # Metoda: pornim mașina curentă (self = obiectul curent)
        self.pornita = True
        print(f"{self.marca} a pornit.")

    def accelereaza(self, delta):
        # Metoda: accelerează mașina curentă
        if not self.pornita:
            print("Nu poti accelera: masina este oprita.")
            return

        self.viteza += delta
        print(f"{self.marca} accelereaza la {self.viteza} km/h.")

    def opreste(self):
        # Metoda: opreste masina curenta
        self.pornita = False
        self.viteza = 0
        print(f"{self.marca} s-a oprit.")

# ----- creare obiecte -----
# masina1 și masina2 sunt obiecte (instanțe) ale clasei Masina
masina1 = Masina("verde", "Dacia", "Diesel")
masina2 = Masina("rosie", "BMW", "Benzina")
masina1.porneste() # Dacia a pornit
masina1.accelereaza(30) # Dacia accelerează la 30 km/h
masina2.porneste() # BMW a pornit
masina2.accelereaza(50) # BMW accelerează la 50 km/h
masina1.opreste() # Dacia s-a oprit

```



IMPORTANT

În programarea orientată pe obiecte (POO) în Python, **self** se referă la instanța curentă a clasei, adică obiectul concret asupra căruia lucrezi. Rolurile lui **self**:

- ▶ Acces la atributele obiectului: **self.atribut**;
- ▶ Diferențiază variabilele locale de cele ale obiectului: **culoare** = parametru primit, **self.culoare** = atribut al obiectului.
- ▶ Permite apelarea altor metode din aceeași clasă.

CLASĂ

Definește șablonul (atribute și metode)



INSTANȚIERE

Se creează o instanță din clasă



OBIECT

Instanța devine un obiect utilizabil





2.2.4. Clase predefinite în Python

Paradigma de programare pe care este construit limbajul Python este aceea a Programării Orientate pe Obiecte. Limbajul de programare Python conține multe clase utile, numite clase predefinite, prezentate în cele ce urmează.



IMPORTANT

În Python, `int`, `float`, `str` sunt imutabile, ceea ce înseamnă că valoarea unui obiect din unul din aceste tipuri nu se poate modifica după ce acesta a fost creat. De exemplu, dacă avem `x = 10` și apoi scriem `x = x + 1`, nu se modifică obiectul `10`, ci se creează un nou obiect `11`, iar variabila `x` va face referire la acesta.

În limbajul Python, bibliotecile și modulele sunt colecții de cod deja scris, pe care programatorii le pot folosi fără a le rescrie de la zero.

► Un *modul* este un fișier Python care conține instrucțiuni și funcții.

Exemplu: modulul pentru calcule matematice `math`.

► O *bibliotecă* este o colecție mai mare de module, clase care oferă funcționalități pentru diverse domenii. *Exemplu:* biblioteca pentru interfețe grafice `Tkinter`.

2.2.4.1. Clasa „int”

În Python, `int`, `float` și `str` nu sunt doar tipuri de date, ci și clase care definesc comportamentul acestor date. Atunci când scriem `x = 10`, nu doar atribuim o valoare, ci creăm de fapt un obiect (o instanță) al clasei `int`. Cu alte cuvinte, `10` este rezultatul instanțierii clasei `int`, iar variabila `x` face referire la acest obiect. Acest lucru reflectă principiul fundamental din Python conform căruia aproape totul este un obiect, iar valorile au atât date, cât și metode asociate.

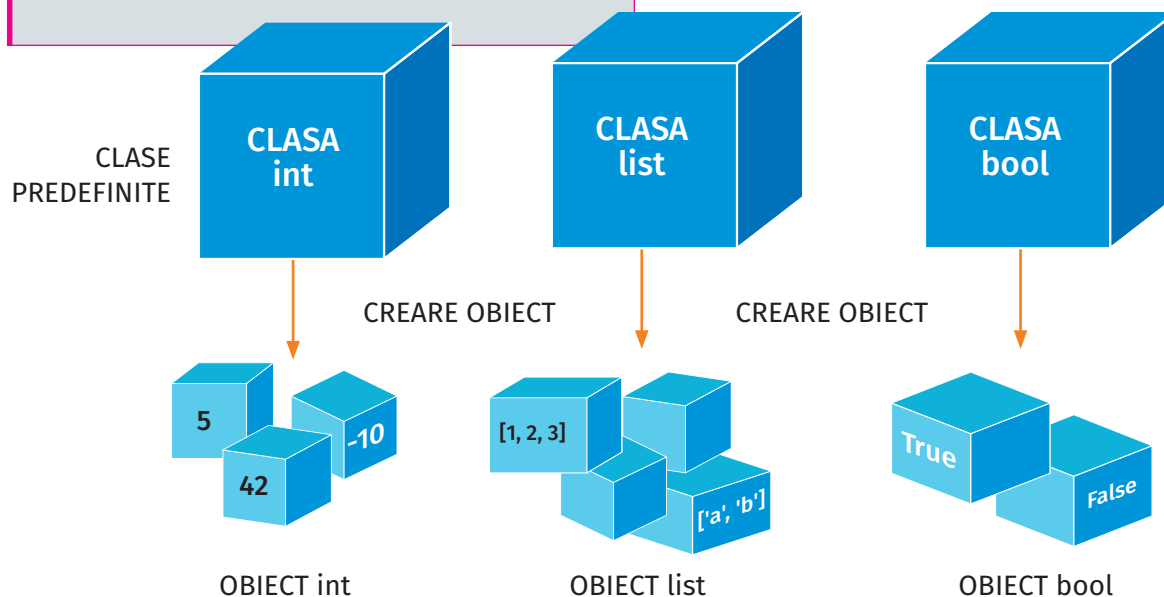
2.2.4.2. Clasa „list”

În Python, `list` este o clasă care definește comportamentul listelor. Când scriem `l = [1, 2, 3]`, de fapt creăm un obiect (instanță) al clasei `list`. Echivalent, putem scrie și `l = list([1, 2, 3])`, ceea ce arată mai clar instanțierea clasei.

Variabila `l` face referire la acest obiect, iar el are metode proprii, cum ar fi `l.append(4)`, care modifică lista. Astfel, la fel ca `int`, și `list` este o clasă, iar valorile de tip listă sunt obiecte create din ea.

2.2.4.3. Clasa „bool”

În Python, un `bool` (tipul `bool`) reprezintă o valoare logică: `True` sau `False`. `Bool` apare în orice expresie logică și atunci când verificăm condiții precum: Numărul este mai mic decât `10`? (`True/False`) sau Parola este corectă? (`True/False`).





FIȘĂ DE LUCRU 4



1. Pentru următoarele clase:

- Identifică: numele clasei, atributele, metodele, obiectele create și mesajele afișate în consolă.
- Creează 3 noi obiecte, având diverse valori pentru câmpurile definite.
- Ce alte atribute și metode putem adăuga claselor date?
- Pentru clasa `Produs`, creează metoda `majorare (self, procent)` care afișează prețul unui produs după majorarea acestuia cu procentajul primit ca parametru.
- Compară produsele `p1` și `p2` și afișează numele produsului cu prețul maxim.
- Pentru toate obiectele instanțiate în program, calculează suma totală a prețurilor acestora.

Coduri Python

```
class Produs:
    def __init__(self, nume, pret):
        self.nume = nume
        self.pret = pret
    def afiseaza(self):
        print(self.nume, "-", self.pret, "lei")
p1 = Produs("Paine", 5)
p2 = Produs("Lapte", 7)
p1.afiseaza()
p2.afiseaza()

class Semafor:
    def __init__(self, culoare):
        self.culoare = culoare
    def actiune(self):
        if self.culoare == "verde":
            print("Treci!")
        elif self.culoare == "galben":
            print("Atentie!")
        else:
            print("Oprește!")
s = Semafor("rosu")
s.actiune()
```

2. Folosind modelul claselor anterioare, definește clasa `Elev` cu atributele `nume`, `clasa` și metoda `salut()`, care afișează: "Sunt elevul X din clasa Y".

- Creează 2 elevi și apelează metoda.
 - Verifică dacă cei doi elevi sunt în aceeași clasă și afișează un mesaj corespunzător.
- 3.a. Creează clasa `Telefon` cu atributele `brand`, `baterie` și metoda `incarca()`, în care bateria crește cu 10, în cazul în care valoarea acesteia este mai mică decât 100.
- b. Instanțiază un obiect din clasa `Telefon`. Să se verifice dacă `bateria` este sub 30, iar în acest caz să se apeleze metoda `încarcă`.

IMPORTANT



Clasă = șablon; **Obiect** = instanță (exemplar).
Obiectele au **atribute** (date) și **metode** (acțiuni)
Accesul la metode se face cu `.(punct): obiect.metoda()`.



2.3. Recapitulare

1. Scrie un program în Python care citește de la tastatură soldul unui client al unei bănci comerciale. Definește două funcții: una care actualizează soldul după depunerea unei sume în cont și alta care determină soldul după retragerea unei sume (dacă este posibil). Testează funcționarea ambelor funcții în programul principal.
2. Definește o funcție Python care să rezolve ecuația de gradul al doilea cu coeficienți reali, citiți de la tastatură. Funcția primește ca parametri coeficienții ecuației, iar apelul acesteia se va realiza în programul principal.
3. Se citesc de la tastatură n numere naturale (n – natural, $1 \leq n \leq 10$). Verifică dacă numărul format cu prima cifră a fiecărui număr citit este palindrom. Pentru rezolvare se vor defini cel puțin două funcții.
4. Se consideră următoarea clasă:

```
class Dreptunghi:
    def __init__(self, lungime, latime):
        self.lungime = lungime
        self.latime = latime

    def calculeaza_aria(self):
        return self.lungime * self.latime
```

- a. Identifică atributele și metodele clasei.
- b. Ce reprezintă parametrul `self`?
- c. Definește metoda `calculeaza_perimetru()` pentru un dreptunghi.
- d. Creează o instanță a clasei `Dreptunghi` și afișează mesajul *Adevărat* dacă perimetrul dreptunghiului este mai mare decât aria acestuia, altfel afișează mesajul *Fals*.
- e. Pentru obiectul instanțiat anterior, verifică dacă este pătrat (lungimea este egală cu lățimea).
- f. Creează metoda `calculeaza_diagonala(self)` care returnează valoarea diagonalei (folosiți teorema lui Pitagora).
- g. Pentru obiectul instanțiat anterior, afișează pe ecran lungimea diagonalei.

MINIPROIECTE – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVİ

1. Proiectează o aplicație modulară Python, de tip calculator matematic, pentru cele patru operații de bază: `+`, `-`, `*`, `/`, aplicate pe date numerice de tip real. Veți implementa un meniu care să permită selectarea, în mod repetat, a operației dorite. Cerințe:
 - a) descompune problema în cel puțin 4 subprograme și stabilește specificațiile fiecărui subprogram: rol, parametri, valori returnate; realizează proiectarea modulară în limbaj pseudocod;
 - b) implementează, în limbajul Python, soluția problemei.
2. Se consideră următorul scenariu: știind datele a doi elevi (nume, clasă, nota 1 și nota 2), determină media notelor pentru fiecare elev, precum și numele elevului cu media cea mai mare. Cerințe:
 - a) implementează soluția problemei în variantă procedurală;
 - b) implementează soluția problemei în variantă POO (Programare Orientată pe Obiecte).

R E C A P I T U L A R E

2.4. Evaluare sumativă

V
A
L
U
A
R
E

I. Rezolvă:

1. Precizează afirmația care descrie corect o funcție în Python? 0,5 p.
- Este o variabilă specială care memorează valori reale.
 - Este un bloc de instrucțiuni care poate fi apelat de mai multe ori.
 - Este o structură repetitivă.
 - Este un tip de date numeric.

2. Stabilește valoarea de adevăr a următoarelor afirmații
(Adevărat sau Fals): 4x0,25 p. = 1 p.
- O funcție poate să nu aibă parametri.
 - Definiția unei funcții trebuie să conțină instrucțiunea `return`.
 - O funcție poate să returneze mai multe valori.
 - O variabilă locală poate fi accesată în programul principal.

3. Rezultatul execuției programului este: 0,5 p.

<pre>def f(a, b=2, c=3): return a + b + c print(f(1, c=5))</pre>	a) 6 b) 8 c) 9 d) Eroare
--	--

4. Stabilește rezultatul execuției programului: 0,5 p.

```
nr=3
def f(nr):
    return 3*nr
print(f(nr), f(1))
```

5. Alege două dintre clasele date și completează, pentru fiecare dintre ele, cel puțin 3 atribute și cel puțin 3 metode. Exemple de clase: Telefon, Bicicletă, Profesor, Cont bancar, Animal de companie. 1 p.

6. Completează spațiile libere, astfel încât: 2x0,25 p. = 0,5 p.

<pre>#a) class Clasa1: x=10 a1=_____ print(_____)</pre>	– să definești <code>a1</code> ca obiect al clasei <code>Clasa1</code> utilizând obiectul <code>a1</code>, – să afișezi valoarea lui <code>x</code>
<pre>#b) x = 300 def Functia1(): _____ x = 200 Functia1() print(x)</pre>	Programul să afișeze valoarea <code>200</code>

R E C A P I T U L A R E

7. Asociază fiecare termen din **coloana A** cu definiția corespunzătoare din **coloana B**.

4x0,25 p. = 1 p.

Coloana A	Coloana B
Clasă	Acțiune sau comportament (definit într-o clasă) care va caracteriza un obiect.
Obiect	Șablon sau model care descrie atributele și metodele viitoarelor obiecte.
Metodă	Dată sau proprietate (definită într-o clasă) care va caracteriza un obiect.
Atribut	O realizare concretă a clasei, o instanță a clasei.

II. Miniproiect

1x2 p. = 2 p.

Realizează, la alegere, unul dintre cele două miniproiecte rezolvate în cadrul secțiunii *Recapitulare*.

III. Portofoliu

2x1 p. = 2 p.

La finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul *Unitate2_NumeElev*, următoarele rezolvări: *Fișa de lucru 3* și *Fișa de lucru 4*.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*. Se acordă 1 punct din oficiu.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

CAPITOLUL 3

Memorarea și prezentarea datelor în limbaj de programare

COMPETENȚE
SPECIFICE:

1.4, 2.4,
3.4, 4.4,
5.4, 6.4

3.1 Fișiere text

3.2 Biblioteca Tkinter pentru interfețe grafice

3.3 RECAPITULARE

3.4 EVALUARE SUMATIVĂ





3.1. Fișiere text



În domeniul programării, eficiența unei soluții depinde foarte mult de modul în care alegem să organizăm datele în cadrul codului. O colecție de date, caracterizată prin *relațiile* dintre acestea și prin *operațiile* care pot fi realizate cu datele respective, poartă numele de **structură de date**.



ȘTIAI CĂ...?

Sistemul hexazecimal (baza 16) utilizează, pentru codificare, simbolurile: cifrele de la 0 la 9 și caracterele A, B, C, D, E, F.

Fișierele text joacă un rol esențial în dezvoltarea aplicațiilor software din următoarele motive:

- ▶ persistența datelor – permit stocarea datelor pentru utilizări ulterioare, păstrând rezultatele chiar și după ce execuția programului s-a încheiat;
- ▶ schimbul de date – facilitează comunicarea între programe diferite prin funcții de export și import (rapoarte, statistici);

3.1.1. Caracteristici, principii de lucru

Clasificarea structurilor de date, în funcție de tipul de memorie în care sunt create:

- ▶ *Structuri de date interne* – sunt memorate în memoria internă (RAM) a calculatorului și sunt utilizate în timpul execuției programelor (exemplu: listă – structură de date ce va fi studiată în Unitatea 4).
- ▶ *Structuri de date externe* – sunt memorate pe suporturi de stocare externe (hard disk, SSD) și sunt folosite pentru a păstra cantități mari de date pe termen lung (exemplu: fișiere).

În programare, un **fișier text** este o structură de date externă care stochează datele sub formă de **secvențe de caractere** (ASCII sau Unicode).

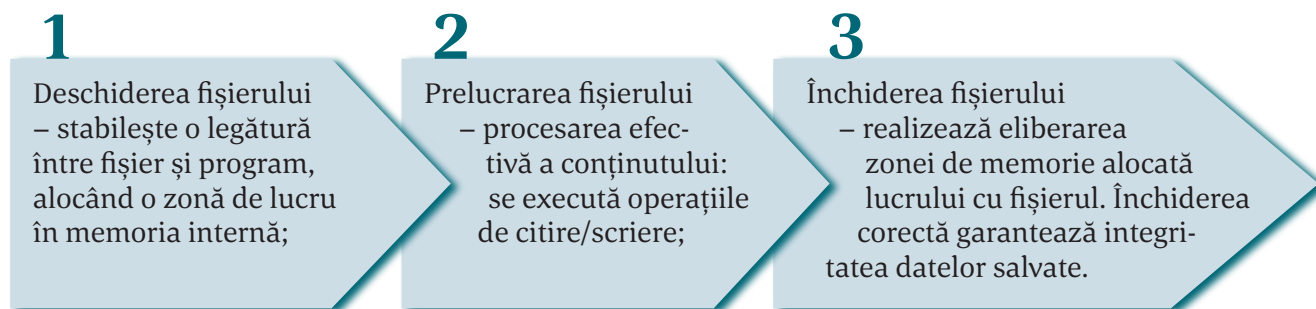
- ▶ **ASCII** – este un sistem de codificare a caracterelor alcătuit din caractere imprimabile (litere ale alfabetului englez, cifre, diferite simboluri) și caractere neimprimabile (coduri de control). Fiecărui caracter îi este asociat un număr natural unic, denumit cod ASCII, de exemplu caracterul **A** are codul **65**, iar caracterul **a** are codul **97**.
- ▶ **Unicode** – este un standard de codificare a caracterelor din diverse limbi ale lumii, a simbolurilor și chiar a emoticoanelor. Codul Unicode este alcătuit din prefixul U+, urmat de exprimarea în sistem hexazecimal a simbolului (de exemplu: caracterul "Ș" are codul **U+0218**).

Caracteristicile unui fișier text:

- este alcătuit din linii de caractere de lungimi diferite;
- poate fi creat și modificat cu un editor de texte (Notepad) sau direct în editorul unui limbaj de programare;
- are un nume și o extensie specifică pe suportul extern (exemplu .txt, .log, .py)
- sfârșitul de linie este marcat prin caracterul *sfârșit de rând* **EOL** (End Of Line);
- sfârșitul de fișier este marcat prin caracterul *sfârșit de fișier* **EOF** (End Of File).

- ▶ fundamentul în tehnologie – există aplicații care se bazează pe formatul de fișiere text (pagini web, coduri sursă ale programelor, fișiere de configurare);
- ▶ mentenanța – simplitatea deschiderii și modificării manuale a fișierelor ușurează procesele de testare și întreținere;
- ▶ scalabilitatea – permit prelucrarea unor volume de date care depășesc capacitatea curentă a memoriei de lucru.

Principii de lucru cu fișiere text. Există trei etape de lucru, realizate în următoarea ordine:



3.1.2. Implementarea fișierelor text în limbajul de programare Python

3.1.2.1 Deschiderea fișierelor text

Se utilizează funcția *built-in* `open()`, cu următoarea sintaxă:

```
nume_logic_fis=open(nume_fizic_fis, mod_desch, encoding="utf-8")
```

Exemplu: `f=open("date.txt", "r")`

Deschide fișierul text, cu numele fizic `date.txt`, și returnează instanța unui obiect de tip fișier al clasei predefinite `TextIOWrapper`, atribuie noul obiect creat variabilei `f` (`f` numele logic al fișierului); fișierul este deschis în vederea operației de citire (`mod_deschidere` este `"r"`). Operația de deschidere:

- ▶ stabilește o relație între numele fizic al fișierului și numele logic al acestuia; după operația de deschidere, se va opera în codul Python cu numele logic;
- ▶ stabilește operația permisă asupra fișierului, indicată prin modul de deschidere;
- ▶ instanțiază clasa predefinită `TextIOWrapper`.



IMPORTANT

Primii doi parametri sunt de tip șir de caractere. Pentru situația în care fișierul conține strict caractere ASCII, al treilea parametru este opțional.



ȘTIAI CĂ...?

Fișierele `.py` sunt și ele fișiere text? Un program Python reprezintă un simplu fișier text salvat cu extensia `.py` care conține instrucțiuni specifice limbajului de programare.

Moduri de deschidere disponibile pentru fișierele text:

Caracterul	Semnificație
<code>r</code>	Deschide fișierul în vederea operației de citire (este modul implicit, poate să lipsească). Fișierul trebuie să existe în folderul de lucru curent (sau în calea explicită, dacă este precizată), în caz contrar, la execuție, se obține eroarea <code>FileNotFoundError</code> .
<code>w</code>	Deschide fișierul în vederea operației de scriere. Se va crea un nou fișier în folderul de lucru curent (sau în calea explicită, dacă este precizată). Dacă fișierul există, se va realiza suprascriere.
<code>x</code>	Deschide fișierul pentru scriere, cu crearea unui nou în folderul de lucru curent (sau în calea explicită, dacă este precizată), în caz că nu există. Dacă fișierul există, la execuție se obține eroarea <code>FileExistsError</code> .
<code>a</code>	Deschide fișierul în vederea operației de adăugare la sfârșitul fișierului, dacă există. Dacă fișierul nu există, se va crea unul nou în folderul de lucru curent (sau în calea explicită, dacă este precizată).
<code>t</code>	Deschide fișierul în mod text (modul implicit și poate să lipsească).



IMPORTANT

- numele fizic al fișierului poate să conțină și calea explicită către fișier. *Exemplu:*
`f=open("D:/date.txt", "w")`,
 fișierul va fi creat în rădăcina discului **D:**
 - la deschiderea unui fișier, **poziția de citire** (pe care o denumim **cursor** sau **pointer**) se va afla pe **primul caracter, aflat pe poziția 0**.
 - datele pe care le citim/ scriem, din/în fișiere text, sunt de tip șir de caractere.

» **Scrierea datelor într-un fișier text** (`mod_deschidere` este **"w"**)

Metoda `write(sir_de_caractere)` – scrie în fișier șirul de caractere specificat și returnează numărul de caractere scrise.
 Obiectele de alt tip, cum ar fi valorile numerice, necesită convertirea la tipul șir de caractere.

» **Adăugarea datelor într-un fișier text** (`mod_deschidere` este **"a"**)

Se utilizează metoda `write(sir_de_caractere)` pentru adăugare la sfârșitul fișierului.

3.1.2.2. Prelucrarea fișierelor text

Metodele pe care le putem utiliza pentru prelucrarea unui fișier de tip text le regăsim în clasa predefinită `TextIOWrapper`. Pentru referirea la o metodă vom folosi sintaxa:
`nume_logic_fișier.metoda()`

» **Citirea datelor dintr-un fișier text** (`mod_deschidere` este **"r"**)

1. Metoda `read(valoare)` – unde `valoare` este un număr întreg, funcția returnează șirul de caractere citit.

- Dacă `valoare` este negativ sau lipsește, atunci întregul conținut al fișierului este citit și returnat de funcție ca șir de caractere.
- Dacă `valoare` este pozitiv, atunci se citește un număr de caractere egal cu `valoare`.

Funcția returnează șirul de caractere citit.

Atenție!

- Caracterele *linie nouă* (`"\n"`), aflate în fișierul text, se memorează în șirul returnat de funcția `read()`.
- Sfârșitul de fișier (**EOF**) se detectează prin șirul vid (șirul care nu conține niciun caracter: `" "`).

2. Metoda `readline()` – citește o singură linie din fișier, caracterul *linie nouă* (`"\n"`) se memorează la sfârșitul șirului de caractere (mai puțin pentru ultima linie, în cazul în care nu există acest caracter). Funcția returnează șirul de caractere citit.



3.1.2.3. Închiderea fișierelor text

După ce am prelucrat fișierul, urmează operația de închidere cu metoda `close()`.

Această operație este extrem de importantă din următoarele motive: se eliberează resursele sistemului pe care prelucrarea fișierului le-a blocat și se elimină eventualele erori sau pierderi de date ce pot apărea la prelucrarea fișierelor.



Pentru următoarele exemple, *fișierul text **date.txt***, are trei linii de text, editate fără diacritice, și nu are caracterul *linie nouă* ("**\n**") la final.

date.txt

31 28
Ianuarie Februarie
Gerar Faurar

Cod Python

```
#Deschide fișierul pentru citire, citește
#întregul conținut și îl memorează în variabila sir
f=open("date.txt","r")
sir=f.read()
print(sir)
#Închide fișierul
f.close()

#Deschide pentru citire
f=open("date.txt")
#Citește șirul alcătuit din primele 5 caractere
sir=f.read(5)
print(sir)
f.close()

#Deschide și citește fișierul pe linii
f=open("date.txt")
#Citește prima linie, are la final "\n"
sir=f.readline()
while sir!="":    #test de EOF
    #Afișează linia citită curent
    print(sir)
    # Citește următoarea linie
    #dacă nu este ultima, are la final "\n"
    sir=f.readline()
f.close()

#Deschide și citește fișierul pe linii
f=open("date.txt")
for sir in f:
    print(sir)
f.close()

#Deschide pentru scriere, creează un nou fișier
#dacă există, suprascrie
f=open("rezultate.txt","w")
f.write("Prima linie\n")
f.write("A doua linie")
#trebuie să convertim valoarea numerică la str
f.write(str(12))
f.close()

#Deschide pentru adăugare la sfârșitul fișierului
f=open("rezultate.txt","a")
f.write("\n")
f.write("A treia linie")
f.close()
```

Exemplu de rulare

31 28
Ianuarie Februarie
Gerar Faurar

31 28

31 28

Ianuarie Februarie
Gerar Faurar

31 28

Ianuarie Februarie
Gerar Faurar

Fișierul rezultate.txt:

Prima linie
A doua linie12

Fișierul rezultate.txt:

Prima linie
A doua linie12
A treia linie



INVESTIGAȚIE. LUCRU ÎN ECHIPĂ

Scop: Compararea timpului de executare între un program care citește datele de la tastatură și unul care citește datele din fișier; se analizează impactul privind performanța programului. **Formă de organizare:** Grupe de câte doi elevi.

Mod de desfășurare: Considerând cele două coduri Python de mai jos, stabiliți timpul de executare pentru fiecare variantă. Se vor utiliza aceleași date de intrare: 1,2,3,4,...,15. În fișierul text, datele sunt scrise câte unul pe fiecare rând.

Indicație: S-a utilizat metoda `time()`, din modulul predefinit `time`, care returnează un număr real (timpul curent, exprimat în secunde, de la momentul de referință 1 Ianuarie 1970).

```
#Varianta 1 - citire tastatură
import time
start = time.time()
n = 15
for i in range(n):
    a = int(input())
stop = time.time()
print(stop - start, "secunde")
```

```
#Varianta 2 - citire din fișier
import time
start = time.time()
n = 15
f=open("date.txt")
for i in range(n):
    a =int(f.readline())
f.close()
stop = time.time()
print(stop - start, "secunde")
```

Analiza datelor și concluzii: Discutați cu profesorul vostru despre rezultatele obținute și formulați concluziile, completând (pe caiet) un tabel ca cel de mai jos.

Criteriu	Tastatură	Fișier
Timp aproximativ de execuție (secunde)		
Viteză de execuție (lentă/rapidă)		
Dependență de utilizator (Da/Nu)		
Potrivit pentru volum mare de date (Da/Nu)		

3.1.2.4. O tehnică de bună practică în lucrul cu fișierele text

Există situații în care un program conține o eroare la interpretare și atunci metoda de închidere a fișierului nu se mai execută.

O bună practică de lucru este utilizarea instrucțiunii `with`, care are marele avantaj că **fișierul este închis automat**.

În cadrul instrucțiunii `with`, toate instrucțiunile referitoare la prelucrarea fișierului păstrează aceeași indentare.



IMPORTANT

După ce un obiect fișier a fost închis (fie apelând funcția `close()`, fie prin utilizarea instrucțiunii `with`), orice încercare de a prelucra fișierul va genera eroare. Rezolvăm această problemă, dacă deschidem din nou fișierul.



>>> EXEMPLE

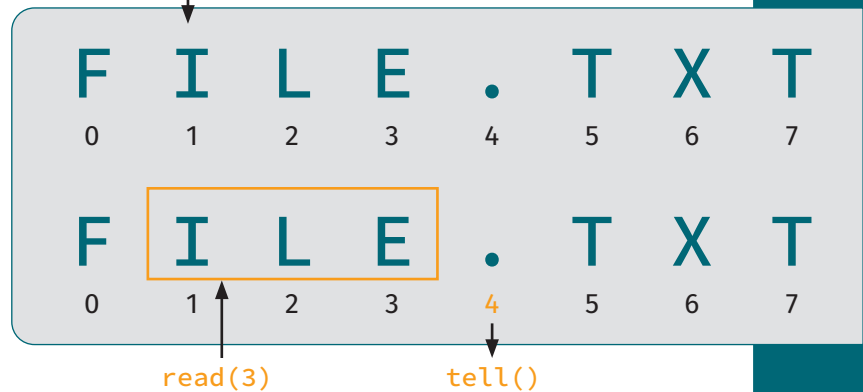
Cod Python	Exemplu de rulare
<pre>#Deschide fișierul pentru citire with open("date.txt", "r") as f: sir = f.readline() while sir != "": print(sir) sir = f.readline() #fișierul este închis automat #Deschide fișierul pentru scriere with open("rezultate.txt", "w") as f: f.write("Prima linie\n") f.write("A doua linie") f.write(str(12)) #fișierul este închis automat #Deschide fișierul pentru adăugare with open("rezultate.txt", "a") as f: f.write("\n") f.write("A treia linie") # fișierul este închis automat</pre>	<pre>31 28 Ianuarie Februarie Gerar Faurar</pre>
	Fișierul rezultate.txt: <pre>Prima linie A doua linie12 A treia linie</pre>

3.1.2.5. Alte metode utile în prelucrarea fișierelor text

- 1 Metoda `tell()`** –
returnează poziția curentă
a pointerului (cursorului)
de citire.
- 2 Metoda `seek(poz)`** –
mută pointerul de citire
la poziția `poz` și returnează
valoarea `poz`.

EXEMPLU >>>

`seek(1)` → Fișierul `date.txt` are conținutul: `FILE.TXT`



Cod Python	Exemplu de rulare
<pre>with open("date.txt") as f: #poziționare pe al 2-lea caracter (cel de indice 1) f.seek(1) #citește 3 caractere, le memorează în variabila sir #și avansează pointerul pe următorul caracter sir=f.read(3) print(sir) print(f.tell())</pre>	<pre>ILE 4</pre>



3.1.2.6. Probleme rezolvate



PROBLEMA 1

Enunț: O aplicație software, folosită pentru colectarea de mesaje de la participanții unui atelier de comunicare, primește n mesaje text ($n \geq 1$), fiecare mesaj putând să conțină și **diacritice românești**. Pentru a organiza un fișier de arhivă, aplicația va salva, în fișierul text `date.txt`, fiecare mesaj pe câte o linie, atașând, la finalul fiecăruia, un **emoticon zâmbitor** (de exemplu: simbolul 😊, ce are codul Unicode `U+1F60A`). Ulterior, pentru verificare, aplicația va afișa pe ecran conținutul fișierului `date.txt`, numerotând mesajele și afișându-le pe linii separate.

Analiza problemei: Mesajele includ diacritice și caractere grafice speciale, din acest motiv este necesară deschiderea fișierului prin specificarea obligatorie a celui de-al treilea parametru `encoding="utf-8"`.

Cod Python

```
#Citire de la tastatură / scriere în fișier
n = int(input("n="))
with open("date.txt","w",encoding="utf-8") as f:
    for i in range(n):
        mesaj=input("mesaj=")
        f.write(mesaj)
        #Codul Unicode al emoticonului și sfârșit de linie
        f.write("\U0001F60A"+"\\n")

#Citire din fișier / scriere pe ecran
with open("date.txt","r",encoding="utf-8") as f:
    for i in range(n):
        mesaj=f.readline()
        print(str(i+1),mesaj)
```

Testare

```
n=3
mesaj=A fost o experiență utilă și interesantă!
mesaj=Organizarea a fost foarte bună!
mesaj=Mă bucur că am avut ocazia să fiu aici!

1 A fost o experiență utilă și interesantă! 😊

2 Organizarea a fost foarte bună! 😊

3 Mă bucur că am avut ocazia să fiu aici! 😊
```



PROBLEMA 2

Enunț: O companie care dezvoltă senzori pentru monitorizarea calității aerului, dorește să testeze modulul software de analiză a datelor. Pentru simulare, se vor genera automat n ($n \geq 1$), valori numerice aleatoare, reprezentând concentrația zilnică a unui anumit poluant (valorile sunt cuprinse între 1 și 99). Programul trebuie să memoreze cele n valori în fișierul text `date.txt` și să determine câteva informații statistice: valoarea maximă înregistrată și numărul zilelor în care nu s-a depășit pragul valorii de 50.

Analiza problemei: Se va utiliza modulul predefinit `random`, care conține definiția mai multor funcții cu rol în generarea valorilor aleatoare. Una dintre aceste funcții este `randint(a,b)`, unde a și b sunt numere întregi, $a \leq b$; funcția returnează un număr aleator întreg cuprins între a și b .

Cod Python

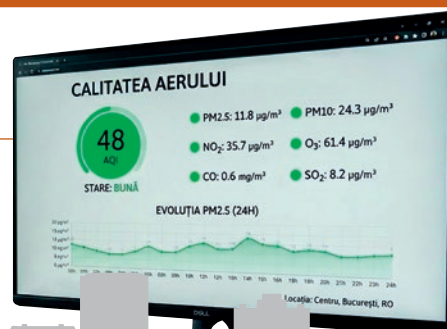
```
# instrucțiune de import
import random
n = int(input("n="))
maxv=0
nrz=0
with open("date.txt", "w") as f:
    for i in range(n):
        # apelul funcției randint
        #returneaza un nr aleator cuprins între 1 si 99
        a=random.randint(1,99)
        if a>maxv:
            maxv=a
        if a<=50:
            nrz+=1
        # scriere in fisier
        f.write(str(a))
        f.write("\n")
print("Cea mai mare valoare înregistrată:", maxv)
print("Nr de zile cu valori de cel mult 50:",nrz)
```

date.txt

97
40
40
94
12
45
1
32
10
29

Exemplu de rulare

n=10
Cea mai mare valoare înregistrată: 97
Nr de zile cu valori de cel mult 50: 8





AUTOEVALUARE



Completează spațiile libere astfel încât programul alăturat să citească un număr natural nenul `n` din fișierul text `date.txt` și să scrie, în fișierul `rezultate.txt`, toți divizorii numărului `n`, pe prima linie, despărțiți de câte un spațiu. Testează în Python codul obținut.

```
with open("date.txt", "r") as f:
    n = int(-----)

g = open("rezultate.txt", "w")
for i in range(1, -----):
    if -----:
        g.write(-----)
    -----
```

Barem de evaluare: Se vor acorda câte 2 puncte pentru fiecare spațiu liber completat corect; total punctaj: 10 puncte.



FIȘĂ DE LUCRU 5

1. Se consideră următoarea secvență de cod:

```
contor=0
with open("date.txt", "r") as f:
    sir = f.readline()
    while sir != "":
        contor += 1
        print(int(sir))
        sir = f.readline()
print("Fișierul are", contor, "linii")
```

Testează programul, în situațiile precizate mai jos, și explică efectul obținut.

- Fișierul `date.txt` nu există în folder-ul de lucru curent.
- Fișierul `date.txt` nu are conținut (este gol).
- Fișierul `date.txt` are următorul conținut:

varianta 1	varianta 2	varianta 3
23	23	23
ab		1
17	17	17

- Verifică dacă două fișiere text conțin același număr de linii, afișând un mesaj corespunzător.
- În fișierul `date.txt` se află mai multe numere naturale scrise pe linii diferite. Verifică dacă numerele din fișierul `date.txt` sunt în ordine crescătoare, afișând pe ecran mesajul *Sunt crescătoare*, respectiv *Nu sunt crescătoare*, în caz contrar.

3.2. Biblioteca Tkinter pentru interfețe grafice

3.2.1. Motivația utilizării interfețelor grafice

Interfețele grafice sunt folosite pentru a face programele mai ușor de utilizat și mai intuitive pentru utilizatori. În loc să scriem comenzi într-o consolă, putem interacționa cu aplicațiile folosind elemente vizuale precum ferestre, butoane, meniuri sau câmpuri de text. Acest mod de lucru este mai familiar pentru majoritatea utilizatorilor, deoarece este similar cu aplicațiile pe care le folosim zilnic pe calculator sau pe telefon.

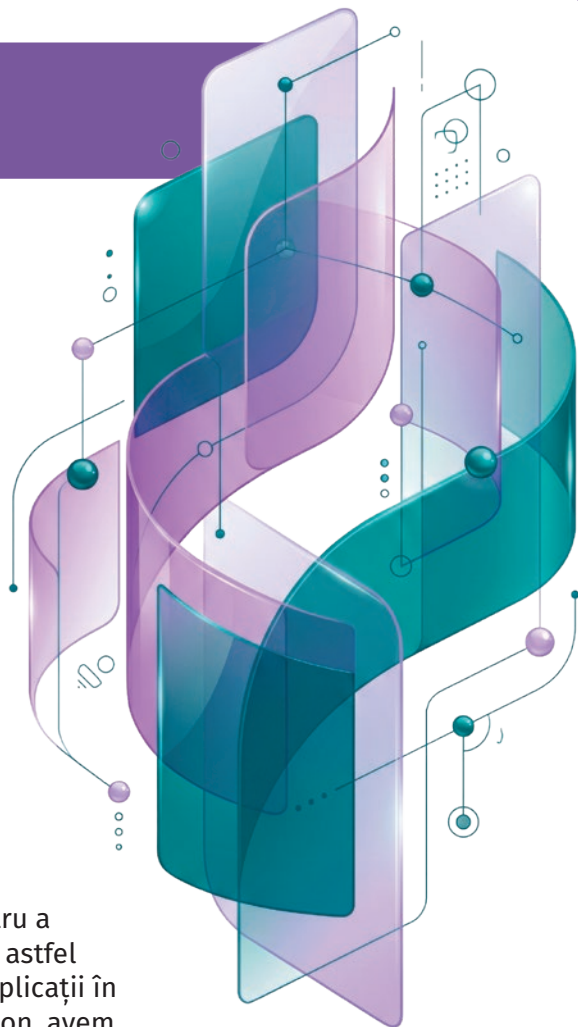
Utilizarea interfețelor grafice permite organizarea clară a informațiilor și a funcțiilor unui program.

Aplicațiile dezvoltate devin mai prietenoase și mai ușor de folosit, chiar și pentru persoane fără cunoștințe de programare. În plus, acestea ajută la crearea unor programe mai atractive și mai apropiate de aplicațiile reale.

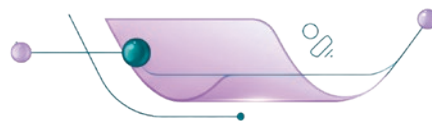
Totuși, interfețele grafice au și unele dezavantaje. **Dezvoltarea lor este, de obicei, mai complexă și necesită mai mult timp decât aplicațiile realizate în linie de comandă.** De asemenea, pot consuma mai multe resurse ale calculatorului, cum ar fi memoria și procesorul. În anumite situații, utilizatorii avansați pot considera linia de comandă mai rapidă și mai eficientă pentru sarcini simple.

În viața reală, interfețele grafice ne ajută să folosim mult mai ușor programele. De exemplu, pe telefon sau calculator apăsăm pe pictograme, butoane și meniuri pentru a deschide aplicații sau a realiza acțiuni. **Fără interfețe grafice, ar trebui să introducem comenzi complicate pentru operații simple, ceea ce ar îngreuna utilizarea dispozitivelor.**

Interfețele grafice permit și o interacțiune rapidă între utilizator și program. Atunci când utilizatorul apasă un buton sau introduce un text, aplicația răspunde imediat prin afișarea unui rezultat, a unui mesaj sau prin deschiderea unei noi ferestre. Această comunicare directă face aplicațiile mai dinamice și mai ușor de utilizat.



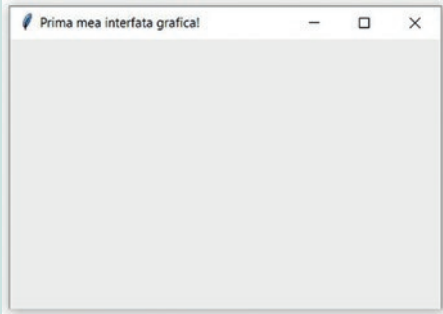
Pentru a crea astfel de aplicații în Python, avem nevoie de biblioteci speciale care oferă componente grafice și metode pentru organizarea lor într-o fereastră. Una dintre cele mai simple și utilizate biblioteci este **Tkinter**, care permite realizarea rapidă a aplicațiilor grafice educative și practice.



3.2.2. Introducere. Ce este Tkinter și de ce îl folosim?

Tkinter este o bibliotecă standard din Python pentru crearea interfețelor grafice (GUI): ferestre, butoane, câmpuri de text, zone de desen etc. Cu Tkinter poți transforma un program care rulează doar în consolă într-o aplicație cu ferestre, similară cu cele pe care le folosești zilnic.



Cod Python	Exemplu de rulare
<pre>print("Prima mea interfata grafica!")</pre>	în consolă: <pre>Prima mea interfata grafica!</pre>
<pre>import tkinter as tk root = tk.Tk() root.title("Prima mea interfata grafica!") root.geometry("400x250") root.mainloop()</pre>	interfața grafică creată: 

Pregătire: instalare și primul test. În majoritatea instalărilor Python (mai ales pe Windows), Tkinter vine deja inclus. Iată cum testăm dacă acesta este instalat:

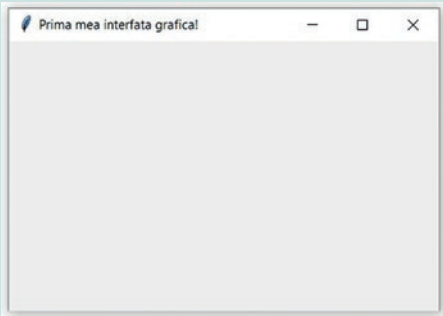
Cod Python	Exemplu de rulare
<pre>import tkinter as tk print("Tkinter functioneaza!")</pre>	în consolă: <pre>Tkinter functioneaza!</pre>

Dacă nu apare nicio eroare, totul a funcționat corect, biblioteca fiind disponibilă pentru utilizare.

3.2.2.1. Primul program cu fereastră ▶ `root = tk.Tk()` = creează fereastra principală a unei aplicații grafice în Python folosind biblioteca Tkinter.

Idee importantă:

- `tk` este prescurtarea pentru modulul Tkinter;
 - `Tk()` creează fereastra principală a aplicației, rezultatul este un obiect de tip fereastră;
 - variabila `root` stochează acest obiect creat în care se salvează această fereastră, pentru a putea fi modificată ulterior.
- ▶ `mainloop()` = aplicația așteaptă acum evenimente (click, apăsarea unei taste).

Cod Python	Exemplu de rulare
<pre>import tkinter as tk # Creeaza fereastra principala root = tk.Tk() # Titlul ferestrei root.title("Prima mea interfata grafica!") # Latime x inaltime (pixeli) root.geometry("400x250") # Porneste "motorul" aplicatiei root.mainloop()</pre>	interfața grafică creată: 

În codul anterior, `import tkinter as tk` încarcă biblioteca Tkinter și îi atribuie aliasul `tk` pentru a fi folosită mai ușor în program.



3.2.3. Componente de bază

3.2.3.1. Label – text afișat în fereastră

În Tkinter, clasa `Label` este componenta grafică pentru afișarea unui text sau a unei imagini într-o fereastră. Reprezintă o componentă statică în sensul că, implicit, nu primește input de la utilizator (spre deosebire de clasa `Entry`). Cu ajutorul acestei componente se poate afișa un **text** (titlu, instrucțiuni, status), se pot afișa imagini, se poate actualiza dinamic textul (schimbarea textului în timpul rulării).

Cod Python

```
import tkinter as tk
```

```
root = tk.Tk()
root.title("Label")
root.geometry("300x150")
```

Varianta 1 de Label (se obtine interfata din stanga)

```
lbl = tk.Label(root, text="Salut! Eu sunt un Label.")
lbl.pack()
```

```
root.mainloop()
```

Se poate schimba aspectul unui Label inlocuind linia

Varianta 1 cu linia urmatoare (se obtine interfata din dreapta)

```
lbl = tk.Label(root, text="Salut! Eu sunt un Label.", font=("Arial", 18), fg="blue")
```

Parametri utili clasei Label:

- ▶ `text="..."` – textul afișat;
- ▶ `font=("Arial", 14, "bold")` – font;
- ▶ `fg="red" / bg="white"` – culoare text/fundal;
- ▶ `width=20, height=2` – dimensiune (în caractere/linii);
- ▶ `anchor="w"` – aliniere în interior (w/e/n/s/center);
- ▶ `justify="left"` – justificare pentru text pe mai multe linii;
- ▶ `wraplength=200` – limitează lățimea liniei de text, exprimată în pixeli; textul mai lung se rupe automat pe mai multe linii;
- ▶ `padx=10, pady=5` – spațiere internă.

Pentru actualizarea textului unui `Label`, `lbl.config(text="Text nou")` schimbă direct textul afișat într-un `Label` deja creat. `tk.StringVar(value="Initial")` creează o variabilă specială pentru text, cu o valoare inițială, care poate fi modificată dinamic. Când un `Label` este creat cu `textvariable=var`, el afișează conținutul variabilei `var` și se actualizează automat când aceasta se schimbă. Prin `var.set("Actualizat")`, valoarea variabilei este modificată, iar textul din `Label` se actualizează imediat în interfață.

Cod Python

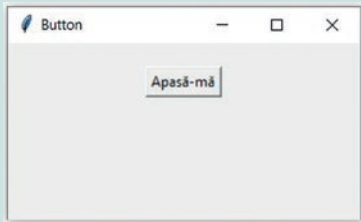
```
lbl.config(text="Text nou")
```

```
var = tk.StringVar(value="Initial")
lbl = tk.Label(root, textvariable=var)
var.set("Actualizat")
```



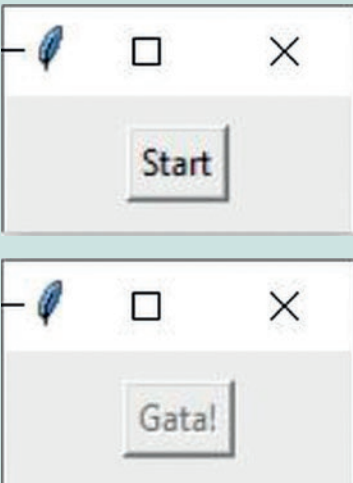
3.2.3.2. Button – buton care execută o acțiune

În Tkinter, clasa `Button` este componenta grafică care creează și afișează în interfața grafică un buton. La momentul apăsării butonului, acesta execută o acțiune (o funcție) oferită prin parametrul `command`.

Cod Python	Exemplu de rulare
<pre>import tkinter as tk def apasat(): print("Ai apăsât butonul!") root = tk.Tk() root.title("Button") root.geometry("300x150") btn = tk.Button(root, text="Apasă-mă", command=apasat) btn.pack(pady=20) root.mainloop()</pre>	 în consolă: <pre>Ai apăsât butonul!</pre>

Parametri utili clasei Button:

- ▶ `text="..."` – textul de pe buton;
- ▶ `command=funcție` – funcția apelată la apăsare
(fără paranteze: `command=actiune`, nu `command=actiune()`);
- ▶ `state="normal" / state="disabled"` – activat/dezactivat;
- ▶ `width=12, height=2` – dimensiune;
- ▶ `font=("Arial", 12)` – font;
- ▶ `fg="white" / bg="blue"` – culori;
- ▶ `activebackground="...", activeforeground="..."` – culori la click;
- ▶ `padx=10, pady=5` – spațiere internă;
- ▶ `relief="raised" / "sunken" / "flat" / "groove" / "ridge"` – stilul marginii.

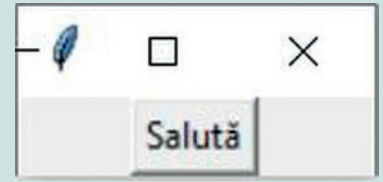
Cod Python (buton care se dezactivează după click)	Exemplu de rulare
<pre>import tkinter as tk def actiune(): btn.config(text="Gata!", state="disabled") root = tk.Tk() btn = tk.Button(root, text="Start", command=actiune) btn.pack(padx=10, pady=10) root.mainloop()</pre>	

Cod Python (buton cu parametri / argumente)

```
import tkinter as tk

def salut(ume):
    print(f"Salut, {ume}!")

root = tk.Tk()
tk.Button(root, text="Salută", command=lambda:
salut("Ana")).pack() #afisare in consola: „Salut, Ana!”
root.mainloop()
```



- Metode utile clasei Button:**
- ▶ `button.config(text="Nou")` – schimbă textul butonului
 - ▶ `button.config(state=tk.DISABLED)` – dezactivează butonul
 - ▶ `button.config(state=tk.NORMAL)` – activează butonul
 - ▶ `button.invoke()` – simulează apăsarea butonului
 - ▶ `button["text"]` – citește textul butonului
 - ▶ `button.pack()` / `button.grid()` / `button.place()` – afișează butonul în fereastră.

Metodele `pack()`, `grid()` și `place()` vor fi detaliate în secțiunea 3.2.4.

3.2.3.3. Entry – câmp pentru o singură linie de text

În Tkinter, clasa `Entry` este componenta grafică pentru introducerea textului pe o singură linie (de exemplu: nume de utilizator, căutare, valoare numerică).

Cod Python

```
import tkinter as tk

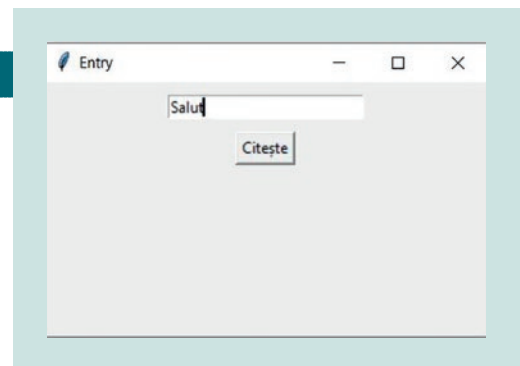
root = tk.Tk()
root.title("Entry")
root.geometry("350x200")

entry = tk.Entry(root, width=25)
entry.pack(pady=10)

def afiseaza_text():
    text = entry.get()
    print("Ai scris:", text) #Ai scris: Salut

btn = tk.Button(root, text="Citește", command=afiseaza_text)
btn.pack()

root.mainloop()
```



- Parametri utili clasei Entry:**
- ▶ `width=30` – lățime (în caractere);
 - ▶ `font=("Arial", 12)` – font;
 - ▶ `fg="black", bg="white"` – culori;
 - ▶ `show="*"` – ascunde caracterele (parolă);
 - ▶ `justify="left" / "center" / "right"` – aliniere text;
 - ▶ `state="normal" / "disabled" / "readonly"` – stare;
 - ▶ `textvariable=StringVar()` – legare la o variabilă Tk (util pentru update ușor).
- Metode utile clasei Entry:**
- ▶ `entry.get()` – citește ce a scris utilizatorul;
 - ▶ `entry.delete(0, tk.END)` – șterge textul;
 - ▶ `entry.insert(0, "text")` – pune text în câmp.

3.2.3.4. Text – zonă pentru text pe mai multe linii

În Tkinter, clasa `Text` este componenta grafică pentru text pe mai multe linii (note, editor simplu, log, descrieri). Spre deosebire de clasa `Entry`, poate conține paragrafe și permite scroll, selecții, formatare pe „tag”-uri etc.

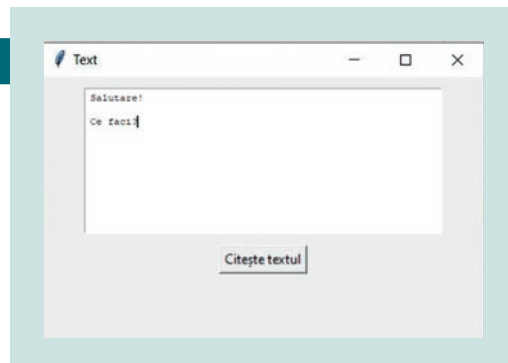
Cod Python

```
import tkinter as tk
root = tk.Tk()
root.title("Text")
root.geometry("400x250")

txt = tk.Text(root, width=40, height=8)
txt.pack(pady=10)

def citeste():
    continut = txt.get("1.0", tk.END)
    print(continut)

btn = tk.Button(root, text="Citește textul", command=citeste)
btn.pack()
root.mainloop()
```



- Parametri utili clasei Text:**
- ▶ `width`, `height` – dimensiune (în caractere/linii);
 - ▶ `wrap="word"` – split pe cuvinte ("`char`" sau "`None`");
 - ▶ `font=("Arial", 12)` – font;
 - ▶ `fg`, `bg` – culori;
 - ▶ `state="normal"` / "`disabled`" – editabil / doar afișare (util ca log).

3.2.3.5. Frame – cutie pentru organizarea elementelor

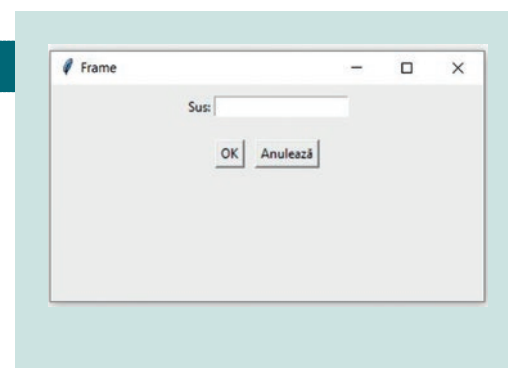
În Tkinter, clasa `Frame` reprezintă un container (o cutie) în care se pot adăuga alte componente grafice pentru a fi grupate și aranjate mai ușor. Această componentă grafică nu afișează informație către utilizator, ci ajută la organizarea mai bună a elementelor în cadrul interfeței dezvoltate.

Cod Python

```
import tkinter as tk
root = tk.Tk()
root.title("Frame")
root.geometry("400x200")

sus = tk.Frame(root)
sus.pack(pady=10)
jos = tk.Frame(root)
jos.pack(pady=10)

tk.Label(sus, text="Sus:").pack(side="left")
tk.Entry(sus).pack(side="left")
tk.Button(jos, text="OK").pack(side="left", padx=5)
tk.Button(jos, text="Anulează").pack(side="left", padx=5)
root.mainloop()
```



În programul implementat, `sus.pack(pady=10)` poziționează acest `Frame` în fereastră folosind managerul de plasare `pack()` și adaugă un spațiu vertical de 10 `pixeli` (`pady=10`) între el și celelalte elemente.



Parametri utili clasei `Frame`:

- ▶ `bg="..."` – fundal pentru layout;
- ▶ `padx, pady` – spațiere internă;
- ▶ `bd=2` – grosime margine (border);
- ▶ `relief="sunken" / "raised" / "groove" / "ridge" / "flat"` – stil margine;
- ▶ `width, height` – dimensiuni (atenție: se ajustează după conținutul care se află în cadrul frame-ului).

Cod Python (Frame + grid - rânduri/coloane)

```
import tkinter as tk
root = tk.Tk()
f = tk.Frame(root, padx=10, pady=10)
f.pack()

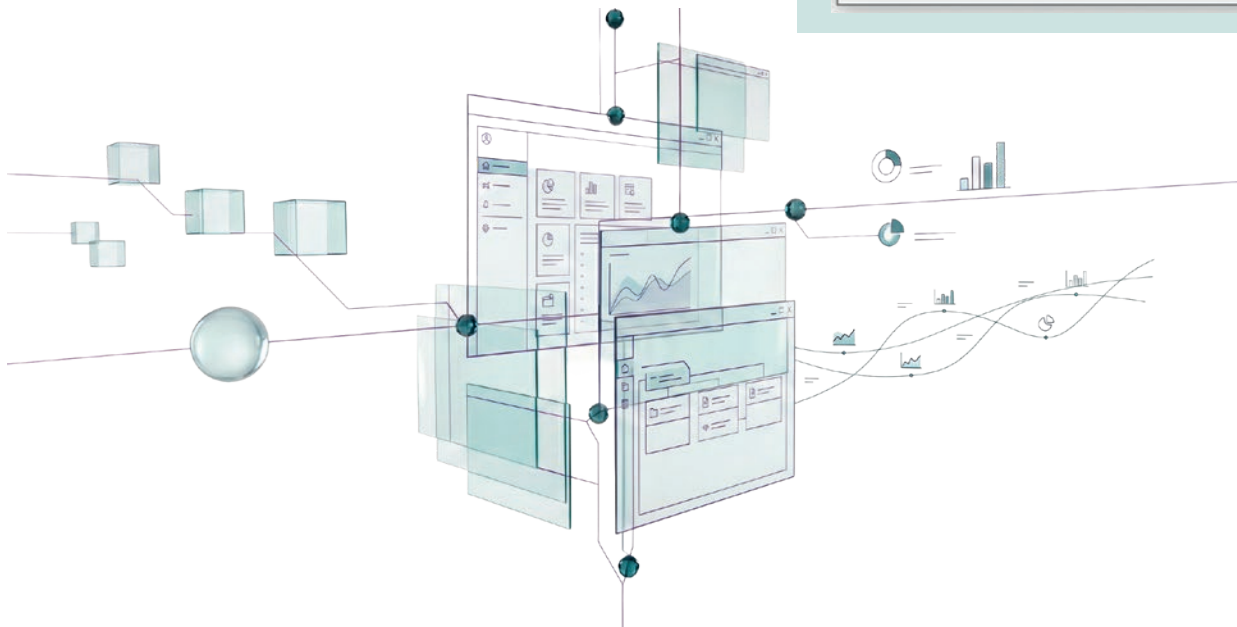
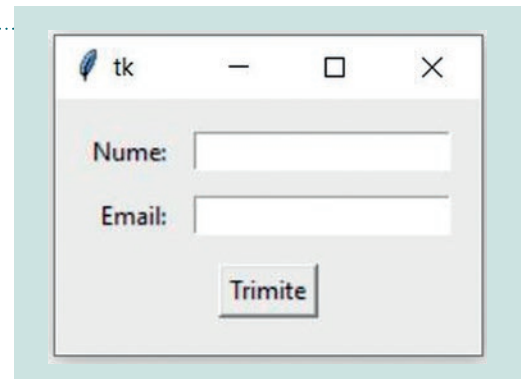
tk.Label(f, text="Nume:").grid(row=0, column=0, sticky="e", padx=5, pady=5)
e1 = tk.Entry(f)
e1.grid(row=0, column=1, padx=5, pady=5)

tk.Label(f, text="Email:").grid(row=1, column=0, sticky="e", padx=5, pady=5)
e2 = tk.Entry(f)
e2.grid(row=1, column=1, padx=5, pady=5)

tk.Button(f, text="Trimite").grid(row=2, column=0, columnspan=2, pady=8)

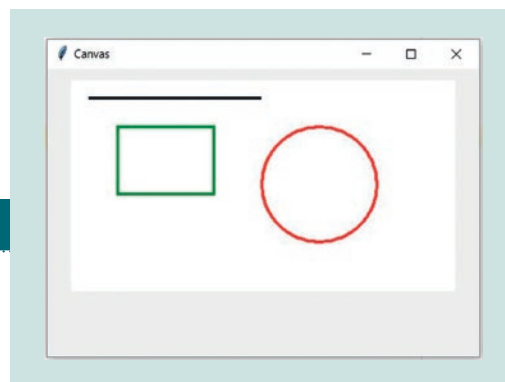
root.mainloop()
```

În programul implementat, `e1.grid(row=0, column=1, padx=5, pady=5)` plasează componenta grafică `e1` într-o grilă (tabel) din interfață. `row=0` îl pune pe primul rând, iar `column=1` pe a doua coloană. `padx=5` adaugă spațiu de 5 `pixeli` pe orizontală, iar `pady=5` adaugă spațiu de 5 `pixeli` pe verticală, pentru a nu fi lipit de alte elemente.



3.2.3.6. Canvas – desen (linii, cercuri, dreptunghiuri)

În Tkinter, clasa `Canvas` este componenta grafică utilizată pentru desen și afișarea elementelor grafice: linii, cercuri, ovale/elipse, dreptunghiuri, poligoane, text (și chiar imagini). `Canvas` este foarte util pentru crearea diagramelor, jocurilor simple, vizualizărilor.



Cod Python

```
import tkinter as tk

root = tk.Tk()
root.title("Canvas")
root.geometry("450x300")

c = tk.Canvas(root, width=400, height=220, bg="white")
c.pack(pady=10)

c.create_line(20, 20, 200, 20, width=3)
c.create_rectangle(50, 50, 150, 120, outline="green", width=3)
c.create_oval(200, 50, 320, 170, outline="red", width=3)

root.mainloop()
```

Parametri utili clasei Canvas:

- ▶ `width`, `height` – dimensiuni;
- ▶ `bg` – fundal;
- ▶ `highlightthickness=0` – scoate conturul de focus.

Parametri utili clasei Canvas pentru obiectele desenate:

- ▶ `fill="..."` – culoare umplere (pentru forme);
- ▶ `outline="..."` – culoare contur;
- ▶ `width=...` – grosime contur/linie;
- ▶ `dash=(4, 2)` – linie punctată;
- ▶ `arrow="last"` – săgeată la capăt (pentru `create_line`).

Cod Python (Evenimente (click/drag) pe Canvas)

```
import tkinter as tk

def click(event):
    # desenează un cerc unde utilizatorul a dat click
    r = 6
    c.create_oval(event.x-r, event.y-r, event.x+r, event.y+r, fill="black")

root = tk.Tk()
c = tk.Canvas(root, width=400, height=300, bg="white")
c.pack()
c.bind("<Button-1>", click)

root.mainloop()
```



3.2.3.7. MessageBox – mesaje (ferestre de alertă)

În Tkinter, cu ajutorul modului `messagebox` (din `tkinter.messagebox`) se pot crea ferestre de alertă sau de confirmare: informații, avertismente, erori, întrebări (Yes/No, Ok/Cancel).

Tipuri uzuale:

- ▶ `messagebox.showinfo(title, message)` – informare;
- ▶ `messagebox.showwarning(title, message)` – avertizare;
- ▶ `messagebox.showerror(title, message)` – eroare;
- ▶ `messagebox.askquestion(title, message)` – returnează "yes" / "no";
- ▶ `messagebox.askyesno(title, message)` – returnează True / False;
- ▶ `messagebox.askokcancel(title, message)` – True / False;
- ▶ `messagebox.askretrycancel(title, message)` – True / False.

Cod Python (Exemplu complet (info + confirmare))

```
import tkinter as tk
from tkinter import messagebox

def arata_info():
    messagebox.showinfo("Info", "Operația s-a terminat cu succes!")

def intreaba():
    ok = messagebox.askyesno("Confirmare", "Ești sigur că vrei să continui?")
    if ok:
        messagebox.showinfo("OK", "Continuăm...")
    else:
        messagebox.showwarning("Anulat", "Ai ales să oprești.")

root = tk.Tk()
tk.Button(root, text="Info", command=arata_info).pack(padx=10, pady=5)
tk.Button(root, text="Confirmare", command=intreaba).pack(padx=10, pady=5)
root.mainloop()
```

3.2.3.8. filedialog – ferestre pentru deschiderea și salvarea fișierelor

În Tkinter, modulul `filedialog` este folosit pentru afișarea ferestrelor standard de selectare a fișierelor și folderelor. Aceasta permite afișarea ferestrelor standard de selectare a fișierelor și folderelor. Acesta permite utilizatorului să deschidă, să salveze sau să aleagă directoare într-o aplicație grafică.

Cod Python

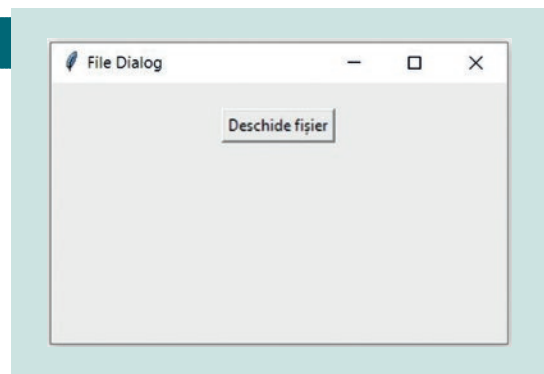
```
import tkinter as tk
from tkinter import filedialog

root = tk.Tk()
root.title("File Dialog")
root.geometry("350x200")

def deschide():
    fisier = filedialog.askopenfilename()
    print("Fișier selectat:", fisier)

btn = tk.Button(root, text="Deschide fișier", command=deschide)
btn.pack(pady=20)

root.mainloop()
```



**Funcții utile
din modulul
filedialog:**

- ▶ `filedialog.askopenfilename()` – selectează un singur fișier;
- ▶ `filedialog.askopenfilenames()` – selectează mai multe fișiere;
- ▶ `filedialog.asksaveasfilename()` – alege locația pentru salvare;
- ▶ `filedialog.askdirectory()` – selectează un folder.

**Parametri utili
din modulul
filedialog:**

- ▶ `title="Titlu"` – setează titlul ferestrei;
- ▶ `initialdir="/"` – folderul inițial;
- ▶ `filetypes=[("Text files", "*.txt")]` – filtrează tipurile de fișiere;
- ▶ `defaulttextextension=".txt"` – extensia implicită la salvare.

3.2.4. Organizarea elementelor în fereastră: pack, grid, place

3.2.4.1. Pack()

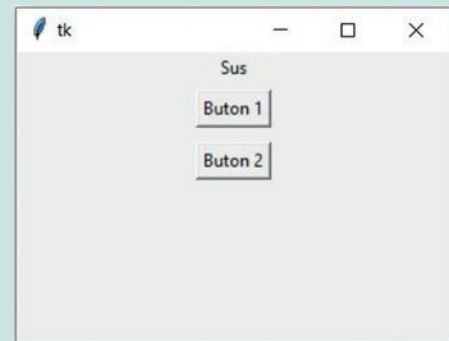
În Tkinter, `pack()` este un sistem de aranjare rapid: așază componentele grafice unul după altul într-un container (`root`, `Frame`), pe verticală (implicit) sau pe orizontală.

Cod Python

```
import tkinter as tk
root = tk.Tk()
root.geometry("300x200")

tk.Label(root, text="Sus").pack()
tk.Button(root, text="Buton 1").pack(pady=5)
tk.Button(root, text="Buton 2").pack(pady=5)

root.mainloop()
```

**Parametri utili:**

- ▶ `padx=10`, `pady=10` – spațiu în jurul componentei grafice;
- ▶ `fill="x" / "y" / "both"` – umple pe axa X/Y sau pe ambele axe;
- ▶ `expand=True` – permite să ocupe spațiul liber;
- ▶ `anchor="w"` – aliniază (lipește) elementele spre o anumită direcție în container (w, e, n, s, center – adică vest, est, nord, sud, centru).

Cod Python

```
import tkinter as tk

root = tk.Tk()

top = tk.Frame(root, bd=2, relief="groove")
top.pack(fill="x")

tk.Label(top, text="Header").pack(side="left", padx=8, pady=6)
tk.Button(top, text="Setări").pack(side="right", padx=8, pady=6)

main = tk.Text(root)
main.pack(fill="both", expand=True, padx=8, pady=8)

root.mainloop()
```

3.2.4.2. Grid() – tabel (rânduri/coloane)

În Tkinter, `grid()` este un sistem de aranjare de tip tabel: se adaugă componente grafice pe rânduri și coloane. Acest tip de aranjare este foarte folosit pentru formulare (`Label` + `Entry`), panouri.

Cod Python

```
import tkinter as tk

root = tk.Tk()
root.title("grid")
root.geometry("350x150")

tk.Label(root, text="Nume:").grid(row=0, column=0, padx=5, pady=5, sticky="e")
e1 = tk.Entry(root)
e1.grid(row=0, column=1, padx=5, pady=5)

tk.Label(root, text="Clasa:").grid(row=1, column=0, padx=5, pady=5, sticky="e")
e2 = tk.Entry(root)
e2.grid(row=1, column=1, padx=5, pady=5)

tk.Button(root, text="Trimite").grid(row=2, column=0, columnspan=2, pady=10)

root.mainloop()
```

Parametri utili:

- ▶ `row=...`, `column=...` – poziția în tabel;
- ▶ `padx=...`, `pady=...` – spațiu în jur;
- ▶ `sticky="w/e/n/s"` – „lipește” componenta grafică de margini (aliniere+întindere);
- ▶ `sticky="w"` = aliniat la stânga;
- ▶ `sticky="e"` = la dreapta;
- ▶ `sticky="we"` = se întinde pe orizontală;
- ▶ `columnspan` / `rowspan` (ocupă mai multe celule);
- ▶ `columnspan=2`: ocupă 2 coloane;
- ▶ `rowspan=2`: ocupă 2 rânduri.



IMPORTANT

Nu amesteca `pack()` și `grid()` în **același container** (același `Frame` / `root`). Dar poți folosi `pack` într-un `Frame` și `grid` în alt `Frame`.



IMPORTANT

Extindere la redimensionare (foarte important): Ca să se redimensioneze uniform când se mărește fereastra, se setează `weight` pe rând/coloană și se folosește `sticky="nsew"`.

3.2.4.3. Place() – coordonate exacte

În Tkinter, `place()` se utilizează pentru poziționarea componentelor grafice folosind coordonate exacte (x, y) sau proporții relative, însă este mai sensibil la redimensionarea ferestrei comparativ cu `pack()` sau `grid()`.

Cod Python

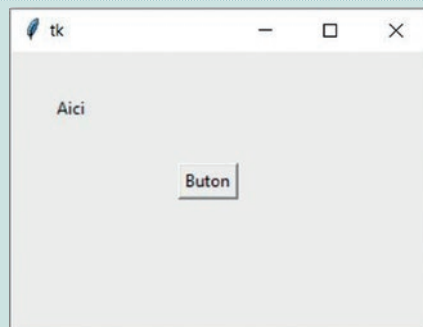
```
import tkinter as tk

root = tk.Tk()
root.geometry("300x200")

tk.Label(root, text="Aici").place(x=30, y=30)
tk.Button(root, text="Buton").place(x=120, y=80)

root.mainloop()
```

Exemplu de rulare



Parametri utili:

- ▶ `x`, `y` – poziție absolută (pixeli);
- ▶ `width`, `height` – dimensiune absolută (pixeli);
- ▶ `relx`, `rely` – poziție relativă (0.0 – 1.0) din lățime/înălțime;
- ▶ `relwidth`, `relheight` – dimensiuni relative (0.0 – 1.0);
- ▶ `anchor` – punct de referință ("nw" implicit, "center", "ne").

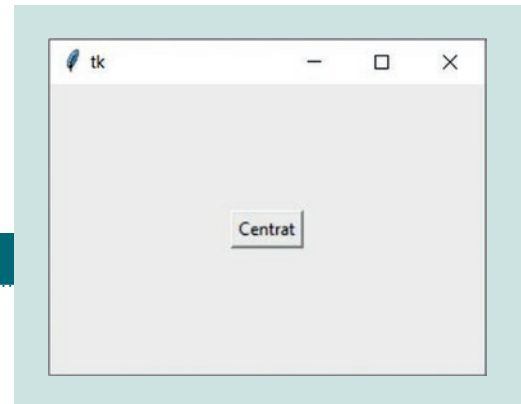
Cod Python (centrare în fereastră)

```
import tkinter as tk

root = tk.Tk()
root.geometry("300x200")

tk.Button(root, text="Centrat").place(relx=0.5, rely=0.5, anchor="center")

root.mainloop()
```

**Când este recomandată utilizarea place()**

- ▶ interfață grafică simplă, cu poziții fixe;
- ▶ overlay/etichete peste un `Canvas` sau o imagine;
- ▶ layouturi „custom” unde `grid/pack` nu ajută.

**IMPORTANT**

Dacă utilizatorul redimensionează fereastra, poziționarea absolută poate duce la un aspect neplăcut (schimbarea poziției elementelor), de aceea `grid()` și `pack()` sunt, de regulă, preferate.

3.2.5. Variabile Tkinter: StringVar, IntVar

În Tkinter, `StringVar`, `IntVar` (și `DoubleVar`, `BooleanVar`) sunt variabile „speciale” care permit legarea (binding) între date și componente grafice. Este important să reținem faptul că, legând o variabilă de o componentă grafică, interfața grafică se actualizează automat atunci când valoarea variabilei se schimbă.

Variabilele `StringVar` și `IntVar` sunt des întâlnite atunci când definim:

- ▶ `Entry` / `Label` (text);
- ▶ `Checkbutton` (de obicei `BooleanVar` / `IntVar`);
- ▶ `Radiobutton` (`StringVar` / `IntVar`);
- ▶ `OptionMenu` / `ttk.Combobox` (de obicei `StringVar`).

Cod Python (creare + citire/setare)

```
import tkinter as tk

root = tk.Tk()

s = tk.StringVar(value="Salut")
i = tk.IntVar(value=0)

print(s.get())    # "Salut"
s.set("Alt text")
print(i.get())    # 0
i.set(10)
root.mainloop()
```

Cod Python (Label + Entry cu textvariable)

```
import tkinter as tk
root = tk.Tk()
name = tk.StringVar(value="Ana")
tk.Entry(root, textvariable=name).pack(padx=10, pady=5)
tk.Label(root, textvariable=name).pack(padx=10, pady=5)
root.mainloop()
```

Cod Python (IntVar (de ex. numărător) + butoane)

```
import tkinter as tk

root = tk.Tk()

cnt = tk.IntVar(value=0)

tk.Label(root, textvariable=cnt, font=("Arial", 16)).pack(pady=5)
tk.Button(root, text="+1", command=lambda: cnt.set(cnt.get() + 1)).pack()

root.mainloop()
```

3.2.6. Evenimente: reacții la taste/click (bind)

În Tkinter, evenimentele sunt acțiuni ale utilizatorului (click, mișcare mouse, taste). Funcția `bind()` leagă o componentă grafică de un eveniment, astfel încât o altă funcție să fie apelată automat când evenimentul are loc.

Evenimente uzuale mouse:	▶ "<Button-1>" – click stânga; ▶ "<Button-3>" – click dreapta (pe macOS poate diferi); ▶ "<Double-Button-1>" – dublu click; ▶ "<Motion>" – mișcare mouse; ▶ "<B1-Motion>" – drag cu butonul stâng apăsat; ▶ "<Enter>" / "<Leave>" – trecerea mouse-ului peste (hover) o componentă grafică.
-------------------------------------	---

Evenimente uzuale tastatură:	▶ "<KeyPress>" sau "<Key>" – apăsarea oricărei taste; ▶ "<Return>" – apăsarea tastei Enter; ▶ "<Escape>" – apăsarea tastei Esc; ▶ "<BackSpace>", "<Tab>", "<space>"; ▶ "<Control-s>", "<Alt-F4>" – combinații.
---	--



IMPORTANT

<Enter> se referă la mouse, NU la tasta Enter – pentru aceasta se folosește <Return>.

Cod Python

```
import tkinter as tk

root = tk.Tk()
root.geometry("300x150")
lbl = tk.Label(root, text="Apasă o tastă...")
lbl.pack(pady=30)

def la_tasta(event):
    lbl.config(text=f"Ai apăsat: {event.keysym}")

root.bind("<KeyPress>", la_tasta)
root.mainloop()
```

Cod Python (click pe Canvas (desenează un punct))

```
import tkinter as tk
def click(event):
    r = 5
    c.create_oval(event.x-r, event.y-r, event.x+r, event.y+r, fill="black")
root = tk.Tk()
c = tk.Canvas(root, width=300, height=200, bg="white")
c.pack()
c.bind("<Button-1>", click)

root.mainloop()
```

Cod Python (taste (update la Label))

```
import tkinter as tk
def key(event):
    lbl.config(text=f"Tastă: {event.keysym}")

root = tk.Tk()
lbl = tk.Label(root, text="Apasă o tastă")
lbl.pack(padx=10, pady=10)

root.bind("<Key>", key) # ascultă pe fereastră
root.focus_set()       # ca să primească taste
root.mainloop()
```

3.2.7. Probleme rezolvate

PROBLEMA 1: „Salut, utilizatorule!”

Enunț: Creează o interfață grafică care permite utilizatorului să introducă numele într-un câmp de text și să primească un mesaj de salut personalizat atunci când apasă un buton. Dacă nu introduce niciun nume și încearcă să trimită, se afișează o avertizare care îl îndeamnă să completeze câmpul.

Cod Python

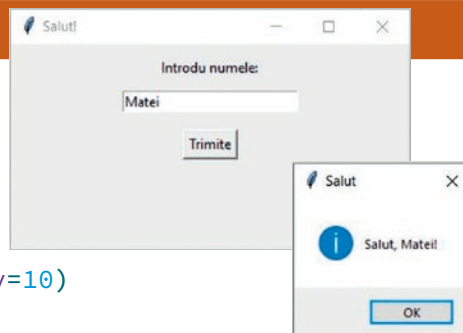
```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.title("Salut!")
root.geometry("350x180")

tk.Label(root, text="Introdu numele:").pack(pady=10)
entry = tk.Entry(root, width=25)
entry.pack()

def saluta():
    # Preia textul introdus în câmpul Entry și elimină spațiile de la început și sfârșit
    nume = entry.get().strip()
    if not nume:
        messagebox.showwarning("Atenție", "Te rog scrie un nume!")
        return
    messagebox.showinfo("Salut", f"Salut, {nume}!")

tk.Button(root, text="Trimite", command=saluta).pack(pady=15)
root.mainloop()
```





PROBLEMA 2: Minicalculator (adunare/scădere)

Enunț: Creează o interfață grafică de tip minicalculator care permite utilizatorului să introducă două numere. După ce apeși butoanele „Adună” sau „Scade”, se afișează rezultatul operației. Dacă unul dintre câmpuri nu conține un număr valid, se afișează un mesaj de eroare care solicită introducerea unor valori corecte.

Cod Python

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.title("Minicalculator")
root.geometry("350x200")

tk.Label(root, text="Număr 1:").grid(row=0, column=0, padx=5, pady=5,
sticky="e")
e1 = tk.Entry(root)
e1.grid(row=0, column=1, padx=5, pady=5)

tk.Label(root, text="Număr 2:").grid(row=1, column=0, padx=5, pady=5,
sticky="e")
e2 = tk.Entry(root)
e2.grid(row=1, column=1, padx=5, pady=5)

rez = tk.Label(root, text="Rezultat: -", font=("Arial", 12))
rez.grid(row=2, column=0, columnspan=2, pady=10)

# În funcția calc(op), try/except este folosit pentru a verifica dacă
# utilizatorul a introdus numere valide înainte de a face calculele necesare.
# Dacă utilizatorul introduce litere sau alte caractere nepermise,
# apare o fereastră cu mesajul de eroare și funcția se oprește cu return.

def calc(op):
    try:
        a = float(e1.get())
        b = float(e2.get())
    except ValueError:
        messagebox.showerror("Eroare", "Introdu numere valide!")
        return

    r = a + b if op == "+" else a - b
    rez.config(text=f"Rezultat: {r}")

tk.Button(root, text="Adună", command=lambda: calc("+")).grid(row=3,
column=0, pady=5)
tk.Button(root, text="Scade", command=lambda: calc("-")).grid(row=3,
column=1, pady=5)

root.mainloop()
```



PROBLEMA 3: Notițe simple (Text + salvare într-un fișier)

Enunț: Creează o interfață grafică pentru notițe care permite utilizatorului să scrie text într-o zonă dedicată și să salveze conținutul într-un fișier text. Dacă zona de text este goală, se afișează o avertizare, iar dacă utilizatorul nu selectează o cale pentru salvare, operația se anulează. La final, după salvare, se afișează un mesaj de confirmare.

Cod Python

```
import tkinter as tk
from tkinter import messagebox, filedialog
root = tk.Tk()
root.title("Notițe")
root.geometry("500x350")

txt = tk.Text(root, width=60, height=15)
txt.pack(padx=10, pady=10)

def salveaza():
    continut = txt.get("1.0", tk.END).rstrip()
    if not continut:
        messagebox.showwarning("Atenție", "Nu ai scris nimic!")
        return

    cale = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Fișiere text", "*.txt")]
    )
    if not cale:
        return

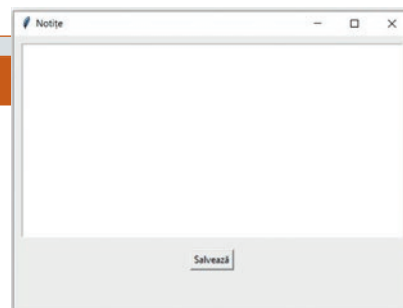
    with open(cale, "w", encoding="utf-8") as f:
        f.write(continut)
    messagebox.showinfo("OK", "Fișier salvat!")
    tk.Button(root, text="Salvează", command=salveaza).pack(pady=5)
root.mainloop()
```

Exemplu de rulare

Programul prezintă o aplicație cu interfață grafică (GUI) în Tkinter și funcționează pe bază de evenimente. Fluxul aplicației:

- ▶ **Interfață:** creează o fereastră (`root`) cu o zonă de text și un buton.
- ▶ **Interacțiune:** utilizatorul scrie text și apasă butonul „Salvează”.
- ▶ **Logică:** funcția verifică dacă există text, apoi deschide un dialog pentru alegerea locației.
- ▶ **Fișiere:** textul este scris într-un fișier `.txt` pe disc.
- ▶ **Control aplicație:** `mainloop()` menține fereastra activă și așteaptă acțiuni ale utilizatorului.

Ideea generală a unei aplicații cu interfață grafică: utilizatorul inițiază acțiuni în cadrul interfeței, aplicația răspunde în mod corespunzător la aceste evenimente, iar datele sunt ulterior procesate și persistate.





PROBLEMA 4: Calculul dublului numerelor (Interfață grafică + salvare în fișier)

Enunț: Creează o interfață grafică care permite utilizatorului să introducă cinci numere întregi în câmpuri separate. După ce apeși butonul „Calculează și salvează”, aplicația calculează dublul fiecărui număr și îl salvează într-un fișier text. Dacă unul dintre câmpuri nu conține un număr întreg, se afișează un mesaj de eroare, iar dacă utilizatorul nu alege o cale pentru fișier, operația de salvare este anulată. La final, se afișează un mesaj de confirmare a salvării.

Cod Python

```
import tkinter as tk
from tkinter import messagebox, filedialog

root = tk.Tk()
root.title("Dublul numerelor")
root.geometry("350x300")

entries = []

# 5 câmpuri pentru introducerea numerelor
for i in range(5):
    tk.Label(root, text=f"Numărul {i+1}:").pack()
    e = tk.Entry(root)
    e.pack(pady=3)
    entries.append(e)

def calc_salv():
    try:
        numere = [int(e.get()) for e in entries]
    except ValueError:
        messagebox.showerror("Eroare", "Introduceți doar numere întregi!")
        return

    dubluri = [2 * n for n in numere]

    cale = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Fișiere text", "*.txt")]
    )

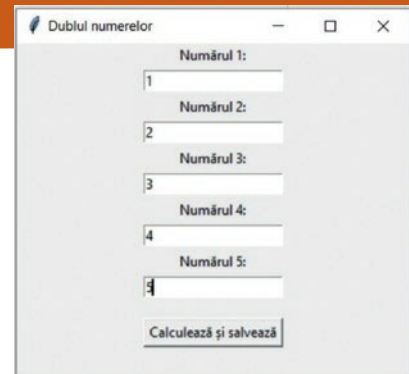
    if not cale:
        return

    with open(cale, "w", encoding="utf-8") as f:
        for n, d in zip(numere, dubluri):
            f.write(f"Număr: {n} -> Dublu: {d}\n")

    messagebox.showinfo("Succes", "Dublul numerelor a fost salvat în fișier!")

tk.Button(root, text="Calculează și salvează", command=calc_salv).pack(pady=15)

root.mainloop()
```





FIȘĂ DE LUCRU 6

1. Rezolvă următoarele cerințe de realizare a unor aplicații Tkinter.
 - a. Creează o aplicație Tkinter care afișează o fereastră cu titlul „Prima aplicație Tkinter” și un `Label` cu textul „Bine ai venit!”.
 - b. Creează o aplicație cu un buton „Apasă-mă”. La apăsarea butonului se afișează în consolă mesajul „Salut!”.
 - c. Creează o aplicație cu un `Entry` pentru introducerea unui text și un buton „Afișează”. La apăsarea butonului, textul introdus va fi afișat într-un `Label`.
 - d. Creează o aplicație cu un `Entry` și un buton „Șterge textul”. La apăsarea butonului, conținutul câmpului va fi șters folosind metoda `delete()`.
 - e. Creează o aplicație în care utilizatorul introduce numele într-un `Entry`, iar la apăsarea unui buton:
 - se afișează mesajul „Salut, NUME!” într-un `messagebox`;
 - dacă nu este introdus nimic, se afișează un mesaj de avertizare.

2. Creează o aplicație cu două `Frame`-uri:
 - Frame superior: un `Label` și un `Entry`;
 - Frame inferior: două butoane „OK” și „Anulează”.

Componentele vor fi organizate folosind `pack()`.

3. Se consideră o aplicație cu un `Canvas`. La fiecare click al mouse-ului, aplicația va desena un cerc mic în locul unde s-a făcut click.

4. Creează un formular utilizând

`grid()`: `Label` + `Entry` pentru Nume, `Label` + `Entry` pentru Email, `Buton` „Trimite”. La apăsarea butonului:

- a) se verifică dacă ambele câmpuri sunt completate;
- b) se afișează mesajul „Date salvate!” într-un `messagebox`.

5. **Aplicație – Agenda clasei.** Creează o aplicație grafică în care utilizatorul introduce: numele elevului, clasa, o observație scurtă. Datele introduse vor fi afișate într-o zonă `Text`.
Extindere: adaugă un buton pentru ștergerea câmpurilor și un mesaj de confirmare la salvare.

6. **Aplicație – Caiet de notițe digitale.**

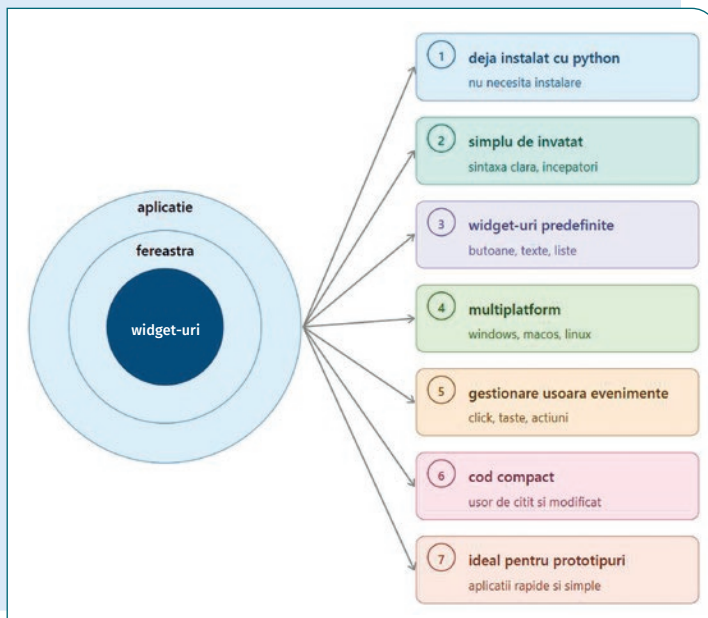
Creează o aplicație în care utilizatorul: introduce titlul notiței (`Entry`), scrie conținutul (`Text`), salvează notița într-un fișier .txt.

Extindere: adaugă butoane „Șterge” și „Salvează” și un mesaj de confirmare după salvare.

7. **Aplicație – Desen simplu.**

Creează o aplicație cu `Canvas` în care utilizatorul poate desena puncte sau cercuri mici prin click.

Extindere: adaugă butonul „Șterge desenul”, `Label` care afișează numărul de click-uri realizate.



RECAPITULARE

3.3. Recapitulare

1. În fișierul text `date.txt` se află mai multe numere întregi, câte unul pe rând. Scrie un program în Python care determină cel mai mare produs de două numere întregi consecutive din fișier și afișează rezultatul pe ecran.
2. Construiește un program în limbajul Python care să genereze 30 de numere întregi aleatoare din intervalul $[-50, 50]$ și scrie aceste numere în fișierul `date1.txt`, câte unul pe rând. Copiază valorile negative din fișierul `date1.txt` în fișierul `date2.txt`, pe prima linie despărțite printr-un spațiu. În cazul în care, în fișierul `date1.txt` nu există valori negative, atunci se va scrie în fișierul `date2.txt` mesajul `Nu exista`.
3. În fișierul text `date.txt`, pe linii diferite, se află două numere naturale în ordine strict crescătoare. Scrie un program care citește datele din fișierul `date.txt` și afișează pe ecran toate numerele naturale pare cuprinse între cele două numere din fișierul `date.txt`, câte 5 numere pe un rând (eventual, ultimul rând are mai puțin de 5 numere).

MINIPROIECTE – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVI

1. Minicalculator.

- a. Creează o interfață grafică care permite utilizatorului să introducă două numere reale și care calculează suma, diferența, produsul și împărțirea lor, afișând rezultatul prin utilizarea unei componente de tip `Label`. Afișează un mesaj de eroare dacă utilizatorul introduce date invalide sau încearcă împărțirea la zero.
- b. Compară această aplicație cu interfață grafică cu varianta sa cu interfață text, evidențiind punctele forte și limitările fiecăreia.

2. Pătratul numerelor.

Creează o interfață grafică care permite utilizatorului să introducă cinci numere întregi în câmpuri separate. După apăsarea butonului „Calculează pătratele și salvează”, aplicația calculează pătratul fiecărui număr și îl salvează într-un fișier text. Dacă unul dintre câmpuri nu conține un număr întreg, se afișează un mesaj de eroare și operația nu se execută. La final, dacă operația a fost realizată cu succes, se afișează un mesaj de confirmare a salvării.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

R E C A P I T U L A R E

3.4. Evaluare sumativă

V
A
L
U
A
R
E

I. Rezolvă.

1. Stabilește valoarea de adevăr a următoarelor enunțuri și justifică răspunsul. 5x0,3 p. = 1,5 p.
- Etapele de lucru cu un fișier text se execută în ordinea: deschidere, prelucrare, închidere.
 - Metoda `read()` citește primul rând dintr-un fișier text și returnează un șir de caractere.
 - Instrucțiunea `with open("date.txt", "r") as f:` necesită apelarea metodei `close()`.
 - Dacă dorești să scrii valoarea numerică `n` în fișierul `f`, instrucțiunea corectă este `f.print(n)`.
 - Instrucțiunea `f.seek(3)` mută cursorul de citire, în fișierul `f`, pe al patrulea caracter.
2. Analizează următorul scenariu: un fișier text conține cuvântul "PYTHON". Dacă se execută succesiv `f.read(2)` și apoi `f.read(3)`, ce va returna al doilea apel al metodei? 0,5 p.

a) "THO"

b) "PYT"

c) "HON"

d) "PYTHON"

3. Se consideră secvența de program de mai jos: 2x0,5 p.=1 p.

```
f = open("date.txt", "r")
linie = f.readline()
while linie != "":
    print(linie)
```

- Identifică eroarea logică din cod.
- Rescrie secvența de program astfel încât să se afișeze toate liniile din fișier.

4. De ce este considerată utilizarea instrucțiunii `'with'` o „bună practică” în lucrul cu fișierele text în Python? 0,5 p.

- Transformă automat orice fișier binar într-un fișier de tip text.
- Permite deschiderea fișierelor fără a mai specifica numele fizic al acestora.
- Garantează închiderea automată a fișierului, prevenind pierderile de date chiar și în cazul apariției erorilor.
- Accelerează viteza de scriere a datelor pe hard disk.

5. Creează o interfață grafică în care utilizatorul introduce o temperatură în grade Celsius. Utilizând două butoane „Convertește F” și, respectiv, „Convertește K”, aplicația transformă valoarea citită în grade Fahrenheit și, respectiv, în grade Kelvin (<https://en.wikipedia.org/wiki/Celsius>).

Valorile obținute vor fi afișate tot în interfața grafică. 1,5 p.

II. Miniproiect. Realizează, la alegere, unul din cele două miniproiecte rezolvate în cadrul secțiunii *Recapitulare*.

2 p.

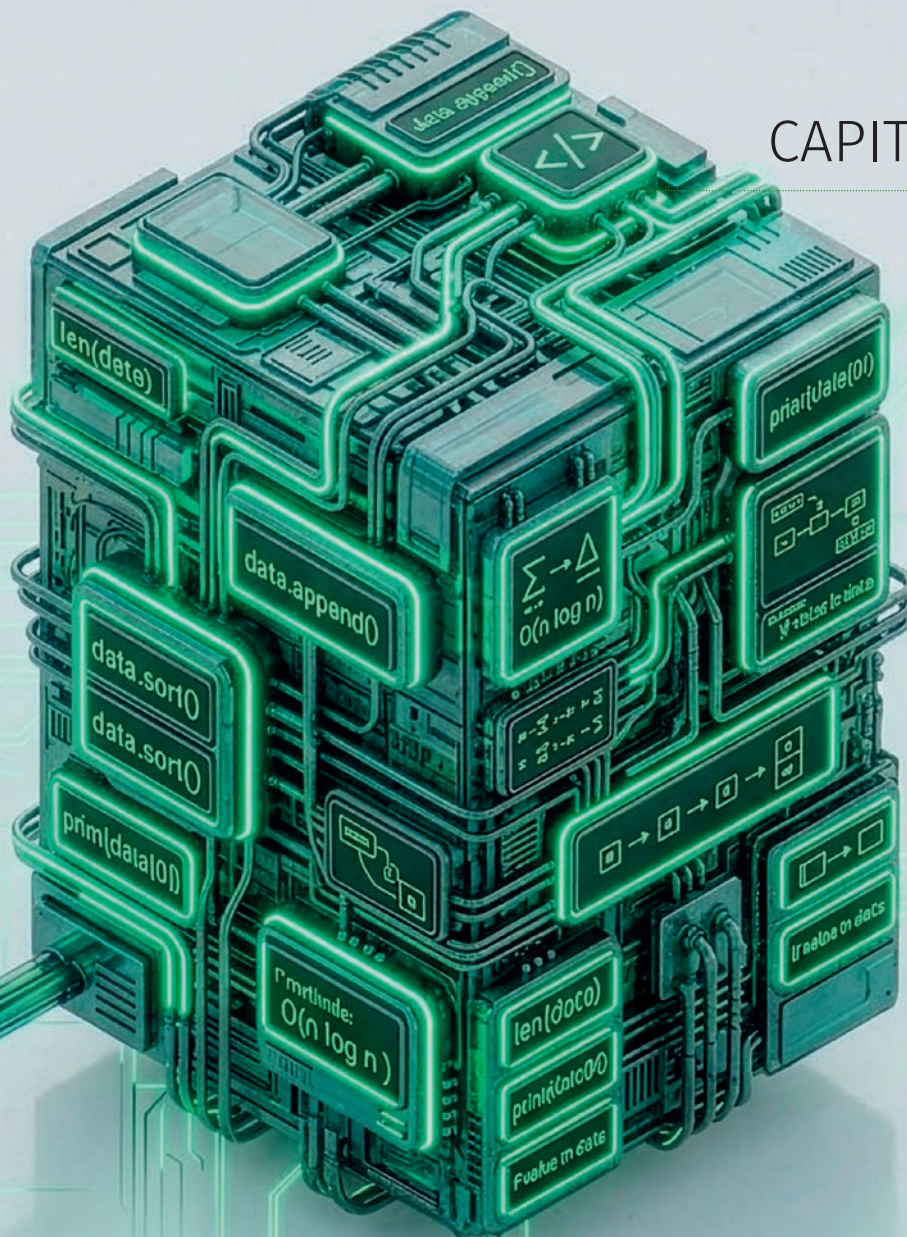
III Portofoliu. La finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul *Unitate3_NumeElev*, următoarele rezolvări: *Fișa de lucru 5* și *Fișa de lucru 6*.

1 x 2 p. = 2 p.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*.
Se acordă 1 punct din oficiu.

CAPITOLUL 4

Liste



- 4.1. Modelul conceptual liniar
- 4.2. Clasa „list” din Python
- 4.3. Metode de generare sistematică a elementelor unei liste
- 4.4. Metode de sortare

- 4.5. RECAPITULARE
- 4.6. EVALUARE SUMATIVĂ

COMPETENȚE SPECIFICE:

1.1, 1.2, 1.4, 1.5,
2.1, 2.2, 2.4, 2.5,
3.1, 3.2, 3.4, 3.5,
4.1, 4.2, 4.4, 4.5,
5.1, 5.2, 5.4, 5.5,
6.1, 6.2, 6.4, 6.5





4.1. Modelul conceptual liniar



4.1.1. Introducere

Oamenii își organizează viața de zi cu zi, fie că își dau seama sau nu, prin intermediul listelor. De la sarcini simple până la obiective importante, acestea devin instrumente esențiale pentru orientarea într-un mediu mereu schimbător, aducând ordine în dezordine și limpezime în momentele de incertitudine.

Probabil cea mai răspândită formă este **lista de cumpărături**. La prima vedere pare un simplu inventar de produse, dar reprezintă de fapt o strategie esențială de organizare și eficiență, ne reîmprospătează memoria, ne ajută, spre exemplu, să respectăm un buget. Pe lângă nevoile materiale, viețile noastre sunt ghidate și de **liste de sarcini (to-do lists)**, care ne permit să prioritizăm, să delegăm și să urmărim progresul, reduc stresul, sporesc productivitatea și ne oferă satisfacția de a bifa o sarcină îndeplinită.

Clasamentele sunt liste care ordonează elemente pe baza unor criterii specifice – cele mai bune universități, topuri muzicale, clasamentele sportive –, ne ajută să înțelegem ierarhiile, să comparăm performanțele și să ne formăm opinii. Ele sunt atât instrumente de informare, cât și de influențare a percepției publice.

Top 6 cărți recomandate adolescenților

De veghe în lanul de seară	(J. D. Salinger)
Să ucizi o pasăre cântătoare	(Harper Lee)
Împăratul muștelor	(William Golding)
Micul Prinț	(Antoine de Saint-Exupéry)
Marile speranțe	(Charles Dickens)
Jurnalul unui adolescent	(Stephen Chbosky)

Nu uităm **listele de dorințe (wishlists)**, care depășesc sfera necesarului și pătrund în cea a aspirațiilor; nu sunt doar simple colecții de obiecte, ci și imagini ale dorințelor noastre, amintindu-ne constant de ceea ce prețuim și ce ne motivează, jucând un rol important în menținerea entuziasmului și optimismului.

În concluzie, listele sunt mult mai mult decât simple înșiruri de cuvinte. Ele sunt o reflectare a modului în care mintea umană procesează informațiile, prioritizează și planifică. Fie că sunt folosite pentru nevoi practice, pentru gestionarea timpului, pentru exprimarea dorințelor sau pentru înțelegerea lumii, listele sunt instrumente indispensabile care ne ajută să navigăm eficient prin complexitatea existenței cotidiene, aducând ordine, claritate și un sentiment de realizare în fiecare aspect al vieții noastre.

Un **model conceptual** explică modul în care informațiile sunt organizate în memorie și felul în care putem lucra cu ele. Prin intermediul unui astfel de model, datele capătă structură și sens, fiind grupate, relaționate și accesate într-o manieră coerentă. Astfel, el nu doar descrie memoria, ci și facilitează utilizarea ei eficientă, reducând complexitatea și făcând sistemele mai ușor de înțeles și de utilizat.



Modele conceptuale frecvent utilizate:

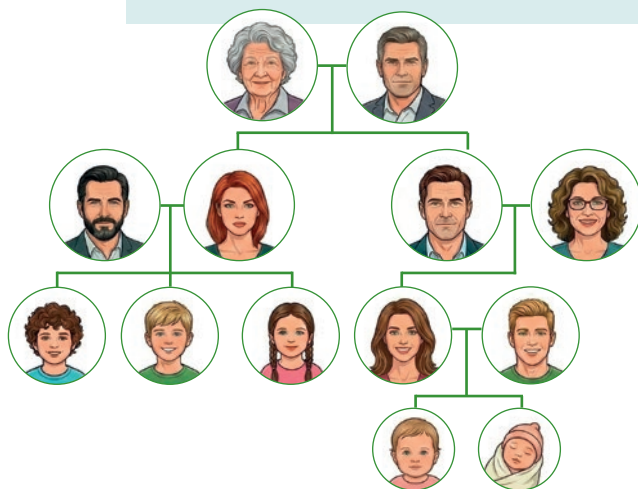
Modelul **liniar** – elementele sunt aranjate într-o secvență (liste, șiruri).

- O coadă la supermarket
- Zilele săptămânii
Lu – Ma – Mi – Jo – Vi – Sâ – Du



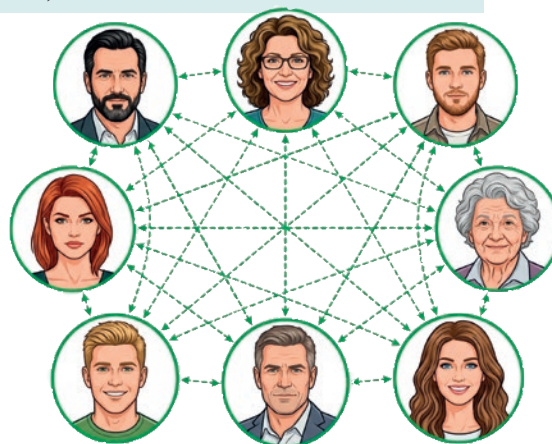
Modelul **ierarhic**

- Organigrama unei companii
- Arborele genealogic



Modelul **rețea/grafic** – noduri conectate prin legături multiple.

- Sistemul de drumuri și autostrăzi
- Rețelele de socializare



4.1.2. Lista

Lista este cea mai folosită structură **liniară** de date. Parcurgând exemplele de mai sus, observăm că:

- a) elementele unei liste sunt, aproape toate, de aceeași natură (spunem „aproape” pentru că, de exemplu, o listă de dorințe poate conține produse electronice, cărți, îmbrăcăminte, rechizite etc.);
- b) locul în care elementele sunt trecute în listă poate reprezenta un număr de ordine;
- c) lista se poate modifica prin ștergerea, modificarea sau adăugarea unor noi elemente.



Din șirul/lista celor 6 baschetbaliști pleacă jucătorul cu indicele 3, iar cei care au rămas își actualizează poziția.



Caracteristicile care definesc lista ca structură liniară de date:

- ▶ **ordonată** – elementele au o poziție fixă, numită **index**. Asta face ca lista să fie indexabilă, adică elementele să poată fi accesate prin poziție. În Python, ca și în alte limbaje de programare, **pri-mul element** al listei va fi la poziția **indexul 0**, ultimul din cele n elemente aflându-se la poziția $n-1$.
- ▶ **mutabilă** – poate fi modificată, operațiile standard pe care le putem efectua fiind:
 - **inserare** – adăugarea unui element nou;
 - **ștergere** – eliminarea unui element existent;
 - **căutare** – identificarea unui element după valoare sau index;
 - **parcurgere** – vizitarea fiecărui element exact o singură dată;
- ▶ **omogenă** – adică toate elementele sale au același tip de date (toate sunt întregi, toate sunt caractere etc.), cazul limbajului C;
- ▶ **eterogenă** – poate conține tipuri diferite de date, precum în Python.



IMPORTANT

În clasa a IX-a folosim doar liste liniare omogene pentru stocarea elementelor de tip numeric sau caracter. În literatura de specialitate sau în alte limbaje de programare, lista în care toate elementele sunt de același tip se numește vector / array, sau tablou unidimensional.



STUDIU DE CAZ 1

Alertă facturi restante

Atunci când volumul de date este uriaș, procesarea secvențială **fără memorarea** rezultatelor intermediare economisește memorie și crește viteza de calcul.

Context real: O companie de gestionare a imobilelor primește lunar un fișier cu mii de înregistrări reprezentând facturile emise către locatari. Fiecare înregistrare conține: numele locatarului, suma facturată și statusul plății (plătit/neplătit).

Obiectiv: Identificarea restanțierilor pentru a trimite notificări de plată, **fără a crea o listă secundară**.

Parcurgere liniară: Va face atâția pași cât este numărul de facturi, verificând fiecare element o singură dată. Când se găsește o factură neplătită, restanțierul este afișat imediat, fără memorarea de informații suplimentare.

Abordare: „Citește → Verifică → Acționează (Print/Alertă) → Uită”



Beneficiu: Indiferent de dimensiunea listei, programul va folosi o cantitate minimă și constantă de memorie.

Acest model de procesare este utilizat în viața reală în:

- » **sisteme bancare** – la detectarea tranzacțiilor suspecte în timp real, pe măsură ce apar;
- » **senzori de temperatură** – dacă într-un depozit temperatura depășește un prag, se declanșează alarma imediat, fără a stoca toate valorile normale precedente.



STUDIU DE CAZ 2

Sistemul de rating 1-5 Stele

Context real: Un restaurant colectează recenzii de la clienți, care pot oferi o notă de la 1 la 5. La finalul unei săptămâni, managerul are o listă cu 1 000 de răspunsuri (exemplu: [5, 4, 5, 2, 1, 5, 4, ...])

Obiectiv: „Suntem un restaurant de 5 stele sau avem probleme?”

Se introduce conceptul de **Listă de frecvențe**: o structură în care indexul (sau cheia) reprezintă varianta de răspuns: **căsuța cu indexul i reține numărul de voturi pentru i stele**.



Prima căsuță, cea cu indice 0, rămâne nefolosită, dar „consumul” de memorie este neglijabil, ba mai mult, putem folosi acea poziție pentru a reține o valoare care contează, de exemplu numărul total de voturi exprimate.

Identificarea tendințelor generale: Prin simpla vizualizare a listei de frecvențe, managerul poate trage concluzii imediate:

► **tendința pozitivă** – Dacă la indicii 4 și 5 am valorile cele mai mari, afacerea merge bine.



► **polarizarea** – Dacă avem multe valori la 1 și multe la 5, dar puține la 3, înseamnă că experiența clienților este inconsistentă (ori foarte bună, ori foarte rea).

► **cea mai frecventă notă** – Nota care are valoarea maximă în lista de frecvențe ne indică percepția dominantă a publicului.

De ce este eficientă această metodă?

- **viteză (performanță)**: parcurgem lista de răspunsuri **o singură dată**;
- **simplitate**: folosirea indexului listei ca „etichetă” pentru notă face codul foarte curat;
- **pregătire pentru vizualizare**: se poate ușor genera un grafic de tip bare (histogramă).

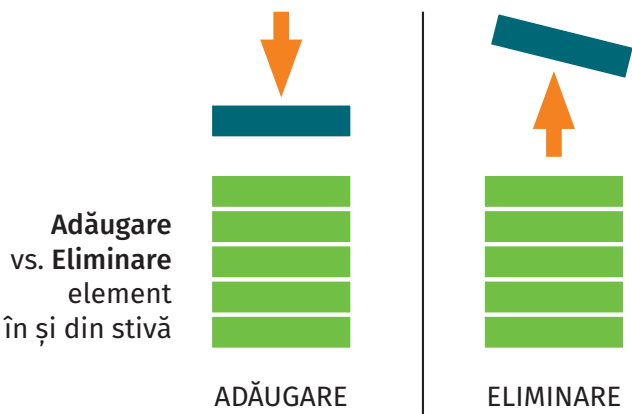
Există situații în care, pentru rezolvarea unei probleme, datele de intrare stocate în liste trebuie prelucrate într-o anumite ordine. Și cum modelele informatice sunt luate din viața reală, au apărut structurile de date ca **stiva** și **coada**.

4.1.3. Stiva

Pentru o structură de tip **Stivă (Stack)** ne putem imagina un teanc de farfurii puse una peste alta: nu poți scoate farfuria de la mijloc fără să le dai la o parte pe cele de deasupra, după cum nici nu poți insera o farfurie în mijlocul vrafului fără a periclita integritatea farfuriilor de deasupra. Operația de adăugare a unei farfurii va însemna plasarea ei deasupra teancului, iar extragerea uneia înseamnă luarea celei din vârful stivei.

De aici și denumirea structurii liniare (listei) ce va suporta operațiile în acest mod, aceea de **listă LIFO – Last In First Out** – ultimul element introdus este primul care va fi scos.





Structura de stivă se va implementa cu ajutorul unei liste, iar operațiile specifice stivei vor fi suportate de listă, dar la vârful (sfârșitul) ei.

Exemple din viața reală:

1

Butonul „Back” (Navigarea în Browser)

Când navighezi pe internet, istoricul sesiunii tale curente este o stivă în care pagina curentă este adăugată peste cea anterioară. Butonul „Back” (Înapoi) te duce la pagina accesată imediat înainte celei curente: dacă ai vizitat site-ul A, apoi B, apoi C, butonul „Back” te va scoate de pe C și te va trimite la B și încă un „Back” te duce în A.

2

Logistica în depozite înguste sau containerele de transport

În depozitele unde accesul se face printr-un singur capăt (LIFO), sortarea se face în adâncime. De exemplu, într-un container de marfă, ultimul palet încărcat la ușa va fi obligatoriu primul descărcat la destinație, așa că planificarea încărcării trebuie să țină cont de ordinea livrărilor.

De ce este utilă stiva? Principalul avantaj este memoria pe termen scurt a contextului: stiva îi permite unui sistem (sau unui om) să întreprună o sarcină, să rezolve o sub-sarcină urgentă și să revină exact în punctul în care a rămas.

>>> **EXEMPLU**

Operații în structura de tip Stivă

Într-o stivă au fost adăugate valorile 31, 16, 5 și 7.
Care este conținutul stivei după extragerea a două elemente, adăugarea valorilor 20 și 99 și apoi extragerea a trei elemente?

Pas	Oper.		Stivă	Pas	Oper.	Elem.	Stivă	Pas	Oper.	Elem.	Stivă	Pas	Oper.	Elem.	Stivă
0	Starea inițială		7	1	Extr 1	7	5	2	Extr 2	5	16	3	Adaug 1	20	20
			5				16				16				16
			16				31				31				31
			31				31				31				31
Pas	Oper.	Elem.	Stivă	Pas	Oper.	Elem.	Stivă	Pas	Oper.	Elem.	Stivă	Pas	Oper.	Elem.	Stivă
4	Adaug 2	99	99	5	Extr 1/3	99	20	6	Extr 2/3	20	16	7	Extr 3/3	16	31
			20				16				16				16
			16				31				31				31
			31				31				31				31

4.1.4. Coadă

Dacă stiva este despre „ultimul venit, primul servit” (LIFO), **Coadă (Queue)** este model de corectitudine și flux continuu: primul care intră în sistem este și primul care este procesat, toate celelalte așteptându-și rândul la ... coadă, așa explicându-se denumirea structurii de **FIFO – First In, First Out** – „primul venit, primul servit”.

Iată câteva exemple din viața reală în care structura de tip coadă este esențială pentru a evita haosul.

1

Serviciile cu publicul (bănci, poștă, supermarket)

Acesta este exemplul clasic. Indiferent că stai fizic la un rând sau primești un bilet cu un număr de ordine, sistemul respectă structura de coadă.



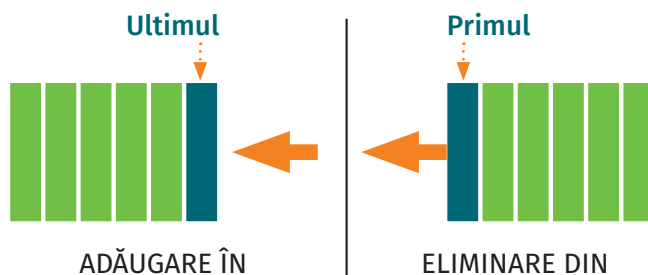
2

Imprimarea documentelor (Print Queue)

Dacă la birou, 10 colegi trimit documente la aceeași imprimantă (aproape) în același timp, dispozitivul nu explodează și nici nu amestecă paginile, deoarece are o memorie (buffer) care stochează cererile sub formă de coadă, astfel că documentul celui care a apăsat „Print” la ora 10:00:01 va fi tipărit integral înaintea celui pentru care a apăsat acest buton la 10:00:05.

Structura de coadă se va implementa cu ajutorul unei liste, iar operațiile specifice cozii vor fi efectuate *în listă* la pozițiile reținute pentru primul (**Eliminarea**) și, respectiv, după ultimul (**Adăugarea**) element.

Adăugare în coadă vs. Eliminare din coadă



3

Semafoarele și managementul traficului

Semafoarele inteligente funcționează pe un principiu de coadă. Fiecare direcție de mers „stă la coadă” pentru a primi culoarea verde, iar semafoarele smart detectează lungimile cozilor de mașini și decid cât de repede să dea verde unei benzi pentru a goli coada mai eficient.



Principalul avantaj al utilizării unei cozi este respectarea unui concept fundamental: ordinea și corectitudinea procesării. În programare, nu totul se poate întâmpla instantaneu, iar o coadă este instrumentul perfect pentru a gestiona „așteptarea”, garantând:

- **echitatea** – când resursele sunt limitate, coada asigură că primul element care a solicitat accesul este primul servit;
- **gestionarea asincronă (Buffering)** – dacă datele sunt produse mai rapid decât procesarea lor, coada acționează ca un „amortizor”.

>>> **EXEMPLU****Operații în structura de tip Coadă**

Într-o coadă au fost adăugate următoarele valori: 3, 10, 2, 8, 6. Care este conținutul cozii după ce s-au efectuat, în ordine, următoarele operații: extragerea unui element, adăugarea valorii 100, extragerea a trei elemente.

Pasul	Operație	Vizualizare Coadă	Element extras
0	Starea inițială	[3, 10, 2, 8, 6]	-
Pasul	Operație	Vizualizare Coadă	Element extras
1	Extragere	[10, 2, 8, 6]	3
2	Adăugare 100	[10, 2, 8, 6, 100]	-
3	Extragere	[2, 8, 6, 100]	10
4	Extragere	[8, 6, 100]	2
5	Extragere	[6, 100]	8

**IMPORTANT**

Când folosim Stiva și când folosim Coadă?
Alegerea între o Stivă și o Coadă depinde în totalitate de prioritatea sarcinii: contează **ordinea** sosirii (Coadă) sau **urgența** de moment (Stiva)?

Tabel comparativ Stivă și Coadă

Caracteristică	Stiva (Stack)	Coadă (Queue)
Principiu	LIFO (Last In, First Out)	FIFO (First In, First Out)
Analogia	Teancul de farfurii	Rândul la bilete
Prioritate	Cel mai recent element	Cel mai vechi element
Scop	Revenirea la starea anterioară	Procesarea corectă și ordonată

**STUDIU DE CAZ 3****Managementul sarcinilor unui programator**

Ne imaginăm un scenariu în care un programator are o listă de activități. Vom vedea cum se schimbă fluxul de lucru în funcție de structura aleasă.

Scenariul A: Organizarea tip Coadă

Context: Planificarea standard a zilei (**To-Do List**) (sarcinile în ordinea importanței sau a succesiunii lor)

- ▶ **Activitatea 1:** Verificarea e-mailurilor (09:00)
- ▶ **Activitatea 2:** Scrierea documentației (09:30)
- ▶ **Activitatea 3:** Ședința de echipă (11:00)

Relevanță: Coadă este ideală pentru **processe predictibile**. Garantează că nicio sarcină nu este uitată și că respectăm ordinea cronologică. Este „corectă” față de planificarea inițială.





Scenariul B: Organizarea tip Stivă

Context: Gestionarea urgențelor sau a întreruperilor – ești în mijlocul scrierii documentației (Activitatea 2 din coada de mai sus), dar apare o eroare critică pe server („Bug Fix”).

» **Sarcina curentă:** Documentație

» **Sarcina nouă (Urgentă):** Bug Fix (Intră în stivă peste documentație!)

» **Sarcina și mai nouă:** Telefon de la manager (Intră peste Bug Fix!)

Relevanță: Stiva este esențială pentru **gestionarea contextului**. Când ești întrerupt, pui „pe pauză” ce făceai, rezolvi noua problemă, apoi te întorci la ce aveai dedesubt. Este modul în care creierul nostru procesează alertele imediate.



IMPORTANT

„Când aleg ce?”

Folosește **Coada** când vrei să fii **metodic și organizat**. Este structura disciplinei.

Folosește **Stiva** când trebuie să fii **reactiv**. Este structura care gestionează stările de moment.

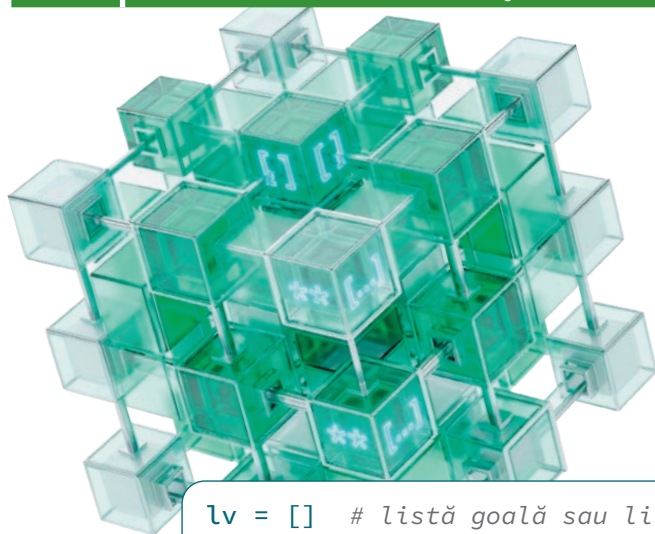


FIȘĂ DE LUCRU 7

- Care sunt indicii primului element și, respectiv, al ultimului element dintr-o listă cu n elemente (n număr natural)?
 - 0 și $n-1$
 - 1 și n
 - -1 și $n+1$
 - depinde de listă.
- Care este principiul fundamental de funcționare al unei stive?
 - acces aleatoriu
 - FIFO
 - LIFO
 - LIFO.
- Într-o stivă goală adăugăm 10, 20, scoatem un element, adăugăm 30. Ce element se află acum în vârful stivei?
 - 10
 - 20
 - 30
 - Stiva este goală.
- Care este principiul de funcționare al unei cozi?
 - FIFO
 - acces aleatoriu
 - LIFO
 - LIFO.
- În ce parte a cozii se fac inserările într-o coadă?
 - depinde de element
 - la început
 - la sfârșit
 - la mijloc.
- Care este conținutul unei cozii inițial goală, după executarea operațiilor următoare: adăugare 15, adăugare 14, adăugare 4, eliminare, adăugare 20, eliminare.
- Într-o stivă inițial goală s-au introdus în ordine valorile 1, 2, 3, iar într-o coadă, tot goală inițial, au fost introduse în ordine valorile 6, 5, 4. Care va fi valoarea din vârful stivei, dacă se extrag toate elementele din coadă și se adaugă, în ordinea extragerii, în stiva dată?



4.2. Clasa „list” din Python



4.2.1. Crearea unei liste

Pentru lucrul cu secvențe de date de tip listă în *Python* există clasa `list`.

O listă se creează scriind elementele între **paranteze drepte** `"[", "]"`, separate prin virgulă. Lista care nu conține elemente se numește listă vidă și poate fi definită prin folosirea de paranteze drepte goale. O listă care conține printre elemente alte liste se numește **listă imbricată**.

EXEMPLUL 1

>>>

```
lv = [] # listă goală sau listă vidă
numere = [1, 2, 3, 4] # listă de numere
nume = ["Alina", "Ion"] # listă de șiruri de caractere
l_eterogena = [[1, 3], "doi", [5, 6, 7]] # liste eterogene
elevi = [("Alina", 10), ("Mihai", 7), ("Irina", 9)] # listă de perechi
```



IMPORTANT

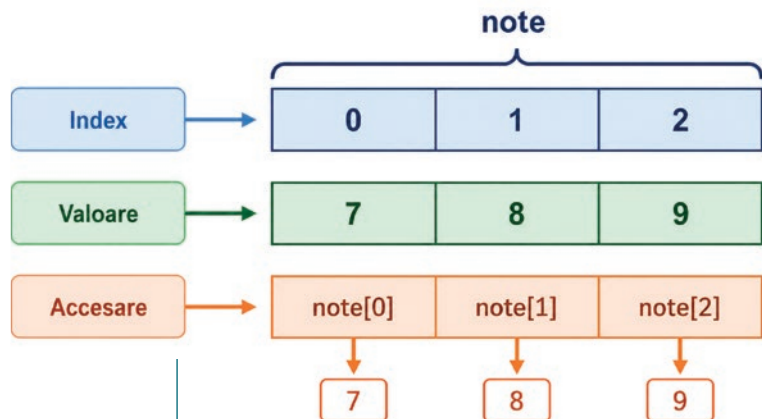
Indexarea începe de la **0**, elementul de pe poziția **i** este `lista[i]`:
`lista[i]` = elementul de pe poziția/indexul **i** (numerotare de la 0)

4.2.2. Indexarea elementelor din listă. Parcurgerea unei liste

Accesarea unei componente se face prin index, care se trece între paranteze drepte. Un element poate fi modificat folosind indexul său.

EXEMPLUL 2

>>>



Inițial avem lista `note = [7, 8, 9]`. Instrucțiunea `note[1] = 10` modifică valoarea de pe indexul 1 (al doilea element din listă).

Iar instrucțiunea `print(note)` va afișa lista `[7, 10, 9]`.

Indicii pot fi numere negative și permit numărarea elementelor de la dreapta spre stânga. Pentru exemplul de mai sus avem:

- `note[-1]` reprezintă ultimul element, 9;
- `note[-2]` reprezintă penultimul element, 10 (după modificare).

Dacă indicele nu este valid, apelul `note[i]` va da eroare (***IndexError***).



APLICAȚIE

Ce va afișa următoarea secvență de cod pentru `lista = [5, 8, 2, 4, 6, 2]`

```
print(lista[0])
print(lista[3])
print(lista[-2])
```



Parcurgerea unei liste înseamnă prelucrarea, pe rând, a fiecărui element al acesteia. În Python, listele pot fi parcurse în două moduri principale.

1 Parcurgerea directă a elementelor

Aceasta este metoda recomandată atunci când avem nevoie doar de valori.

EXEMPLUL 3

> > >

Cod Python

```
lista = [5, 8, 2, 4]
for x in lista:
    print(x)
```

Exemplu de rulare

```
5
8
2
4
```

La fiecare pas al buclei, variabila `x` primește valoarea elementului curent din listă. Este potrivită în majoritatea problemelor: calcul de sumă, numărare, verificare condiții.

2 Parcurgerea prin intermediul indicilor

Această metodă este utilă atunci când avem nevoie de poziția elementului.

EXEMPLUL 4

> > >

Cod Python

```
L = [5, 8, 2, 4]
for i in range(4):
    print("I:", i, "V:", L[i])
```

Exemplu de rulare

```
I: 0 V: 5
I: 1 V: 8
I: 2 V: 2
I: 3 V: 4
```

Variabila `i` reprezintă indexul elementului, iar accesul la valoare se face prin `L[i]`.

4.2.3. Operatori

Clasa `list` conține o serie de operatori utili pentru manipularea listelor.

Operatorul `+` este folosit pentru a concatena două liste.

EXEMPLUL 5

> > >

Cod Python

```
a = [5, 8, 2]
b = [4, 6, 2]
c = a + b
```

Explicații

`c` va avea valoarea `[5, 8, 2, 4, 6, 2]`



Operatorul `*` creează o listă formată prin repetarea listei inițiale de un număr dat de ori.

EXEMPLUL 6

> > >

Cod Python

```
a = [3] * 5
b = [1, 2, 3] * 2
```

Explicații

a va avea valoarea `[3, 3, 3, 3, 3]`
b va avea valoarea `[1, 2, 3, 1, 2, 3]`

Pentru a compara două liste între ele putem folosi operatorii cunoscuți, și anume: `==`, `!=`, `<`, `>`, `<=`, `>=`. Compararea este lexicografică, adică se face element cu element, luând elementele în aceeași ordine. Dacă toate elementele sunt egale până când una se termină, **lista mai scurtă este considerată mai mică**.

EXEMPLUL 7

> > >

Cod Python

```
a = [1, 2, 3]
b = [1, 2, 3]
c = [1, 2, 4]
d = [3, 2, 1]
```

Explicații

a == b va avea valoarea `True`
a != c va avea valoarea `True`
a <= c va avea valoarea `True`
a == d va avea valoarea `False`



APLICAȚIE

Ce valori au expresiile:

```
[1, 40, 2] > [3, 7]
[1, "b"] < [1, "a", 2]
```

Operatorii `in` și `not in` pot fi folosiți pentru a verifica apartenența unui element la o listă. Pot fi utilizați în aplicații de prelucrare a listelor în care datele trebuie validate (de exemplu: dacă elementul introdus face parte dintr-o listă de valori permise sau dacă un element nu trebuie să fie prezent într-o listă). Operatorii `in` și `not in` verifică corect (ambele întorc adevărat sau fals conform structurii listei) și eficient (implementarea internă are complexitate liniară $O(n)$) apartenența unui element la o listă.

EXEMPLUL 8

> > >

```
L = [1, 2, 3, 4, 5, 6, 7, 8]
```

Cod Python	<code>3 in L</code>	<code>3 not in L</code>	<code>11 in L</code>	<code>11 not in L</code>
Rezultat	True	False	False	True

4.2.4. Metode ale clasei *list*

Clasa `list` pune la dispoziție o serie de metode pe care le putem folosi în cadrul problemelor cu liste. În continuare, vom enumera o parte dintre acestea cu detalii și exemple:

- **lista.append (element)** are ca parametru un element și îl adaugă la finalul listei;

<code>note = [7, 9]</code> <code>note.append(8)</code>	<code>note = [7, 9, 8]</code>
---	-------------------------------

- **lista1.extend (lista2)** are ca parametru o listă și, după cum îi spune și numele, extinde lista cu elementele din lista dată ca parametru. Dacă metoda **append** primește ca parametru o listă, atunci întreaga listă va fi adăugată ca un singur element, rezultatul fiind o listă imbricată.

<code>note2 = [6, 5]</code> <code>note.extend(note2)</code>	<code>note = [7, 9, 8, 6, 5]</code>
<code>note3 = [8, 9]</code> <code>note3.append(note2)</code>	<code>note3 = [8, 9, [6, 5]]</code>

- **lista.sort()** are ca efect ordonarea listei în mod crescător.

<code>note.sort()</code>	<code>note = [5, 6, 7, 8, 9]</code>
--------------------------	-------------------------------------

Pentru a șterge elemente dintr-o listă există mai multe posibilități:

- **lista.pop()** poate fi folosită dacă știm indexul elementului dorit, având ca argument indexul. Metoda **pop** modifică lista și returnează elementul șters. Dacă apelăm metoda fără parametru, se va șterge ultimul element al listei.

<code>x = note.pop()</code>	<code>x = 9</code> <code>note = [5, 6, 7, 8]</code>
<code>note.pop(0)</code>	<code>note = [6, 7, 8]</code>

- **lista.remove()** poate fi folosită dacă vrem să ștergem o anumită valoare, pentru care nu știm poziția pe care se află. Cu această metodă putem șterge doar prima apariție a valorii dată ca parametru.

<code>note.extend([8, 9])</code>	<code>note = [6, 7, 8, 8, 9]</code>
<code>note.remove(8)</code>	<code>note = [6, 7, 8, 9]</code>

- **lista.insert(poz, elem)** are ca argumente o poziție și o valoare și inserează elementul pe poziția dată;

<code>note.insert(1, 8)</code>	<code>note = [6, 8, 7, 8, 9]</code>
--------------------------------	-------------------------------------

- **lista.reverse()** inversează ordinea elementelor din listă;

<code>note.reverse()</code>	<code>note = [9, 8, 7, 8, 6]</code>
-----------------------------	-------------------------------------

- **lista.copy()** realizează o copie a conținutului listei într-un obiect nou;

<code>ls = note.copy()</code> <code>ls.append(4)</code>	<code>note = [9, 8, 7, 8, 6]</code> <code>ls = [9, 8, 7, 8, 6, 4]</code>
--	---

- **lista.count(x)** numără de câte ori apare elementul dat ca parametru în listă;

<code>note.count(8)</code>	2
<code>note.count(1)</code>	0

- **lista.index(x)** returnează poziția (indexul) primei apariții a valorii x;

<code>note.index(8)</code>	1
<code>note.index(4)</code>	eroare

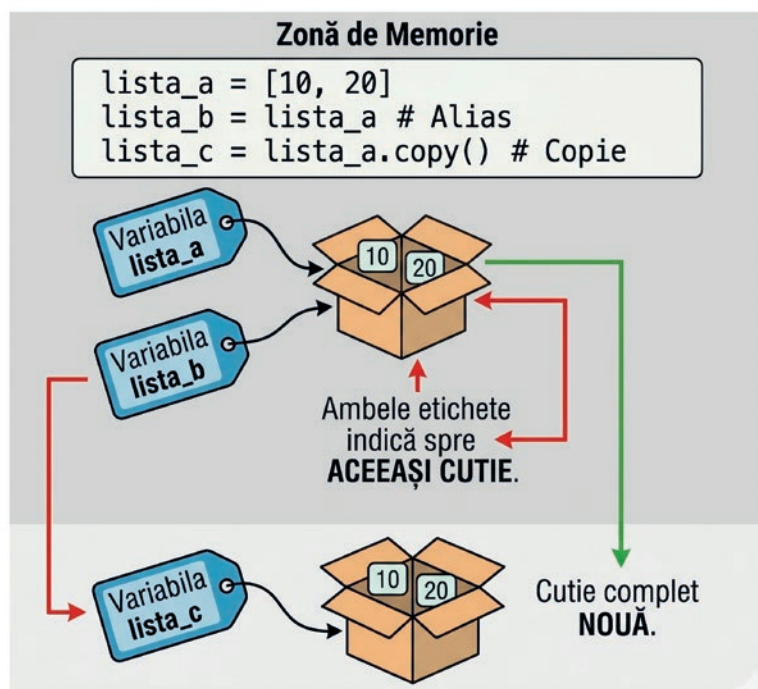
- **lista.clear()** are ca efect ștergerea tuturor elementelor din listă.

<code>note.clear()</code>	<code>note = []</code>
---------------------------	------------------------



Sinteză

Metodă	Efect	Parametri	Returnează
<code>append(x)</code>	adaugă un element la finalul listei	<code>x</code> – valoarea adăugată	<code>None</code>
<code>extend(l)</code>	adaugă la final toate elementele din lista <code>l</code>	<code>l</code> – listă	<code>None</code>
<code>sort()</code>	ordonează lista crescător	opțional <code>reverse=True</code>	<code>None</code>
<code>pop()</code>	șterge ultimul element	opțional – indexul de șters	elementul șters
<code>remove(x)</code>	șterge prima apariție a lui <code>x</code>	<code>x</code> - valoare	<code>None</code>
<code>insert(i,x)</code>	inserează elementul <code>x</code> pe poziția <code>i</code>	<code>i</code> – indice <code>x</code> – valoare	<code>None</code>
<code>count(x)</code>	numără aparițiile valorii <code>x</code>	<code>x</code> – valoare	numărul rezultat
<code>index(x)</code>	returnează indexul primei apariții	<code>x</code> – valoare	indice
<code>clear()</code>	golește lista	–	<code>None</code>
<code>reverse()</code>	inversează ordinea elementelor	–	<code>None</code>
<code>copy()</code>	copiază lista	–	lista nouă

Atenție: Diferența dintre “=” și `.copy()`**PROBLEMĂ
REZOLVATĂ 1****Enunțul problemei:**

Se citesc de la tastatură notele unui elev la materia Informatică, se presupune că elevul are 5 note. Să se modifice a doua notă obținută de elev (cu indexul/poziția 1), să se ștergă nota de pe indexul 2, apoi să se mai adauge două note.

Analiza problemei:

- *Date de intrare:* cele 5 note inițiale, valoarea pentru nota a doua și cele două note de adăugat.
- *Date de ieșire:* lista cu notele actuale ale elevului.



Rezolvare:

Program Python

```
note = []
for i in range(5):
    n1 = float(input(f"Citeste nota {i + 1}: "))
    note.append(n1)

print("\nNote initiale:", note)

# Modificarea celei de-a doua note
n2 = float(input("Citeste noua nota: "))
note[1] = n2
print("Dupa modificare:", note)

# Ștergerea notei de pe index 2
n3 = note.pop(2)
print(f"Dupa ștergerea notei ({n3}):", note)

# Adăugarea a două note noi
n4 = float(input("Citeste prima nota: "))
n5 = float(input("Citeste a doua nota: "))
note.append(n4)
note.append(n5)

# note.extend([n4,n5])

# Afișarea listei finale cu note
print("\nLista finala cu note:", note)
```

Explicații

Se folosește un `for` pentru a citi 5 note și a le adăuga în listă cu `append()`.

Elementul este accesat prin index. Mutabilitatea listei permite modificarea directă.

`pop(2)` elimină elementul de la indexul 2 și îl returnează.

`append()` adaugă fiecare element la sfârșitul listei, păstrând ordinea.

Se putea face și cu `extend()`.

Afișarea rezultatului final: Lista finală reflectă toate modificările succesive.

```
Citeste nota 1: 7
Citeste nota 2: 8
Citeste nota 3: 10
Citeste nota 4: 6
Citeste nota 5: 8

Note initiale: [7.0, 8.0, 10.0, 6.0, 8.0]
Citeste noua nota: 9
Dupa modificare: [7.0, 9.0, 10.0, 6.0, 8.0]
Dupa ștergerea notei (10.0): [7.0, 9.0, 6.0, 8.0]
Citeste prima nota: 4
Citeste a doua nota: 5

Lista finala cu note: [7.0, 9.0, 6.0, 8.0, 4.0, 5.0]
```

Exemplu de rulare



PROBLEMĂ REZOLVATĂ 2

Enunțul problemei:

Se citește de la tastatură lista participanților la un eveniment. Intrarea la eveniment se face în ordinea de pe listă. Să se modifice lista de participanți adăugând un nou participant la finalul listei, introducând un participant pe o poziție specifică și eliminând un participant de pe o poziție dată.

Analiza problemei:

- *Date de intrare:* numărul inițial de participanți, lista inițială de nume, numele noilor participanți și pozițiile de inserare, respectiv, ștergere.
- *Date de ieșire:* lista actualizată.

Rezolvare:

Program Python

```
# Citirea numărului inițial de participanți
n = int(input("Nr participanți: "))

# Citirea listei inițiale
lista = []
for i in range(n):
    nume = input(f"Nume {i + 1}: ")
    lista.append(nume)

print("\nLista inițială:", lista)

# Adăugarea la finalul listei cu append

nume = input("Nume de adăugat la final: ")
lista.append(nume)
print("Dupa adăugarea la final:", lista)

# Introducerea unui participant
# pe o poziție specifică cu insert
nume = input("Nume de inserat: ")
poz = int(input("Pozitia: "))
lista.insert(poz, nume)
print("Dupa inserare:", lista)

# Ștergerea unui participant
# de pe o poziție dată cu pop
poz = int(input("Pozitia de sters: "))
nume_sters = lista.pop(poz)

print(f"Participantul sters: {nume_sters}")
print("Lista finală:", lista)
```

Exemplu de rulare

```
Nr participanti: 3
Nume 1: Ana
Nume 2: Mihai
Nume 3: Ioana

Lista initiala: ['Ana',
'Mihai', 'Ioana']
Nume de adaugat la final:
George
Dupa adaugarea la final:
['Ana', 'Mihai', 'Ioana',
'George']
Nume de inserat: Elena
Pozitia: 2
Dupa inserare: ['Ana',
'Mihai', 'Elena', 'Ioana',
'George']
Pozitia de sters: 1
Participantul sters: Mihai
Lista finala: ['Ana',
'Elena', 'Ioana', 'George']
```

4.2.5. Funcții predefinite pentru liste

Python pune la dispoziție o serie de funcții predefinite ce pot fi folosite pentru a prelucra liste, fără a mai fi necesar să le definim de fiecare dată. În continuare, vom enumera cele mai utilizate funcții cu detalii și exemple:

- ▶ **len(lista)** – determină lungimea unei liste;

<code>numere = [4, 7, 2, 9]</code>	<code>len(numere)</code> este 4
------------------------------------	---------------------------------

- ▶ **min(lista)** și **max(lista)** – determină minimul și maximum dintr-o listă. Lista trebuie să aibă cel puțin un element, altfel programul va da eroare.

<code>numere = [4, 7, 2, 9]</code>	<code>min(numere)</code> este 2 <code>max(numere)</code> este 9
------------------------------------	--

- ▶ **sum(lista)** – calculează suma elementelor dintr-o listă, poate fi utilizată doar pentru liste cu elemente de tip numeric;

<code>numere = [4, 7, 2, 9]</code>	<code>sum(numere)</code> este 22
------------------------------------	----------------------------------

- ▶ **sorted(lista)** – returnează o listă nouă ordonată, fără a modifica lista inițială.

<code>numere = [4, 7, 2, 9]</code> <code>ordonata = sorted(numere)</code> <code>print(numere)</code> <code>print(ordonata)</code> <code>numere.sort()</code> <code>print(numere)</code>	Rezultatele afișărilor sunt: [4, 7, 2, 9] [2, 4, 7, 9] [2, 4, 7, 9]
--	--



IMPORTANT

Diferența fundamentală dintre metoda `.sort()` și funcția `sorted()` constă în modul în care gestionează memoria și datele originale. Atunci când apelezi `lista.sort()`, operația se realizează „*in-place*” (direct pe structura de date de intrare), ceea ce înseamnă că elementele listei inițiale sunt rearanjate permanent, memoria este optimizată, iar mecanismul returnează valoarea `None`. În schimb, funcția globală `sorted(lista)` este neinvazivă: ea lasă lista originală complet intactă și creează în memorie o copie nouă, ordonată, pe care o poți stoca într-o altă variabilă.



PROBLEMĂ REZOLVATĂ 3

Enunțul problemei:

La disciplina Informatică se înregistrează notele obținute de mai mulți elevi. Pentru fiecare elev se introduc de la tastatură notele obținute, una câte una. Introducerea notelor pentru un elev se oprește atunci când se introduce valoarea 0, care reprezintă o valoare de oprire și nu face parte din lista notelor.

Să se realizeze un program Python care:

- citește numărul n de elevi;
- pentru fiecare elev citește notele și le memorează într-o listă;
- pentru fiecare elev determină: lista notelor introduse, numărul de note, nota minimă, nota maximă, suma notelor, media aritmetică a notelor.

Analiza problemei:

Programul va utiliza subprograme pentru citirea notelor și pentru generarea raportului statistic.



Rezolvare:

Program Python

```
def citire_note():
    # Citește notele unui elev până la introducerea valorii 0
    note = []
    while True:
        nota = float(input("Introduceti o nota (0 pentru oprire): "))
        if nota == 0:
            break
        note.append(nota)
    return note

def raport_note(note): # Parametrul note - de tip listă
    if len(note) == 0:
        return None
    numar_note = len(note)
    nota_min = min(note)
    nota_max = max(note)
    suma_note = sum(note)
    media = suma_note / numar_note
    return numar_note, nota_min, nota_max, suma_note, media

# Program principal
n = int(input("Introduceti numarul de elevi: "))
for i in range(n):
    print(f"Elevul {i + 1}")
    lista_note = citire_note()
    rezultat = raport_note(lista_note)
    print("Lista notelor:", lista_note)
    if rezultat is None:
        print("Elevul nu are note.")
    else:
        numar_note, nota_min, nota_max, suma_note, media = rezultat
        print("Numar note:", numar_note)
        print("Nota minima:", nota_min)
        print("Nota maxima:", nota_max)
        print("Suma notelor:", suma_note)
        print("Media:", media)
```

```
Introduceti numărul de elevi: 2
Elevul 1
Introduceti o nota
(0 pentru oprire): 7
Introduceti o nota
(0 pentru oprire): 9
Introduceti o nota
(0 pentru oprire): 10
Introduceti o nota
(0 pentru oprire): 0
Lista notelor: [7.0, 9.0, 10.0]
Numar note: 3
Nota minima: 7.0
Nota maxima: 10.0
Suma notelor: 26.0
Media: 8.666666666666666
```

```
Elevul 2
Introduceti o nota
(0 pentru oprire): 8
Introduceti o nota
(0 pentru oprire): 6
Introduceti o nota
(0 pentru oprire): 0
Lista notelor: [8.0, 6.0]
Numar note: 2
Nota minima: 6.0
Nota maxima: 8.0
Suma notelor: 14.0
Media: 7.0
```



PROBLEMĂ REZOLVATĂ 4

Enunțul problemei:

Pentru un magazin online se știe istoricul prețurilor unui produs dat. Prețurile pot proveni din două surse: citire de la tastatură (toate valorile sunt șiruri de caractere la citire) sau procesare internă de date (lista poate fi eterogenă, adică poate conține `int`, `float`, `str`). Se cere să se determine lista finală cu prețuri (toate elementele convertite la `float`) și să se calculeze prețul minim și prețul maxim pentru lista aceasta. Să se rezolve această problemă în două moduri, folosind funcțiile `min()` și `max()` și calculând manual valorile.

Analiza problemei:

- *Date de intrare:* lista cu prețuri. Presupunem că toate elementele se pot converti la float, sunt prețuri valide și lista nu este goală.
- *Date de ieșire:* valorile corespunzătoare pentru minim și maxim și lista finală de prețuri.

Explicații:

- Pentru elemente numerice `min` și `max` funcționează corect direct, deoarece elementele sunt comparabile.
- Pentru elemente sub formă de text, funcțiile compară lexicografic, nu numeric. Deci avem nevoie de conversie de la `str` la tip numeric.
- Pentru listele eterogene (valori numerice și text amestecate), aplicarea directă a funcțiilor `min` și `max` poate genera eroare deoarece nu se pot compara direct `int/float` cu `str`.
- Pentru lista citită de la tastatură elementele sunt șiruri de caractere, deci conversia numerică este obligatorie.
- Putem folosi un subprogram (funcție) care primește lista inițială și transformă în float toate elementele, apoi aplică funcțiile `min` și `max` pe lista prelucrată.
- Calculul manual pentru minim și maxim ajută elevii să înțeleagă algoritmi din spatele funcțiilor.

Rezolvare:

Program Python

```
def prelucrare(lista1):
    # Convertim elementele în float
    lista2 = []
    for x in lista1:
        lista2.append(float(x))
    return lista2
# Citire listă de la tastatură
prod1 = []
nr = int(input("Citeste nr: "))
for x in range(nr):
    prod1.append(float(input("Citeste element:")))
# Exemplu de listă eterogenă
prod2 = [23, '15.5', 18, 30.0, '40']
preturi = prod1 + prelucrare(prod2)
print(f"Lista finala preturi: {(preturi)}")
# Min și max folosind funcții predefinite
if preturi:
    print(f"Pret minim: {min(preturi)}")
    print(f"Pret maxim: {max(preturi)}")
else:
    print(f"Lista nu are preturi valide")
```

Exemplu de rulare

```
Citeste nr: 3
Citeste element: 5
Citeste element: 10
Citeste element: 39
Lista finala preturi:
[5.0, 10.0, 39.0, 23.0,
15.5, 18.0, 30.0, 40.0]
Pret minim: 5.0
Pret maxim: 40.0
Pret minim manual: 5.0
Pret maxim manual: 40.0
```



Program Python

```
# Min și max calcul manual
if preturi:
    # inițializăm minimul și maximul cu prima
    # valoare din listă
    # comparăm restul elementelor cu min și max
    # calculate până la pasul curent și actualizăm unde
    # este cazul
    min_manual = preturi[0]
    max_manual = preturi[0]
    for p in range(1, len(preturi)):
        if preturi[p] < min_manual:
            min_manual = preturi[p]
        if preturi[p] > max_manual:
            max_manual = preturi[p]
    print(f"Pret minim manual: {min_manual}")
    print(f"Pret maxim manual: {max_manual}")
else:
    print(f"Lista nu are preturi valide")
```

Exemplu de rulare

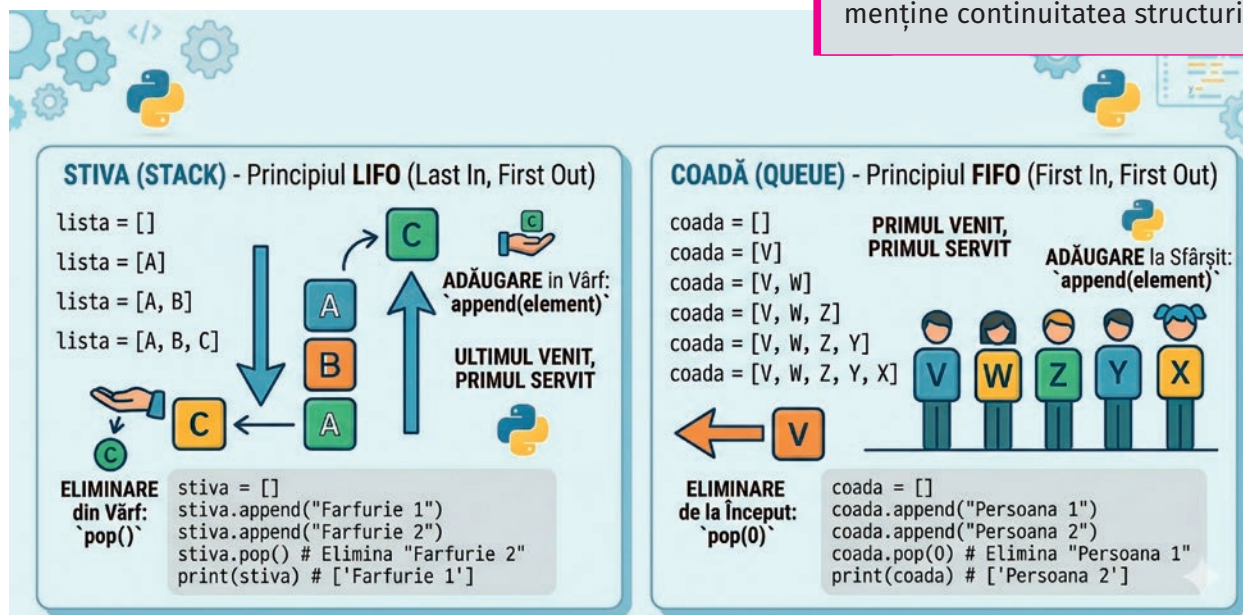
4.2.6. Stive și cozi

Stivele (*stack*) și cozile (*queue*) sunt structuri de date utilizate pentru organizarea și prelucrarea elementelor într-o anumită ordine de acces. Acestea pot fi implementate în Python folosind liste. Operațiile principale pentru stivă sunt adăugarea unui element în vârf (folosind `append()`) și eliminarea elementului din vârf (folosind `pop()`). În schimb, pentru o coadă operațiile principale sunt adăugarea unui element la sfârșit (`append()`) și eliminarea elementului aflat la început (`pop(0)`).



IMPORTANT

Operațiile `append()` și `pop()` au complexitatea $O(1)$, deoarece accesul la ultimul element nu implică deplasarea altor elemente. În schimb, operația `pop(0)` are complexitatea $O(n)$, deoarece necesită deplasarea tuturor elementelor rămase cu o poziție spre stânga pentru a menține continuitatea structurii.





PROBLEMĂ REZOLVATĂ 5



Enunțul problemei:

Într-un centru de livrări, comenzile trebuie organizate astfel încât să fie procesate eficient. Pentru acest lucru se pot utiliza trei tipuri de structuri de date: **liste**, **stive** și **cozi**, fiecare având un rol diferit în gestionarea comenzilor.

- **Lista** este folosită pentru stocarea tuturor comenzilor înregistrate. Aceasta permite memorarea elementelor în ordinea introducerii și accesul la ele prin poziție.
- **Coadă** este utilizată pentru procesarea comenzilor obișnuite.
- **Stiva** este folosită pentru comenzile urgente.

Prin combinarea acestor structuri, se poate organiza eficient fluxul comenzilor: cele obișnuite sunt procesate în ordinea sosirii, iar cele urgente pot avea prioritate.

Rezolvare:

Cod Python	Explicații
<pre># lista tuturor comenzilor comenzi = [] # coada comenzilor de procesat coada = [] # stiva comenzilor urgente stiva = [] def adauga_comanda(cod): comenzi.append(cod) coada.append(cod) def comanda_urgenta(cod): comenzi.append(cod) stiva.append(cod) def proceseaza_comanda(): if len(stiva) > 0: # dacă există comenzi urgente return stiva.pop() # LIFO elif len(coada) > 0: # altfel se procesează coada return coada.pop(0) # FIFO else: return None</pre>	Programul folosește trei liste: ► comenzi – lista generală cu toate comenzile; ► coada – comenzile obișnuite; ► stiva – comenzile urgente. Operațiile principale sunt: ► append() – adaugă un element la sfârșitul listei; ► pop() – elimină ultimul element (pentru stivă); ► pop(0) – elimină primul element (pentru coadă). Astfel: ► stiva funcționează cu append() și pop(); ► coada funcționează cu append() și pop(0).
<pre># exemplu de utilizare adauga_comanda("C101") adauga_comanda("C102") comanda_urgenta("C103") adauga_comanda("C104") comanda_urgenta("C105")</pre>	<pre>print("Se proceseaza:", proceseaza_comanda()) print("Se proceseaza:", proceseaza_comanda()) print("Se proceseaza:", proceseaza_comanda()) print("Toate:", comenzi) print("Urgente:", stiva) print("Restul, neprocesate:", coada)</pre>
Rezultat rulare:	
Se procesează: C105	Toate: ['C101', 'C102', 'C103', 'C104', 'C105']
Se procesează: C103	Urgente: []
Se procesează: C101	Restul, neprocesate: ['C102', 'C104']



FIȘĂ DE LUCRU 8

1. Se consideră programul:

```
L = [2, 7, 6, 4, 9, 8, 1, 3]
lpar = []
limpar = []
for i in range(len(L)):
    if L[i] % 2 == 0:
        lpar.append(L[i])
    else:
        limpar.append(L[i])
print(lpar)
print(limpar)
```

Cerințe:

- Ce se afișează la execuția programului?
- Ce reprezintă variabila `i`?
- Cum se poate rescrie programul fără utilizarea indicilor?

2. Se consideră programul:

```
L = [1, 2, 3, 4]
for x in L:
    if x % 2 == 0:
        L.remove(x)
print(L)
```

Cerințe:

- Ce se dorește să facă programul?
- De ce rezultatul poate fi incorect? Dă un exemplu.
- Propune o variantă corectă.

3. Un antrenor dorește să analizeze performanța unui grup de sportivi.

Se citesc de la tastatură un număr natural n și cele n scoruri obținute de sportivi. Să se calculeze scorul minim, scorul maxim și media scorurilor.

4. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se calculeze numărul de perechi de valori pare aflate pe poziții consecutive din listă. Verifică dacă algoritmul este corect testând cazurile limită (extremități, liste scurte, liste fără elemente pare).

5. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se creeze o listă cu elementele pozitive, iar la final să fie cele negative.

6. Se consideră o listă formată din valori numerice. Se cere să se realizeze rotirea la stânga a listei cu k poziții, utilizând exclusiv operații specifice unei cozi.

7. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se determine numărul de elemente pare din listă.

8. Se citesc de la tastatură un număr natural n , un număr întreg x și o listă de n numere întregi. Să se șteargă din listă toate aparițiile numărului x .

9. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se determine lungimea maximă a unui șir de elemente egale consecutive.

Exemplu: $n = 8$; lista = 1 1 2 2 2 3 3 2; rezultat: 3

10. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se determine subsecvența crescătoare de lungime maximă.

Exemplu: $n = 7$, lista = 1 2 5 3 4 6 1; rezultat: 3 4 6.

4.3. Metode de generare sistematică a elementelor unei liste

În informatică, multe probleme presupun **generarea unei secvențe de valori** care respectă anumite proprietăți sau reguli. În loc să introducem valorile manual, putem folosi algoritmi simpli pentru a construi automat aceste secvențe sub forma unor liste.

Exemple

- ▶ Toate numerele pare între 2 și 20.
- ▶ Termenii unei expresii matematice $2*n+1$ pentru $n=0,1,2,\dots,5$.
- ▶ Primii 6 termeni ai unui șir Fibonacci.

Pentru a genera liste, cea mai folosită metodă este parcurgerea unui interval de valori, adăugând elementele într-o listă goală.

EXEMPLE

> > >

```
numere_pare = []
for i in range(2, 21, 2):
    # start=2, stop=21, pas=2
    numere_pare.append(i)
print(numere_pare)
```

[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

1) secvență de numere pare;
2) termeni ai unei expresii matematice, de exemplu: $2*n+1$ pentru $n=0,1,2,3,4,5$;

```
termeni = []
for n in range(6):
    termeni.append(2*n + 1)
print(termeni)
```

[1, 3, 5, 7, 9, 11]

3) termeni ai unor șiruri recurente, de exemplu șirul Fibonacci (primele 6 valori).

```
fibonacci = [0, 1]
for i in range(2, 6):
    fibonacci.append(fibonacci[i-1] + fibonacci[i-2])
print(fibonacci)
```

[0, 1, 1, 2, 3, 5]



Pașii pentru a genera secvențe sunt următorii:



1. Identificarea regulilor – ne punem întrebarea „Cum se obține fiecare element?”
Exemplu: $x = 2 \cdot n + 1$, Fibonacci: $F_n = F_{n-1} + F_{n-2}$
2. Stabilirea intervalului sau numărului de termeni.
– `range(start, stop, pas)` sau numărul de pași
3. Inițializarea listei.
– De obicei: `lista = []` sau lista cu valori de început (ex. `[0,1]` la Fibonacci)
4. Parcurgerea intervalului și adăugarea elementelor.
– `for i in range(...)` și `append()`
5. Aplicarea algoritmilor de bază.
– Sumă, medie, numărare, determinarea maximului/minim, verificări condiționale.



PROBLEMĂ REZOLVATĂ 1

Enunțul problemei: Un exemplu concret este generarea **listei temperaturilor medii anuale** pentru un oraș, într-un scenariu de încălzire globală, știind că temperaturile cresc treptat, de la un an la altul.

Analiza problemei:

Pașii pentru construirea listei sunt următorii:

1. **Identificarea regulii:** Trebuie să știm cum se modifică fiecare termen.
Exemplu: temperatura crește cu 0,2 °C față de anul precedent, notat **increment**:
– temperatura anului i = temperatura inițială + increment $\times i$
2. **Stabilirea valorii inițiale și a numărului de termeni:**
– temperatura din primul an este punctul de plecare, de exemplu: 14,0 °C;
– numărul de ani ne dă câți termeni trebuie să generăm (exemplu: 10).
3. **Crearea unei liste goale:**
– `temperaturi = []`
4. **Parcurgerea intervalului și adăugarea elementelor conform regulii:**
– folosim o buclă `for`;
– calculăm fiecare valoare și o adăugăm cu `append()`.
5. **Lista poate fi folosită ulterior pentru alte calcule.**

Rezolvare:

Cod Python

```
T = []
T0 = 14.0
nr_an = 10
increment = 0.2
for i in range(nr_an):
    T1 = T0 + i * increment
    T.append(T1)
print(T)
```

Exemplu de rulare

```
[14.0, 14.2, 14.4, 14.6, 14.8, 15.0, 15.2,
15.4, 15.6, 15.8]
```



PROBLEMĂ REZOLVATĂ 2



Enunțul problemei: Se citește de la tastatură un număr natural n . Să se calculeze suma $S=1+1*2+1*2*3+\dots+1*2*3*\dots*n$.

- Să se calculeze suma S folosind un **algoritm de calcul direct**, în care fiecare termen este determinat independent.
- Să se calculeze aceeași sumă folosind un **algoritm de generare recurent**, în care fiecare termen este obținut pe baza termenului precedent.
- Să se justifice utilizarea algoritmului recurent prin compararea eficienței celor două metode.

Explicații:

a) Algoritm de calcul direct

Ideea: pentru fiecare k , calculăm separat produsul $1*2*3*\dots*k$, apoi îl adunăm la sumă. Pentru fiecare termen reluăm calculul produsului de la 1. De exemplu, pentru $k = 5$ refacem și $1*2*3*4$ deja calculate anterior.

b) Algoritm de generare recurent

Observăm relația: $1*2*3*\dots*n = [1*2*3*\dots*(n-1)]*n$. Putem construi fiecare termen din precedentul și nu mai recalculăm produsul, folosim rezultatul anterior și îl înmulțim cu k curent.

c) Implementare Python

Algoritm de calcul direct	Algoritm de generare recurent
<pre>n = int(input("n = ")) S = 0 for k in range(1, n + 1): prod = 1 for i in range(1, k + 1): prod = prod * i S = S + prod print("S =", S)</pre>	<pre>n = int(input("n = ")) S = 0 prod = 1 for k in range(1, n + 1): prod = prod * k S = S + prod print("S =", S)</pre>
<pre>n = 4 S = 33</pre>	<pre>n = 4 S = 33</pre>

d) Compararea eficienței

	Algoritm de calcul direct	Algoritm de generare recurent
Număr operații	$1 + 2 + \dots + n = \frac{n * (n + 1)}{2}$	$2*n$ (o înmulțire și o adunare la fiecare pas)
Complexitate	$O(n^2)$	$O(n)$

e) Concluzie

Algoritmul recurent este mult mai eficient deoarece:

- evită recalcularea inutilă a produselor;
- reduce numărul de operații de la $O(n^2)$ la $O(n)$;
- pentru valori mari ale lui n , diferența devine semnificativă.



FIȘĂ DE LUCRU 9



1. Se consideră programul:

```
n = 6
a = 1
for i in range(1, n):
    a = a * 2 + i
    print(a)
```

Cerințe:

- a) Ce se afișează la execuția programului?
- b) Corectează codul astfel încât bucla să execute și pentru $i=n$

2. Se consideră programul:

```
numere = [3]
for k in range(3):
    urmatorul = numere[k] + 5
    numere.append(urmatorul)
print(numere)
```

Cerințe:

- a) Ce se afișează la execuția programului?
- b) Care este formula șirului recurent?

3. Un cont bancar are o sumă inițială și o dobândă anuală (procentuală). Să se determine numărul de ani după care soldul contului depășește o valoare dată. Se citește de la tastatură suma inițială s_0 , dobânda r , și valoarea țintă s_1 și se vor afișa lista soldurilor pentru fiecare an și numărul minim de ani necesari.

4. Se citește de la tastatură un număr natural n . Să se determine lista primelor n pătrate perfecte.

5. Se consideră șirul lui Fibonacci definit prin relația de recurență:

$$F_0 = 0, F_1 = 1, F_n = F_{(n-1)} + F_{(n-2)}, n \geq 2$$

Se cunoaște un termen din șirul lui Fibonacci și se dorește afișarea termenilor în **ordine descrescătoare**:

$$F_n, F_{n-1}, F_{n-2}, \dots, F_1.$$

Să se analizeze două metode de generare a termenilor și să se determine **ordinul de complexitate** al fiecărei metode.

6. Se consideră șirul definit prin $a_0 = 3$ și relația de recurență: $a_n = 2 * a_{n-1}$, $n > 0$. Să se scrie un program Python eficient care afișează primii n termeni ai șirului.

7. Se citește de la tastatură un număr natural n și o listă de n numere întregi. Să se genereze lista ce are ca valori medii aritmetice dintre oricare două elemente consecutive ale listei.

Exemplu: din lista $[2, 4, 6, 10]$ obținem $[3, 5, 8]$.

8. Se citește un număr natural n . Să se construiască lista cu cifrele acestuia.

9. Se consideră șirul definit prin $a_0 = 1$ și relația de recurență: $a_n = a_{n-1} + n$, pentru $n > 0$. Să se scrie un program Python care afișează primii n termeni ai șirului.

4.4. Metode de sortare



4.4.1. Introducere

Dacă în viața de zi cu zi a sorta înseamnă a alege, a aranja, a repartiza, în programarea calculatoarelor, sortarea elementelor este sinonimă cu ordonarea lor crescătoare sau descrescătoare după anumite criterii. Ordonarea elementelor unei liste stă la baza eficienței în informatică, cel mai puternic argument fiind viteza de regăsire a informației.

Iată principalele motive pentru care sortăm datele:

1 Căutarea este substanțial mai rapidă
Pentru a găsi un element într-o listă nesortată trebuie să verifici fiecare poziție în parte (căutare liniară), pe când într-o listă ordonată se pot folosi algoritmi de căutare rapidă.

2 Identificarea duplicatelor
Într-o listă neordonată, ca să vezi dacă un element se repetă, trebuie să îl compari cu toate celelalte. Într-o listă sortată, duplicatele sunt adiacente (stau unul lângă altul).

3 Găsirea extremelor
Dacă ai nevoie de cel mai mic sau cel mai mare element, o listă sortată crescător îți oferă răspunsul instantaneu: minimul este la indexul 0 iar maximul este la indexul $n-1$.

4 Pregătirea datelor pentru alți algoritmi
Multe procese complexe depind de date ordonate pentru a funcționa corect sau rapid:

- ▶ algoritmi de compresie a datelor;
- ▶ randare grafică (pentru a desena obiectele de la spate spre față);
- sisteme de baze de date (indexarea).



IMPORTANT



Sortarea se efectuează nu pentru a avea o listă „frumoasă”, ci pentru a reduce complexitatea computațională a tuturor acțiunilor ulterioare asupra datelor.

Analogie din viața reală

1. Să ne imaginăm că suntem într-o bibliotecă unde cărțile nu sunt sortate alfabetic sau pe domenii, ci sunt aruncate la întâmplare pe rafturi. Ca să fie găsită o anumită carte, ar trebui să fie citite cotoarele tuturor cărților. Sortarea transformă orele de căutare în secunde.

2. De fiecare dată când pe un site al unui magazin online se folosește un filtru (de exemplu: „Preț: de la mic la mare”), se rulează un algoritm de sortare.

3. Sortarea alfabetică (**lexicografică**) a agendei telefonice permite căutarea/găsirea rapidă a unui contact.





Există foarte multe metode de sortare și o paletă largă a domeniilor de aplicare:

- **Informatică** – aproximativ 40 de algoritmi clasici, plus metode pentru situații specifice.
 - **Logistică și industrie**
 - *Cross-Belt Sorting*: folosită în aeroporturi (mini-benzi perpendiculare pe banda principală);
 - *Sortarea gravitațională*: obiectele alunecă pe jgheaburi diferite în funcție de greutate.
 - **Statistică și Știința datelor**
 - *Sortarea Topologică*: stabilirea ordinii sarcinilor care depind unele de altele.
 - **Metode „umane” și de arhivare**
 - *Sistemul decimal Dewey*: folosit în biblioteci pentru a sorta cărțile pe domenii și autori.
 - *Triajul medical*: sortarea pacienților în funcție de urgență (cod roșu, galben, verde).
- De ce există atât de multe? Pentru că niciun algoritm nu este perfect. Metoda ce urmează a fi utilizată se poate alege în funcție de:
- natura obiectului: (e un număr în memorie sau un colet de 10 kg ?);
 - volum: (sunt 10 elemente sau 10 miliarde?);
 - cost: (consum de procesor vs. consum de curent/ resurse etc.).

4.4.2. Sortarea prin Selecție – Selection Sort

Este una dintre metodele cele mai des folosite pentru ordonarea unei liste de valori datorită simplității ideii pe care se bazează: în lista ordonată crescător, elementul de pe prima poziție este cel mai mic, al doilea este al doilea cel mai mic etc. Astfel, algoritmul caută cel mai mic element din listă și îl pune direct la locul lui final în lista ordonată.

Descrierea algoritmului (pentru ordonarea crescătoare)

Se pleacă cu toată lista inițială neordonată și se construiește pe parcurs o parte **sortată** (la stânga) lăsând partea **nesortată** (la dreapta) din ce în ce mai mică, executându-se următoarele:

- se caută cel mai mic număr din partea nesortată;
- se schimbă acest număr cu primul element din zona nesortată;
- se mută „granița” dintre cele două zone cu o poziție la dreapta;
- se repetă procesul până când toată lista este sortată.

> > >

EXEMPLU

Lista de valori [29, 10, 14, 37, 13]

Poziție/ indice 0 1 2 3 4

Pas	Operație
1	„Granița” dintre zone este la 0. Minimul din zona nesortată (toată lista) este 10, la poziția 1 . Schimbăm minimul cu elementul de pe „graniță” și lista devine [10, 29, 14, 37, 13].
2	„Granița” dintre zone este la 1. Minimul din zona nesortată (de la 1 la n-1 în lista) este 13, la poziția 4 . Schimbăm minimul cu elementul de pe „graniță” și lista devine [10, 13, 14, 37, 29].
3	„Granița” dintre zone este la 2. Căutăm minimul în [14, 37, 29]; este 14 , la locul lui, lista rămâne aceeași [10, 13, 14, 37, 29].
4	„Granița” dintre zone este la 3. Căutăm minimul în [37, 29]; e 29 și lista devine [10, 13, 14, 29, 37].

Observație: Ne oprim după n-1 pași, deoarece dacă sunt fixate pe locurile lor cele mai mici n-1 elemente, al n-lea rămas este automat cel mai mare și își are locul pe ultima poziție.

Implementarea este foarte intuitivă: folosim o variabilă `min_idx` pentru a ține minte unde se află cel mai mic element găsit, „granița” la pasul `i` fiind tocmai `i`.



Cod Python

```
def selection_sort_cresc(lista):
    n = len(lista)
    for i in range(n-1): # Parcurgem lista până la penultimul element
        min_idx = i      # Inițializarea poziției minimului
        # Căutăm un eventual element mai mic în dreapta graniței(zona nesortată)
        for j in range(i + 1, n):
            if lista[j] < lista[min_idx]:
                min_idx = j # Un nou minim la dreapta, la poziția j
        # minimul găsit la min_idx se interschimbă cu elementul de pe granița i
        lista[i], lista[min_idx] = lista[min_idx], lista[i]
```

Exemplu de rulare

```
date = [29, 10, 14, 37, 13]
selection_sort_cresc(date)
print("Lista sortata crescător prin selecție: ",date)
# Pe ecran se va afișa
Lista sortata crescător prin selecție:  [10, 13, 14, 29, 37]
```

IMPORTANT



Interschimbarea/swap
valorilor variabilelor
a și **b** se poate scrie
a, b = b, a

Analiza Performanței

Observație: Algoritmul de sortarea prin selecție nu se oprește niciodată mai devreme.

A. Complexitatea Timp

Analiza pașilor pentru o lista cu n elemente:

- pentru primul loc, comparăm elementul curent cu restul de $n-1$ elemente;
- pentru al doilea loc, comparăm cu restul de $n-2$ elemente;
- continuăm până la ultima comparație.

Matematic, numărul total de comparații este suma: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$, adică $(n^2 - n)/2$, deci **Complexitatea** este **$O(n^2)$** (pătratică, „de ordin n^2 ”).

Caz	$O()$	Explicație
Cel mai nefavorabil (Worst case)	$O(n^2)$	Lista este în ordine inversă.
Mediu (Average Case)	$O(n^2)$	Lista este amestecată aleatoriu
Cel mai favorabil (Best case)	$O(n^2)$	Chiar dacă lista este deja sortată, algoritmul o parcurge pe toată pentru a găsi minimul.

B. Complexitatea Spațiu

Câtă memorie ocupă? Algoritmul nu are nevoie să creeze o copie a listei sau să rezerve memorie suplimentară proporțională cu numărul de elemente, folosește doar câteva variabile temporare, adică Sortarea prin Selecție a unei liste de n elemente este un algoritm de complexitate spațială **$O(1)$** („de ordin 1”), adică este un *algoritm „in place”*.



PROBLEMĂ REZOLVATĂ

Enunțul problemei:

Se dă o listă cu n elemente numere naturale și un număr k (n și k numere naturale, $k \leq n$). Ordonează crescător primele k elemente ale șirului folosind sortarea prin selecția maximului și descrescător ultimele $n-k$ elemente folosind sortarea prin selecția minimului.

Exemplu: Pentru $n=6$, $k=3$ și elementele `5, 2, 8, 1, 9, 3` => `2, 5, 8, 9, 3, 1`

Analiza problemei:

Dacă la ora de sport cei n elevi ai unei clase trebuie să formeze un șir în care fetele ocupă primele k poziții, iar băieții ocupă următoarele $n-k$ poziții, ne imaginăm ușor felul în care se reazăază copiii în ordinea cerută (fetele crescător, băieții descrescător) după înălțime. Putem spune că avem două probleme într-una, dar în realitate avem de aplicat același algoritm de sortare prin selecție pe două zone distincte din lista inițială, cu atenție la sensul de parcurgere, la graniță și la elementul căutat (minim sau maxim).

Cod Python

```
# citire n
n = int(input("n = "))
# citire n elemente
v = []
for i in range(n):
    x = int(input("v[" + str(i) + "] = "))
    v.append(x)
k = int(input("k = "))
# sortare crescătoare a primelor k elemente:
# la fiecare pas i căutăm maximul din zona 0..i
# și îl plasăm pe granița i. Sensul parcurgerii
# listei e de la dreapta spre stânga.
for i in range(k - 1, 0, -1):
    poz_max = 0
    for j in range(1, i + 1):
        if v[j] > v[poz_max]:
            poz_max = j
    v[i], v[poz_max] = v[poz_max], v[i]
# sortare descrescătoare
for i in range(n - 1, k, -1):
    poz_min = k
    for j in range(k + 1, i + 1):
        if v[j] < v[poz_min]:
            poz_min = j
    v[i], v[poz_min] = v[poz_min], v[i]
# afișare
print("Șirul final este:\n", v)
```

Exemplu de rulare

```
n = 7
v[0] = 4
v[1] = 3
v[2] = 2
v[3] = 1
v[4] = 6
v[5] = 7
v[6] = 8
k = 4
Șirul final este:
[1, 2, 3, 4, 8, 7, 6]
```

A. Complexitatea Timp

Ordonarea

- fetelor: $O(k^2)$
- băieților: $O((n-k)^2)$

=>
 $O(k^2) + O((n-k)^2)$
 =>
 $O(k^2 + (n-k)^2)$
 =>
 $O(n^2)$

B. Complexitatea Spațiu

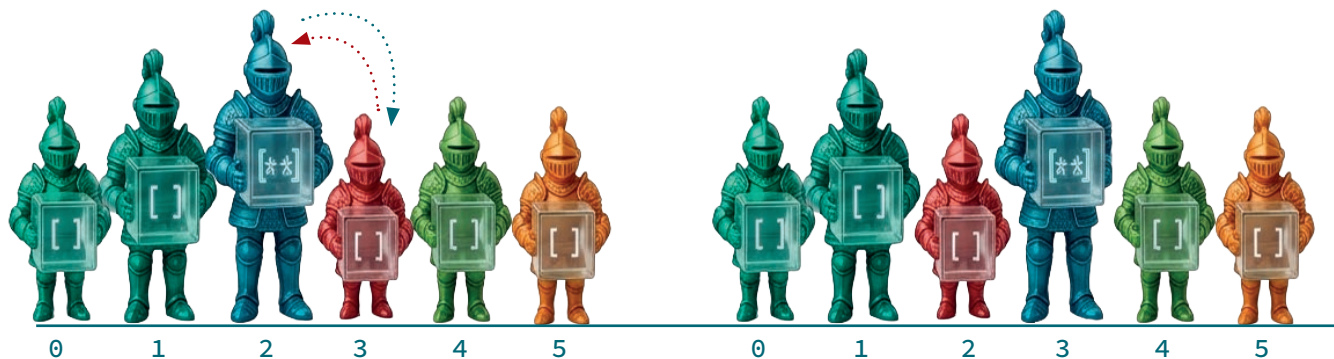
Sortarea prin selecție nu folosește memorie suplimentară semnificativă ci doar câteva variabile auxiliare, adică este „in-place” => Spațiu necesar **$O(1)$**

4.4.3. Metoda Bulelor (Bubble Sort)

Printre elevi, acesta este, probabil, cel mai cunoscut algoritm de sortare, pentru că simplitatea sa îl face ideal pentru a înțelege conceptul de manipulare a datelor unei liste și are la bază o observație simplă: în lista ordonată crescător, un element este mai mic decât vecinul aflat la dreapta sa. Astfel, se parcurge șirul de mai multe ori (mai mulți pași) și se compară două elemente adiacente: dacă acestea nu sunt în ordinea corectă, le interschimbă.

Descrierea algoritmului pentru ordonarea crescătoare: la fiecare pas se adaugă un element la partea **sortată** (la dreapta), lăsându-se partea **nesortată** (la stânga) din ce în ce mai mică, executându-se următoarele:

- ▶ se compară două elemente alăturate din zona nesortată;
- ▶ valoarea mai mare va fi trecută pe poziția din dreapta;
- ▶ se mută „granița” dintre cele două zone cu o poziție spre stânga;
- ▶ se repetă procesul până când toată lista este sortată.



EXEMPLU PAS CU PAS

Lista de valori [29, 10, 13, 37, 14]
Poziție/ indice 0 1 2 3 4 „Granița” dintre zone este la **4**.

Pas	Operație
1	Comparăm de fiecare dată două valori vecine și o trecem pe cea mai mare în dreapta. Comparăm 29 cu 10, interschimbăm și lista devine [10, 29, 13, 37, 14]. Comparăm 29 cu 13, interschimbăm și lista devine [10, 13, 29, 37, 14]. Comparăm 29 cu 37, în dreapta e cel mai mare dintre ele, nu interschimbăm. Comparăm 37 cu 14, interschimbăm și lista devine [10, 13, 29, 14, 37]. A fost plasat elementul pe „graniță”, iar aceasta se decrementează, devenind 3 .
2	Reluăm parcurgerea de la poziția 0 spre dreapta (până la graniță) și efectuăm aceleași operații. Comparăm 10 cu 13, nu interschimbăm. Comparăm 13 cu 29, nu interschimbăm. Comparăm 29 cu 14, interschimbăm și lista devine [10, 13, 14, 29, 37]. Am ajuns la „graniță” (poziția 3), ne oprim; noua „graniță” dintre zone este la 2 .
3	Vom relua operațiile de comparare a două elemente vecine și observăm că nu va fi efectuată nicio interschimbare, ceea ce înseamnă că lista are elementele ordonate crescător.

**Observații:**

- La fiecare **pas i**, valoarea maximă din zona nesortată (de la 0 până la $n-i$) va fi plasată pe **graniță** (poziția $n-i$).
- Ținem cont de legătura dintre pasul curent și graniță.
- Dacă la un pas nu s-au efectuat interschimbări, lista este ordonată crescător.

Implementarea este foarte intuitivă și folosește o variabilă g pentru a ține minte unde se află granița și o variabilă booleană `sortat` cu rol de *fanion/flag*, care ne va spune dacă elementele sunt ordonate crescător înainte de consumarea tuturor pașilor.

Cod Python

```
def bubble_sort(lista):
    n = len(lista)
    g = n - 1          # g e granița
    sortat = False     # Inițializăm cu False pentru a intra în loop
    # Rulăm cât timp granița e validă ȘI nu am confirmat că lista e sortată
    while g > 0 and not sortat:
        sortat = True  # Presupunem că lista e sortată
        for i in range(0, g):
            if lista[i] > lista[i + 1]:
                lista[i], lista[i + 1] = lista[i + 1], lista[i]
                sortat = False # Resetăm flagul
        g -= 1         # Restrângem zona de căutare
```

Exemplu de rulare

```
numere = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(numere)
print("Lista sortată crescător: ", numere)
# Pe ecran se va afișa
Lista sortată crescător: [11, 12, 22, 25, 34, 64, 90]
```

Analiza Performanței

Observație: Algoritmul de sortare prin metoda bulelor – *bubble sort* – se poate opri și înainte de a fi executată cea maxim $n-1$ pași (vezi rolul variabilei `sortat`).

A. Complexitatea Timp

La prima parcurgere comparăm $n-1$ perechi de elemente, pentru al doilea loc comparăm $n-2$ perechi de elemente etc. Numărul total de comparații este $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$, adică $(n^2 - n)/2$, deci complexitatea este $O(n^2)$ („de ordin n^2 ”).

Caz	$O()$	Explicație
Cel mai nefavorabil (Worst case)	$O(n^2)$	Lista este în ordine inversă.
Mediu (Average Case)	$O(n^2)$	Lista este amestecată aleatoriu
Cel mai favorabil (Best case)	$O(n)$	Dacă lista este deja sortată

B. Complexitatea Spațiu

Algoritmul nu are nevoie să creeze o copie a listei, ci folosește doar câteva variabile temporare, adică este tot un *algoritm „in place”*, deci are complexitate spațială $O(1)$ („de ordin 1”).



PROBLEMĂ REZOLVATĂ

Enunțul problemei: Pentru fiecare dintre cei n copii numerotați de la 1 la n (n număr natural nenul), cunoaștem numărul de pagini citite în weekend – valori exprimate prin numere naturale. Afișează numerele de ordine ale copiilor în ordinea crescătoare a numărului de pagini citite.

Exemplu

Pentru 7 copii cu numărul de pagini citite, în ordine, de 128, 120, 160, 145, 110, 140 și 102. Se va afișa: 7, 5, 2, 1, 6, 4, 3.

Explicație:

Cele mai puține pagini, 102, le-a citit copilul al 7-lea, deci prima valoare din rezultat este 7, al doilea, cu 110 pagini citite, este inițial pe poziția a 5-a ș.a.m.d., cel mai „cititor” fiind copilul de pe poziția inițială 3, cu 160 de pagini, deci 3 este ultima valoare în șirul cerut de problemă.

Analiza problemei:

Este evident că trebuie să ordonăm copiii crescător după numărul de pagini citite, dar trebuie să reținem și pozițiile inițiale ale lor. Pentru aceasta, vom folosi o listă suplimentară care să rețină indicii și care se va modifica la fel cum se modifică și lista cu numărul de pagini, pe tot parcursul algoritmului de sortare a acesteia. Astfel, pentru exemplu pornim cu listele inițiale pagini = [128, 120, 160, 145, 110, 140, 102] și indici = [0, 1, 2, 3, 4, 5, 6].



IMPORTANT

◀ Deși în problemă indexarea este de la 1, noi folosim indexarea din Python de la 0, ținând minte ca la final să adunăm 1 la fiecare valoare tipărită din lista indici.

◀ În program sunt folosite două variante pentru popularea unei liste:

- citirea de la tastatură și adăugarea cu `append` (pentru vectorul/lista pagini); pentru claritatea introducerii, pe ecran este afișat numele listei și indicele elementului pentru care se introduce valoarea:

```
("pagini["+ str(i+1) +"]="=");
```
- inițializarea lui `indici[i]` cu `i`;

◀ Afișarea pe ecran a elementelor listei ordonate folosind `end = " "`, care îi spune lui `print`:

- nu trece pe linie nouă după afișare;
- pune în schimb `" "` (spațiu) la finalul fiecărei afișări.



Cod Python

```

n = int(input("n = ")) # citim nr de pagini
pagini = []
for i in range(n):
    x = int(input("pagini["+ str(i+1) +"]="))
    pagini.append(x)
indici = [0] * n      # inițializăm lista de indici
for i in range(n):
    indici[i] = i
# Bubble Sort pentru lista pagini
g = n - 1
sortat = False
while g > 0 and not sortat:
    sortat = True
    for i in range(0, g):
        if pagini[i] > pagini[i + 1]:
            pagini[i], pagini[i + 1] = pagini[i + 1], pagini[i]
            indici[i], indici[i + 1] = indici[i + 1], indici[i]
            sortat = False
    g -= 1

# afișare rezultat
print("Șirul final este:")
for i in range(n):
    print(indici[i] + 1, end=" ")

# end=" " îi transmite lui print:
# nu treci pe linie nouă după o afișare
# pui " " (spațiu) la finalul fiecărei afișări

```

EXEMPLU DE RULARE

```

n = 7
pagini[1]=128
pagini[2]=120
pagini[3]=160
pagini[4]=145
pagini[5]=110
pagini[6]=140
pagini[7]=102
Sirul final este:
7 5 2 1 6 4 3

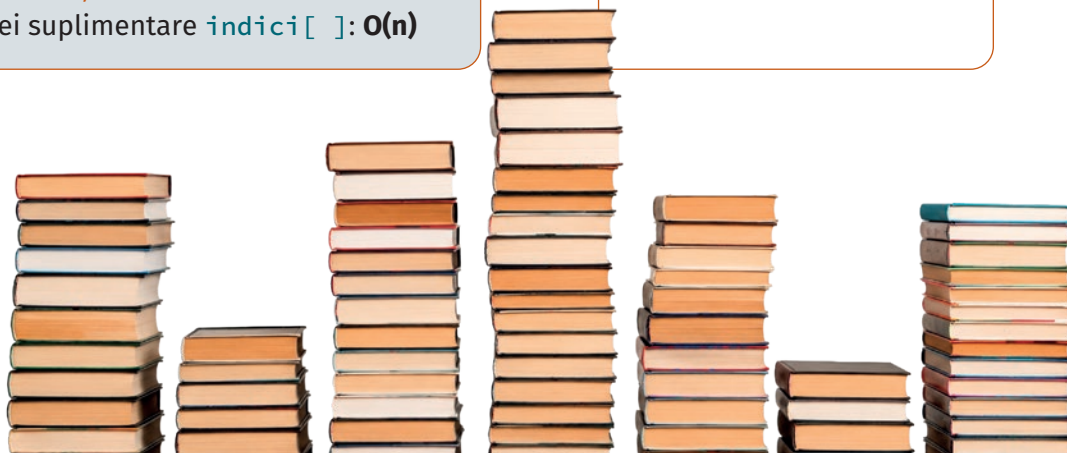
```

A. Complexitatea Timp

- Sortarea listei `pagini[ ]`: $O(n^2)$
- Modificarea listei `indici[ ]` în timpul sortării: $O(n^2)$
 $\Rightarrow O(2 \cdot n^2) \Rightarrow O(n^2)$

B. Complexitatea Spațiu

- Folosirea listei suplimentare `indici[ ]`: $O(n)$



4.4.4. Selection Sort vs. Bubble Sort

A. Comparații vs. Interschimbări (Swaps)

În analiza complexității a fost calculat numărul de pași executați de fiecare algoritm și observăm că operațiile de bază executate sunt comparațiile și interschimbările (swaps).

Metoda \ Operații	Comparații	Interschimbări
	Comparații	Interschimbări
prin Selecție	$O(n^2)$	$O(n)$
prin Bule	$O(n^2)$	$O(n^2)$

De ce contează? Fiecare interschimbare/swap înseamnă trei operații de scriere în memorie. Dacă ai un sistem unde scrierea este lentă (de exemplu, un card de memorie vechi sau o bază de date la distanță), Bubble Sort va fi încet deoarece „mișcă” datele la fiecare pas, în loc să le așeze direct la locul lor final.

Deși ambele metode au o complexitate teoretică similară, comportamentul lor practic este destul de diferit, principalele diferențe fiind cuprinse în tabelul următor.

B. Tabel comparativ

Caracteristică	Selection Sort (Selecție)	Bubble Sort (Bule)
Ideea de bază	Caută cel mai mic element și îl pune la început.	Compară vecinii și îi interschimbă dacă nu sunt în ordine.
Complexitate Timp (Worst)	$O(n^2)$	$O(n^2)$
Complexitate Timp (Best)	$O(n^2)$ (Verifică tot, indiferent de ordine)	$O(n)$ (Dacă lista e sortată)
Nr. interschimbări	$O(n)$	$O(n^2)$
Stabilitate	Instabil (Poate schimba ordinea relativă a elementelor egale).	Stabil (Pentru implementarea cu > se păstrează ordinea elementelor egale)
Direcția sortării	Fixează elementele de la stânga la dreapta (minimele).	Fixează elementele de la dreapta la stânga (maximele).
Utilizare recomandată	Când scrierea în memorie este costisitoare (puține swap-uri).	Când lista este aproape sortată (detectează rapid ordinea).

C. Concluzii

Selection Sort este în general mai rapid decât Bubble Sort pentru cazurile în care operațiile de scriere/citire sunt lente, deoarece face mult mai puține interschimbări/swaps.

Bubble Sort este mai „inteligent” în cazul unei liste „aproape” sortate. Cu un singur flag se oprește imediat, pe când *Selection Sort* va căuta minimul de $n-1$ ori, chiar dacă numerele sunt deja la locul lor.

Stabilitatea. Dacă ai o listă de elevi sortați după nume și vrei să îi sortezi apoi după notă, *Bubble Sort* va păstra ordinea alfabetică pentru elevii cu aceeași notă, dar *Selection Sort* s-ar putea să îi amestece.

4.4.5. Sortare cu liste de frecvență

Sortarea cu liste de frecvență (cunoscută și sub numele de **Counting Sort**) este un algoritm de sortare neconvențional, deoarece nu se bazează pe comparații între elemente (ca Selection Sort sau Bubble Sort), ci pe **numărarea aparițiilor** fiecărei valori.

Este o metodă extrem de eficientă atunci când avem de sortat un **volum uriaș de date** cu valori dintr-un **interval relativ mic** (de exemplu, note de la 1 la 10 sau vârste de la 0 la 100).

Cum funcționează?

Să ne imaginăm că trebuie ordonate punctajele (numere naturale între 0 și 100) pentru 200.000 de elevi: un algoritm de comparare (prin selecție sau metoda bulelor) compară notele între ele de miliarde de ori (200.000^2 , adică 40.000.000.000).

Cu **lista de frecvență**, procesul este mult mai simplu. Deoarece numărul de valori posibile este relativ restrâns – între 0 și 100 –, vom folosi 101 „cutii”

etichetate de la 0 la 100, inițial toate goale, fiecare reținând câte valori egale cu eticheta sa au apărut. Aceste cutii vor fi de fapt elementele unei liste de lungime 101 cu valoare inițială 0

(`frecventa [ ] = [0, 0, ....., 0]`).

Numărarea (scanarea datelor) se va realiza trecând o singură dată prin șirul celor 200.000 de note: la întâlnirea unei note, valoarea cutiei corespunzătoare se mărește cu 1 – ai găsit un 88, aduni 1 la poziția 88; ai găsit un 75, aduni 1 la poziția 75, iar la final te uiți în fiecare cutie și vei ști exact câți de 88 sunt, câți de 75 etc., fără să fi făcut vreo comparație.

Sortarea propriu-zisă se realizează pur și simplu prin scrierea punctajelor în ordine, fiecare fiind afișat de atâtea ori cât este frecvența de apariție: dacă în cutia „75” ai 500, scrii de 500 de ori 75, dacă în cutia „88” ai 1200, scrii 88 de 1200 de ori... și tot așa până la 100.

indici/ note	→	0	1		75		88		95		100
nr. apariții	→	21	112		500		1200		977		710

Cod Python

```
def sort_frecventa(lista):
    n = len(lista)
    frecventa = [0] * 101 # Inițializăm lista de frecvență cu 0
    for i in range(n):
        frecventa[lista[i]] += 1 # Completăm lista
    print("Valorile sortate sunt:")
    # Parcurgem lista de frecvență de la 0 la 100 pentru afișare
    for i in range(101):
        for j in range(frecventa[i]): # Dacă 'i' apare
            print(i, end=" ") # Îl afișăm de frecventa[i] ori
```

Exemplu de rulare

```
date = [75, 88, 67, 90, 75, 64, 88, 75]
sort_frecventa(date)
#Pe ecran se va afișa :
Valorile sortate sunt:
64 67 75 75 75 88 88 90
```

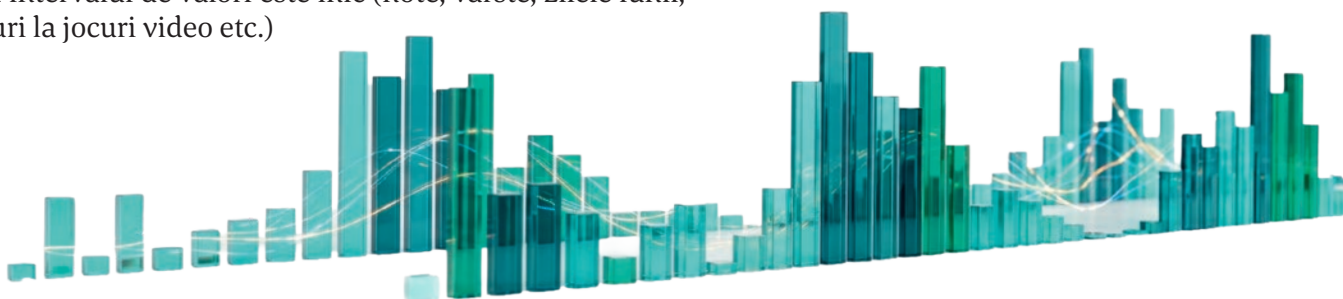
Avantaje și limitări



Avantaje	Limitări
Viteză mare: se execută $(n + k)$ pași (citirea tuturor celor n elemente și apoi parcurgerea celor k căsuțe)	Consum de memorie: Dacă numerele sunt foarte mari (ex: vrei să sortezi CNP-uri), ai avea nevoie de un vector de frecvență uriaș, ceea ce ar bloca memoria.
Simplitate: Codul este foarte scurt și ușor de înțeles.	Doar numere întregi: Nu poți folosi ușor liste de frecvență pentru nume sau cuvinte.

Când se folosește?

- ▶ Când sunt multe elemente (milioane).
- ▶ Când intervalul de valori este mic (note, vârste, zilele lunii, scoruri la jocuri video etc.)



STUDIU DE CAZ 1

Este posibil ca intervalul de valori, deși restrâns, să acopere și valori negative. În acest caz, pentru a ne putea folosi de indicii elementelor unei liste, se va face o translație către valorile naturale prin adunarea unei valori pozitive la fiecare valoare din listă. De exemplu, dacă avem o listă a temperaturilor zilelor unui an calendaristic în care minima a fost de -35°C , iar maxima de 40°C , prin adunarea lui 35 la fiecare valoare vom obține doar temperaturi pozitive între 0 și 75, deci vom putea folosi o listă de frecvență. Trebuie doar să nu uităm să scădem 35 la afișarea fiecărei temperaturi sau la efectuarea unor eventuale calcule (temperaturi medii etc.).

indici	→	0	1	2	3	...	35	36	...	73	74	75
temperaturi	→	-35°	-34°	-33°	-32°	...	0°	1°	...	38°	39°	40°

Activitate în echipă

Să se scrie un program Python care, pe baza temperaturilor înregistrate într-o lună de iarnă, determină frecvențele de apariții ale temperaturilor, temperaturile minimă și, respectiv, maximă înregistrate, precum și temperatura medie a lunii.

Pe ecran se afișează pe primul rând $T_{\min} T_{\max} T_{\text{Med}}$

iar pe fiecare din următoarele linii, crescător după T , o pereche $T F$, unde T este temperatura, iar F este numărul de zile în care s-a înregistrat T .



STUDIU DE CAZ 2

De multe ori, notele rezultate în urma unor examinări sau evaluări au valori numere reale pozitive cu 2 zecimale, mai mici sau egale cu 10. Este evident că nu putem utiliza „cutii” pentru fiecare valoare posibilă, dar **dacă înmulțim cu 100 fiecare notă, vom obține numere naturale** între 0 și 1000; trebuie să nu uităm ca la afișare valoarea să fie împărțită cu 100 și scrisă ca număr real cu două zecimale.

indici	→	100	200		783		850		950		1000
note	→	1	2		7,83		8,50		9,50		10

Activitate în echipă

Să se adapteze programul anterior pentru afișarea notelor (numere reale pozitive cu două zecimale mai mici sau egale cu 10) obținute de cei n candidați în ordine descrescătoare.



FIȘĂ DE LUCRU 10

1. Fie enunțul: „pentru a sorta descrescător prin metoda selecției o listă cu 20 de elemente numere reale, nu sunt necesare mai mult de x determinări ale valorii maxime”. Enunțul este adevărat dacă x este egal cu:

- a) 0 b) 10 c) 19 d) 20 e) 190 f) 400

2. Numărul de interschimbări pe care le efectuează Bubble Sort în cazul sortării descrescătoare a șirului de numere consecutive 1, 2, 3, ..., 8, 9, 10 este:

- a) 0 b) 10 c) 11 d) 45 e) 55 f) 121

3. Care este vectorul de frecvență pentru lista [2, 1, 3, 2, 1, 0, 2]?

4. Având vectorul de frecvență $f = [1, 2, 0, 3]$, care este lista inițială sortată?

5. Scrie în dreptul fiecărei afirmații **A** (dacă este adevărată) sau **F** (dacă este falsă).

- Counting sort: a) este un algoritm eficient pentru valori foarte mari;
b) folosește comparații între elemente;
c) este eficient pentru valori mici;
d) nu funcționează direct pentru valori reale arbitrare.

6. Se dă o listă cu n numere naturale (n este număr natural nenul).

Să se afișeze în ordine crescătoare valorile prime din această listă.

Date de intrare: Programul citește de pe prima linie a fișierului `date.in` numărul n ($n \leq 1000$), iar de pe linia următoare, cele n numere naturale (mai mici decât 1.000.000.000).

Date de ieșire: Programul va scrie în fișierul `date.out` valorile prime din vector, în ordine crescătoare, separate prin exact un spațiu.

Exemplu:

date.in	date.out
7	3 7 11 13
13 1 10 15 3 7 11	



4.5. Recapitulare

V

A

L

U

A

R

E

1. Se dă fragmentul de cod:

```
v = [5, 8, 2]
i = 1
print(v[i+1])
```

Ce valoare va fi afișată?

a) 5

b) 1

c) 8

d) 2

2. Dacă inserăm caracterele ,A', ,B', ,C' în această ordine într-o stivă, în ce ordine vor fi extrase?

a) B, A, C

b) C, B, A

c) A, C, B

d) A, B, C

3. Într-o coadă au fost adăugate, în această ordine, următoarele valori: 3, 10, 2, 8, 6. Care este conținutul cozii dacă s-au efectuat, în ordine, următoarele operații: extragerea unui element, adăugarea valorii 100, extragerea a trei elemente?

4. Numărul de pași pentru ordonarea crescătoare prin metoda bulelor a listei [6, 5, 1, 20, 10] este:

a) 6

b) 5

c) 4

d) 3

e) 2

f) 1

5. După câte parcurgeri ale șirului [2, 40, 17, 1, 51, 34, 20, 63], valoarea 20 ajunge pe locul final în lista sortată crescător prin metoda selecție minimului?

a) 0

b) 1

c) 2

d) 3

e) 4

f) 5

6. Se dă un șir de n (n număr natural) paranteze rotunde. Să se scrie un program Python care verifică dacă șirul este corect parantezat.

7. Să se transforme numărul natural n din baza 10 în baza 2 folosind stiva.

8. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se afișeze elementele pozitive din listă.

9. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se înlocuiască fiecare element cu pătratul său.

10. Se citesc de la tastatură un număr natural n și o listă de n numere întregi. Să se determine suma elementelor prime din listă.

11. Se consideră șirul definit prin $x_0=1$ și relația $x_n=3^n$, n natural strict pozitiv. Să se elaboreze un program Python eficient care afișează primii n termeni ai șirului.

12. Stiva P conține valorile distincte de la 1 la n (n natural nenul) ce trebuie transferate în ordine crescătoare într-o coadă C . Având la dispoziție și o stivă S , să se scrie un program Python care să realizeze, dacă este posibil, transferul elementelor printr-o singură parcurgere a listei P , sau să afișeze "Imposibil" dacă nu se poate.

Exemplu: Pentru $P=[2, 1, 3, 4]$, succesiunea de mutări este: $P \rightarrow S$; $P \rightarrow S$; $P \rightarrow C$; $P \rightarrow C$; $S \rightarrow C$; $S \rightarrow C$.



MINIPROIECTE – ACTIVITATE ÎN ECHIPĂ DE 2 ELEVİ

1. Sortarea urgențelor în UPU

La intrarea în Unitățile de Primiri Urgențe (UPU) pacientul este evaluat și primește un cod ce indică nivelul de urgență și timpul maxim de așteptare până când va fi preluat de un medic.

Codurile de urgență în UPU

Cod Culoare	Nivel Urgență	Semnificație	Timp de așteptare
ROȘU	1 – Resuscitare	Funcții vitale compromise	0 min.
GALBEN	2 – Critic	Risc de afectare a unei funcții vitale	15 min.
VERDE	3 – Urgent	Funcții vitale stabile, necesită investigații	60 min.
ALBASTRU	4 – Non-urgent	Afecțiuni care nu pun viața în pericol	120 min.
ALB	5 – Consult	Cazuri rezolvabile la medicul de familie	240 min.

Cerință: În triajul UPU așteaptă n pacienți pentru care s-au stabilit codurile de urgență (numere de la 1 la 5). Să se afișeze ordinea în care vor fi preluați.

Date de intrare: Din fișierul `triaj.in` citim n – numărul de pacienți, iar de pe următoarea linie citim n numere naturale cu valori între 1 și 5, reprezentând nivelul urgențelor celor n bolnavi.

Date de ieșire: Soluția va fi dată în fișierul `triaj.out` de indicii pacienților în ordinea preluării lor.

Exemplu:

triaj.in	triaj.out	Explicații
5 4 2 5 3 4	2 4 1 5 3	$n=5$ Indexarea în sala de așteptare este de la 1. Ordinea preluării pacienților va fi dată de ordinea crescătoare a codului de urgență. Pentru același cod se alege pacientul cu indice mai mic.

2. Construcție geometrică

Desenează un dreptunghi după următoarele reguli:

La pasul 0 avem un dreptunghi cu laturile de 1 și, respectiv, 2 cm. La fiecare pas, pe latura mai lungă a dreptunghiului deja construit, se „lipește” un pătrat cu latura egală cu latura dreptunghiului, plasările pătratelor făcându-se în același sens (trigonometric, sau în sensul acelor de ceasornic). Pentru dreptunghiul obținut după n pași, să se determine:

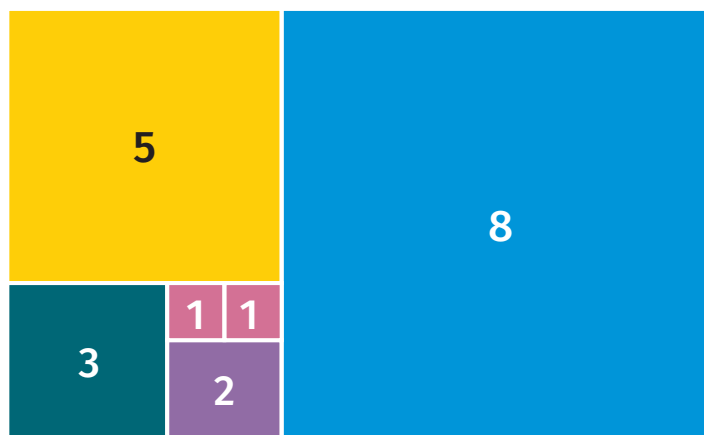
- aria;
- laturile;
- raportul laturilor.
- Se va realiza reprezentarea grafică a dreptunghiurilor folosind biblioteca Tkinter.

Exemplu de construcție în sens orar, cu dreptunghiul inițial „pe orizontală”:

- se începe cu dreptunghiul de 2×1 ;
- se adaugă un pătrat de 2×2 sub el;
- apoi unul de 3×3 în lateral stânga;
- apoi 5×5 deasupra, 8×8 în dreapta etc.;



– de fiecare dată, noul pătrat se lipește pe latura cea mai lungă a figurii deja construite, în sens orar.



3. Gestionarea unei liste de sarcini folosind funcții și meniu interactiv.

Se dorește realizarea unei aplicații în Python pentru administrarea unei liste de sarcini zilnice, utilizând **subprograme (funcții)** și un **meniu interactiv**.

Programul trebuie să permită:

1. adăugarea unei sarcini în listă;
2. eliminarea unei sarcini finalizate (după poziție);
3. afișarea listei de sarcini;
4. sortarea sarcinilor în ordine alfabetică;
5. inversarea ordinii sarcinilor;
6. realizarea unei copii de siguranță a listei;
7. golirea completă a listei.

Lista de sarcini trebuie afișată după fiecare operație.

Programul rulează până când utilizatorul alege ieșirea din meniu.

Indicații de rezolvare:

1. Gândește programul modular, fiecare operație din meniu trebuie să fie implementată într-o funcție separată.
2. Folosește o listă pentru a stoca sarcinile, lista va fi transmisă ca parametru funcțiilor.

Important! Când transmitem o listă unei funcții, funcția nu primește o copie, ci lista originală; de aceea, dacă modificăm lista în funcție, modificarea apare peste tot.

3. Meniul aplicației va fi implementat folosind un ciclu `while True`; se va citi opțiunea utilizatorului cu `input()` și se va folosi `if/elif` pentru a selecta operația.

FIȘĂ DE OBSERVARE A COMPORTAMENTULUI ȘI A ACTIVITĂȚII ELEVULUI

1. Întotdeauna

2. Deseori

3. Uneori

4. Niciodată

Criterii evaluate	Elevul se autoevaluează	Cadrul didactic evaluează
Am ascultat cu atenție explicațiile primite.		
Am participat activ la discuțiile/dezbaterile din clasă.		
Am adresat întrebări de clarificare.		
Am finalizat sarcinile de lucru individuale.		
Mi-am adus contribuția la activitățile de grup.		
Am împărtășit celorlalți colegi rezultatele/achizițiile mele.		

R E C A P I T U L A R E

4.6. Evaluare sumativă

I. Rezolvă.

1. Să se stabilească valoarea de adevăr a următoarelor enunțuri:
- a. O listă în Python poate conține elemente de tipuri diferite.
 - b. Metoda `append()` adaugă un element la începutul listei.
 - c. Operatorul `in` verifică dacă un element aparține unei liste.
 - d. Operația `lista1 + lista2` realizează concatenarea a două liste.
 - e. Metoda `pop()` elimină și returnează un element din listă.

5x0,30 p. = 1,50 p.

2. Se consideră lista: `L = [3, 7, 2, 9]`. Care este rezultatul execuției următoarelor instrucțiuni?

0,5 p.

```
L.append(5)
print(len(L))
```

a) 4

b) 5

c) [3, 7, 2, 9, 5]

d) Eroare de execuție

3. Se consideră secvența de program de mai jos:

2x0,5 p. = 1 p.

```
numere = [5, 2, 8, 1, 9]
minim = 0
for x in numere:
    if x < minim:
        minim = x
print("Minimul este:", minim)
```

- a. Să se identifice eroarea logică din program.
- b. Să se scrie varianta corectă a programului.

4. Se citesc de la tastatură un număr natural `n` și o listă de `n` numere întregi. Să se determine valoarea numărului care apare de cele mai multe ori în listă.

1 p.

5. Se consideră șirul definit prin $x_0=2$ și relația de recurență $x_n = x_{n-1} + 2^n$, $n > 0$. Să se scrie un program Python eficient care afișează primii `n` termeni ai șirului.

1 p.

- II. Miniproiect. La alegere, unul din cele trei miniproiecte rezolvate în cadrul secțiunii de *Recapitulare*.

2 p.

- III. Portofoliu. La finalul acestei unități, portofoliul digital al elevului cuprinde, în dosarul *Unitate4_NumeElev*, următoarele rezolvări: *Fișa de lucru 9* și *Fișa de lucru 10*.

1x2 p. = 2 p.

Nota finală a elevului se obține cumulând punctajele din *Rezolvă*, *Miniproiect* și *Portofoliu*. Se acordă 1 punct din oficiu.



Recapitulare finală



- Realizează o analiză comparativă între cele două structuri de date studiate din Python: `liste` și `fișiere text`. Exemple de criterii de analiză: tip structură, locație hardware utilizată pentru memorare, persistența datelor, accesul la date, tipul datelor stocate.
- Fișierul `date.txt` conține cel puțin trei și cel mult 1 000 000 de numere naturale alcătuite din cel mult 9 cifre fiecare. Numerele în fișier sunt scrise câte unul pe fiecare rând. Se cere să se afișeze, în fișierul `rezultate.txt`, separate printr-un spațiu, două numere distincte, anume cel mai mic număr impar cu două cifre și cel mai mare număr impar cu două cifre care **nu** fac parte din șir. Dacă nu există două astfel de valori, în fișierul `rezultate.txt` se afișează mesajul **Nu există**.
- Un angajat primește un bonus astfel:
 - dacă vechimea este mai mică de 5 ani, nu primește bonus;
 - dacă vechimea este între 5 și 10 ani inclusiv, primește 5% din salariu;
 - dacă vechimea este peste 10 ani, primește 10% din salariu.
 Definește o funcție care primește parametrii: salariul (număr real) și vechimea (număr natural) unui angajat și returnează valoarea bonusului primit. Scrie programul principal care citește de la tastatură datele a `n` angajați. Scrie, în fișierul text `bonus.txt`, pe prima linie și despărțite de un spațiu, bonusurile calculate pentru cei `n` angajați.
- Proiectează o aplicație modulară Python cu cel puțin trei subprograme, care rezolvă următoarea problemă:
 - a. se citesc de la tastatură: un număr natural `b` ($2 \leq b \leq 9$, reprezintă o bază de numerație), un număr natural `c` (reprezintă o cifră $0 \leq c < b$) și o listă având `n` elemente numere naturale distincte scrise în baza 10;
 - b. se afișează pe ecran numărul din listă care poate fi considerat a fi scris în baza `b` și care are cele mai multe cifre egale cu `c`. Dacă există mai multe valori în listă care îndeplinesc condiția din problemă, se va afișa primul dintre ele, iar dacă în listă nu există nicio valoare care îndeplinește condiția, se va afișa mesajul **Nu există**.**Exemplu:** se citesc `b=3`, `c=1`, `n=6` și lista `[12, 420, 110, 211, 230, 1161]`, atunci se afișează 110.
- Se citesc de la tastatură două numere naturale nenule `n` și `m`, și trei liste de numere întregi: `x` și `y`, fiecare cu câte `n` elemente și `z` cu `m` elemente. Să se verifice dacă lista `z` conține toate sumele `x[i]+y[i]`, pentru fiecare `i` de la 0 la `n-1`.

PROIECT 1 – activitate în echipă de 2 elevi

Un profesor păstrează într-un fișier text numit `numere.txt` o listă de numere naturale nenule, câte un număr pe fiecare linie. Profesorul dorește să prelucreze aceste numere. Scrie un program Python organizat modular care:

- Citește numerele din fișierul `numere.txt` și le memorează într-o listă numere.
- Definește următoarele subprograme (funcții):
 - `prim(n)` – returnează `True` dacă `n` este prim, `False` altfel;
 - `suma_cifre_pare(n)` – returnează suma cifrelor pare ale lui `n`;
 - `oglindit(n)` – returnează numărul obținut prin inversarea cifrelor lui `n`;
 - `cmmdc(a, b)` – returnează cel mai mare divizor comun, folosind Algoritmul lui Euclid cu împărțiri repetate;
 - `in_baza(n, b)` – returnează reprezentarea lui `n` în baza `b` (sub formă de număr ale cărui cifre sunt cifrele reprezentării, ca în lecția 1.2.4).

3. **Construiește** o listă `noua_lista`, în care, pentru fiecare număr `n` din fișierul `numere.txt`, se adaugă:
 - `oglindit(n)`, dacă `n` este prim, în caz contrar se adaugă `suma_cifre_pare(n)`.
4. **Calculează** CMMDC al tuturor valorilor **nenule** din `noua_lista`.
5. **Scrie** în fișierul `rezultate.txt`, pe câte o linie:
 - elementele listei `noua_lista`;
 - valoarea CMMDC calculat la punctul 4;
 - reprezentarea CMMDC în baza 2

Exemplu:

<code>numere.txt</code>	<code>rezultate.txt</code>
23	[32, 12, 6]
48	2
56	10

Barem de evaluare propus:

- » Citire din fișier/scriere în fișier conform cerințelor 2 p.
- » Corectitudinea funcțiilor de calcul..... 5 p.
- » Construirea listelor.....1 p.
- » Determinare CMMDC1 p.
- » Din oficiu1 p.

PROIECT 2 – activitate în echipă de 2 elevi

Gigel și Ionel vopsesc câteva dintre cele `n` uluci din gardul bunicului, numerotate de la stânga spre dreapta începând cu 1. Gigel pleacă din stânga și vopsește cu roșu din `g` în `g` uluci, începând cu prima, iar Ionel pleacă din dreapta și vopsește cu albastru, începând cu ultima, din `i` în `i` uluci. Câte și care scânduri vor fi roșii, albastre și respectiv mov (vopsite și cu roșu și cu albastru)?

Date de intrare – din fișierul `date.txt` se citesc trei numere naturale:

`n, g, i` (aflate pe câte un rând al fișierului), unde:

- `n` = numărul total de uluci ale gardului;
- `g` = pasul cu care Gigel vopsește din stânga;
- `i` = pasul cu care Ionel vopsește din dreapta.

Gigel vopsește cu roșu ulucile aflate la pozițiile: 1, 1+`g`, 1+2`g`, ...

Ionel vopsește cu albastru, pornind din dreapta, ulucile aflate la pozițiile: `n`, `n-i`, `n-2i`, ...

O ulucă vopsită de amândoi va fi mov.

Date de ieșire – în fișierul `rezultate.txt` se vor scrie

- `r` = numărul de uluci doar roșii și lista acestora în ordine crescătoare;
- `a` = numărul de uluci doar albastre și lista acestora în ordine crescătoare;
- `m` = numărul de uluci mov și lista acestora în ordine crescătoare;

Creează o interfață grafică pentru ilustrarea soluției, folosind biblioteca `Tkinter`, în care reprezinți: ulucile nevopsite în culoare gri, ulucile roșii, ulucile albastre și ulucile mov (vopsite de ambii copii).

Exemplu:

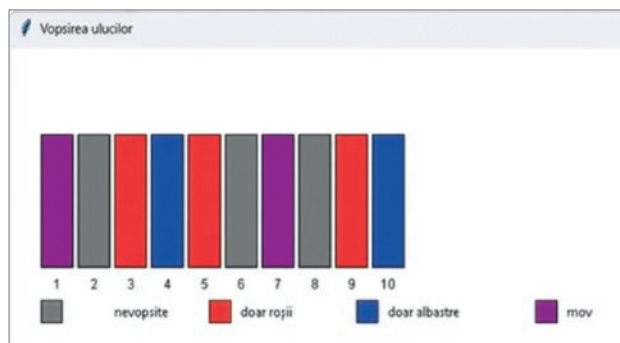
Pentru:

- `n` = 10;
- `g` = 2;
- `i` = 3.

Doar roșii: `r` = 3; [3, 5, 9]

Doar albastre: `a` = 2; [4, 10]

Mov: `m` = 2; [1, 7]



Barem de evaluare propus:

» Citirea datelor	0,5 p.
» Determinarea ulucilor roșii	2 p.
» Determinarea ulucilor albastre	2 p.
» Identificarea ulucilor mov.....	1 p.
» Realizarea desenului în Tkinter.....	3 p.
» Claritatea codului și folosirea unor nume sugestive.....	0,5 p.
» Din oficiu	1 p.

Evaluare finală

- Care este rezultatul apelului unei funcții care nu conține nicio instrucțiune `return` sau conține doar `return` fără a indica o expresie? 0,5 p.
 - Returnează șirul de caractere vid.
 - Produce eroare la execuția funcției.
 - Returnează valoarea specială `None`.
 - Returnează valoarea `0`.
- Care instrucțiune adaugă elementul `10` la sfârșitul listei `L`? 0,5 p.

a) `L.add(10)` b) `L.insert(10)` c) `L.append(10)` d) `append(L,10)`
- Completează corect spațiile libere. 2 x 0,5 p. = 1 p.
 - Cele cinci etape ale elaborării unui program sunt: analiza problemei,, implementarea, testarea și
 - Eroarea care apare la rulare când programul execută o operație invalidă (de exemplu, împărțirea la zero) se numește eroare de
- Asociază fiecărei secvențe de cod problema pe care o rezolvă cu structuri de date de tip listă: 4 x 0,5 p. = 2 p.

1)	<pre>for i in range(n - 1): for j in range(i + 1, n): if a[i]<a[j]: x = a[i] a[i] = a[j] a[j] = x</pre>	A. Verifică dacă lista <code>a</code> are toate elementele egale.
2)	<pre>m = a[0] for i in range(1, n): if a[i] < m: m = a[i]</pre>	B. Determină maximul din lista <code>a</code> .
3)	<pre>m = a[0] for i in range(1, n): if a[i] > m: m = a[i]</pre>	C. Sortare descrescătoare a listei <code>a</code> .
4)	<pre>ok = 0 for i in range(1, n): if a[i] != a[0]: ok = 1</pre>	D. Determină minimul din lista <code>a</code> .

5. Asociază fiecare expresie din coloana A cu rezultatul acesteia din coloana B.

4 x 0,25 p. = 1 p.

Coloana A (expresie Python)	Coloana B (rezultat)
1) <code>25 // 4 + 25 % 4</code>	A) 8
2) <code>(15 - 3) * 2 ** 2 // 6</code>	B) True
3) <code>not (5 > 3) or (4 == 4)</code>	C) 2
4) <code>100%9+ 100 // 9 % 5</code>	D) 7

6. Stabilește valoarea de adevăr a fiecărei propoziții (**Adevărat** sau **Fals**).

4 x 0,25 p. = 1 p.

- Un algoritm trebuie să fie finit, corect, general și complet.
- Operatorul „=” în Python exprimă egalitatea matematică.
- Rezultatul expresiei `math.floor(-2.5)` este -3.
- Pentru a scrie valoarea întreagă 100 într-un fișier text este necesar să utilizăm funcția de conversie `str(100)`.

7. Se citește de la tastatură un număr natural `n` alcătuit din cel puțin două cifre.

Formează o listă cu toate numerele obținute din `n` eliminând, pe rând, exact o cifră, astfel încât cifrele rămase să nu își modifice ordinea. Scrie un program în Python care afișează pe ecran cea mai mare și cea mai mică valoare din lista obținută.

Pentru rezolvarea problemei se vor utiliza cel puțin trei funcții definite de utilizator.

3 p.

Exemplu: Dacă se citește `n=3849`, se obține lista `[384, 389, 349, 849]`; se afișează pe ecran 849 și 349.

Din oficiu:.....1 p.

Total punctaj:..... 10 p.

Soluții (selectiv)

CAPITOLUL 1

Fișa de lucru 1:

1. a) Variabila de control este `x`. Condiția de oprire a buclei este `x > 5` (bucula se execută cât timp `x <= 5`).

b) Tabel de urmărire pentru primele trei iterații:

Iterație	x (la intrare)	suma după suma+x	x par?	suma după if	x (la final)
1	1	1	nu	1	2
2	2	3	da	5	3
3	3	8	nu	8	4

c) Valoarea finală a variabilei `suma` este **19**.

d) Instrucțiunea `if x % 2 == 0` verifică dacă valoarea curentă a lui `x` este pară. Dacă da, se adaugă un bonus de 2 la `suma`. Astfel, numerele pare contribuie cu o valoare mai mare decât cele impare.

Recapitulare: 1. F, F, A, A, A, F, A, F, F, F. 2. Programul calculează suma cifrelor pare ale numărului. Pentru `nr = 1234`, cifrele pare sunt 4 și 2, deci `suma = 4 + 2 = 6`. 3. Răspuns corect: a)

Evaluare sumativă:

1. Pseudocod - O formă intermediară de reprezentare a unui algoritm, între limbajul natural și un limbaj de programare.

Constantă - O valoare care nu se modifică pe parcursul execuției programului.

Expresie logică - O combinație de valori și operatori care produce un rezultat de tip True sau False.

Proiectare modulară - Împărțirea unui algoritm în subprobleme independente, fiecare cu un rol clar.

Complexitate $O(n)$ - Numărul de operații crește proporțional cu dimensiunea datelor de intrare.

2. a) 5.0, float; b) 3, int; c) 2, int; d) True, bool; e) False, bool;

5. Se afișează 8. Rolul variabilei `ok`: este o variabilă de tip flag (marcaj logic) care alternează între `True` și `False` la fiecare valoare impară a lui `x`. Ea controlează dacă, în iterația curentă, se mai adaugă un bonus de 1 la `y`. Practic, `ok` funcționează ca un „comutator” care activează/dezactivează adăugarea bonusului în funcție de paritatea numerelor impare întâlnite anterior.

CAPITOLUL 2

Fișa de lucru 3:

2. a) `c = y%10, return 0, x = x//10, y = y//10, return 1, cifre_e_distincte()==1, print("Adevărat"), print("Fals")`

Evaluare sumativă: 1. b; 2. a) A, b) F, c) A, d) F; 3. b; 4. 9 3; 6. a) `Clasa1()`, `a1.x`; 6. b) `global x`

CAPITOLUL 3

Autoevaluare: `f.read(), n+1, n%i==0, str(i)+" ", g.close()`

Evaluare sumativă: 1. a) A, b) F, c) F, d) F, e) A; 2. a; 3. a) buclă infinită; 4. c

CAPITOLUL 4

Fișa de lucru 9

4. Metoda 1 — Generare completă și afișare inversă: Generăm termenii până la F_n ; îi memorăm într-o listă; Afișăm lista invers. Complexitate timp: $O(n)$ și spațiu $O(n)$ deoarece se memorează toți termenii. Metoda 2 — Reconstituire inversă: Dacă se cunosc doi termeni consecutivi, se pot genera cei anteriori. Complexitate timp: $O(n)$ și spațiu: $O(1)$ - deoarece se folosesc doar câteva variabile.

Fișa de lucru 10: 1. c 2. d 3. [1, 2, 3, 1] 4. [0, 1, 1, 3, 3, 3] 5. a) F ; b) F ; c) A ; d) A ;

Recapitulare: 1. D 2. B 3. 6, 100 4. D 5. E

Evaluare sumativă: 1. a. A b. F c. A d. A e. A 2. b 3. a) Eroarea este inițializarea greșită a variabilei minim

EVALUARE FINALĂ:

1. c); 2. c); 3. a) proiectarea, depanarea b) execuție; 4. 1-C, 2-D, 3-B, 4-A; 5. 1-D, 2-A, 3-B, 4-C;
6. a) A, b) F, c) A, d) A

NOTĂ:

În limbajul de programare Python, se recomandă evitarea utilizării caracterelor cu diacritice în denumirile variabilelor, pentru a menține codul clar, lizibil și ușor de manipulat. În schimb, în mesajele afișate utilizatorului, folosirea diacriticelor este permisă, dar nu obligatorie; în consecință, manualul prezintă exemple corespunzătoare atât cu, cât și fără diacritice.

Bibliografie

Tutorial Python în limba română:

<https://docs.python.org/ro/dev/tutorial/index.html>

Funcții predefinite *built-in*:

<https://docs.python.org/ro/dev/library/functions.html>

Modulul predefinit math:

<https://docs.python.org/ro/dev/library/math.html>

Biblioteca Tkinter:

<https://docs.python.org/3/library/tkinter.html>

Vizualizarea pas cu pas a codului Python și a fluxului de execuție:

<https://pythontutor.com/visualize.html#mode=edit>

Link descărcare PyCharm:

<https://www.jetbrains.com/pycharm/download/>

CEZARA BENEGUI · ANA-CRISTINA IOVA
RODICA SMÎNTÎNĂ · ANDREI PĂUN
CIPRIAN MIHAI CEAUȘESCU
PAUL ENACHE

INFORMATICĂ

Curriculum de specialitate (CS), filiera teoretică,
profilul real, specializarea matematică-informatică – clasa a IX-a